

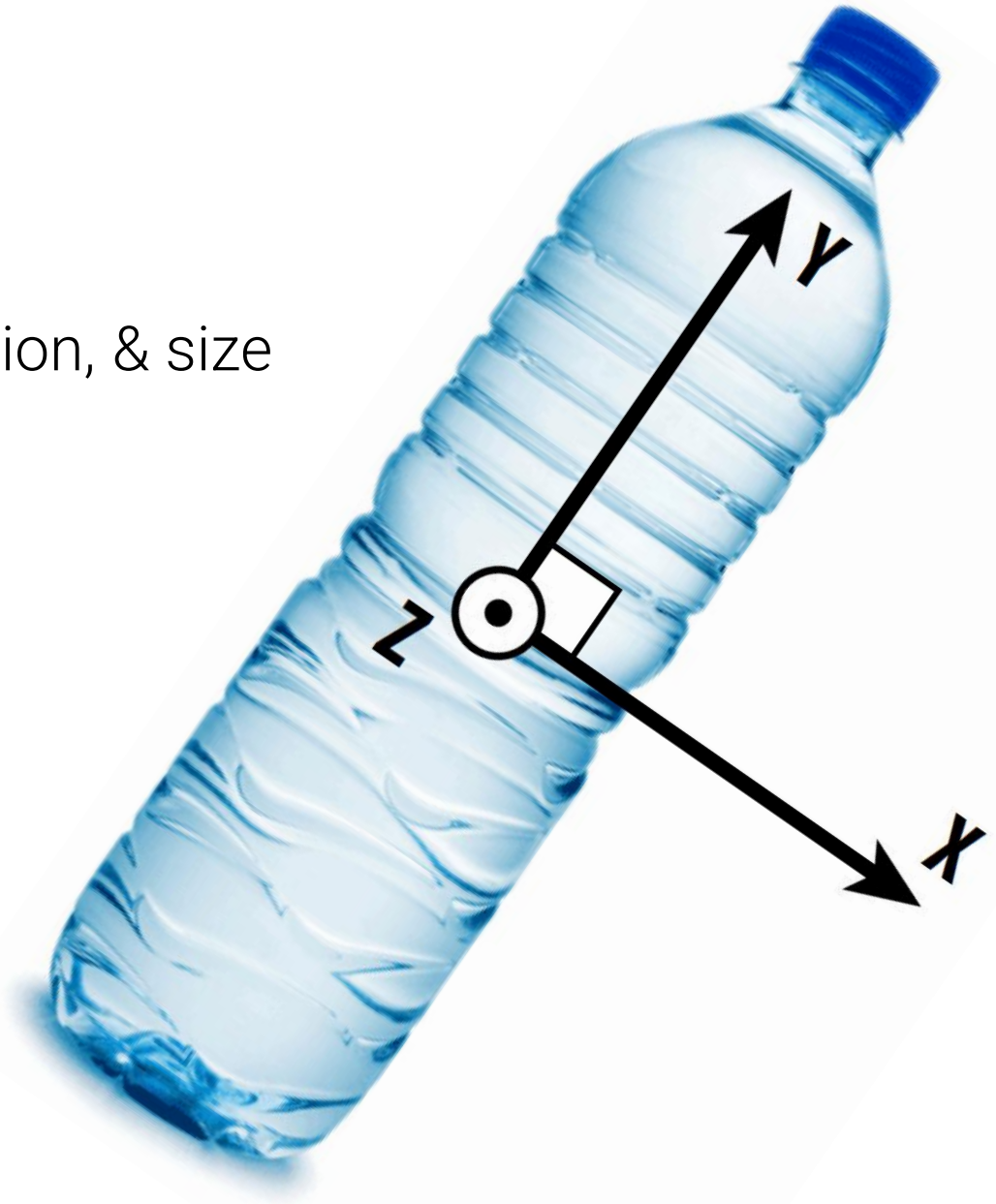
RTR – Transformations

David Schedl

University of Applied Sciences Upper Austria

Transformations

- Why?
  - For changing objects' position, orientation, & size
  - ➔ Translation, Rotation, & Scaling
- Where?
  - WebGL: in the vertex shader
- Objects in computer graphics:
  - Meshes (triangles)
  - Cameras
  - Lights, …



Motivation: Animation & Movement

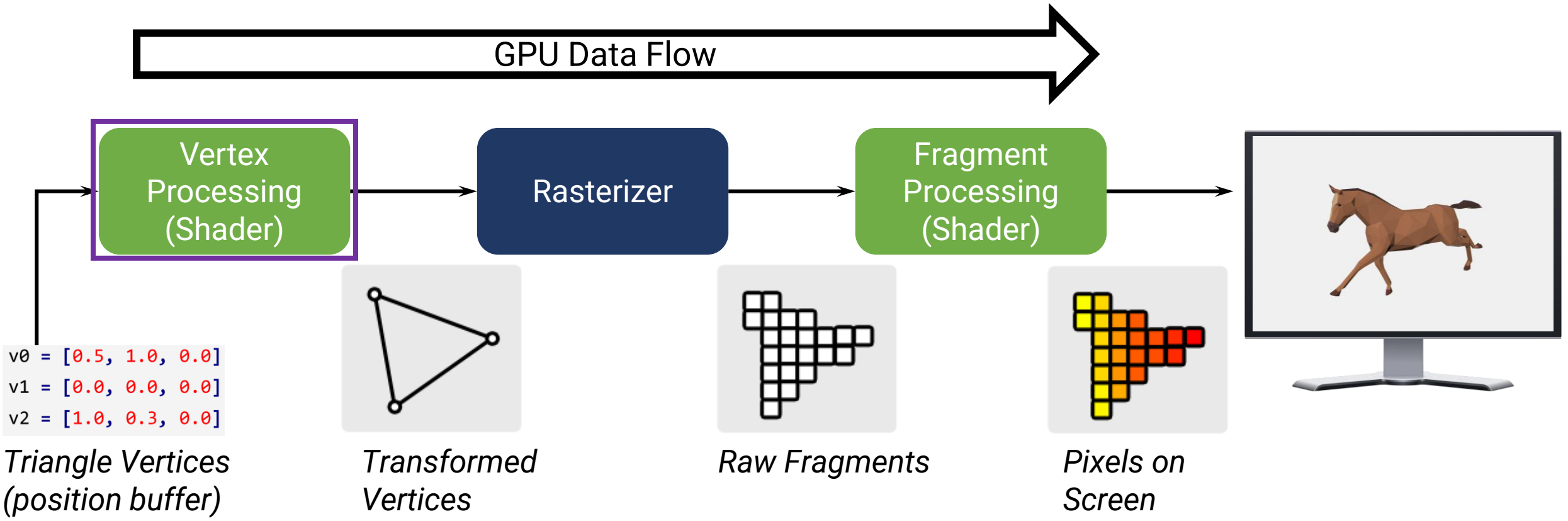


https://threejs.org/examples/webgl_animation_keyframes.html

Motivation: Animation & Movement



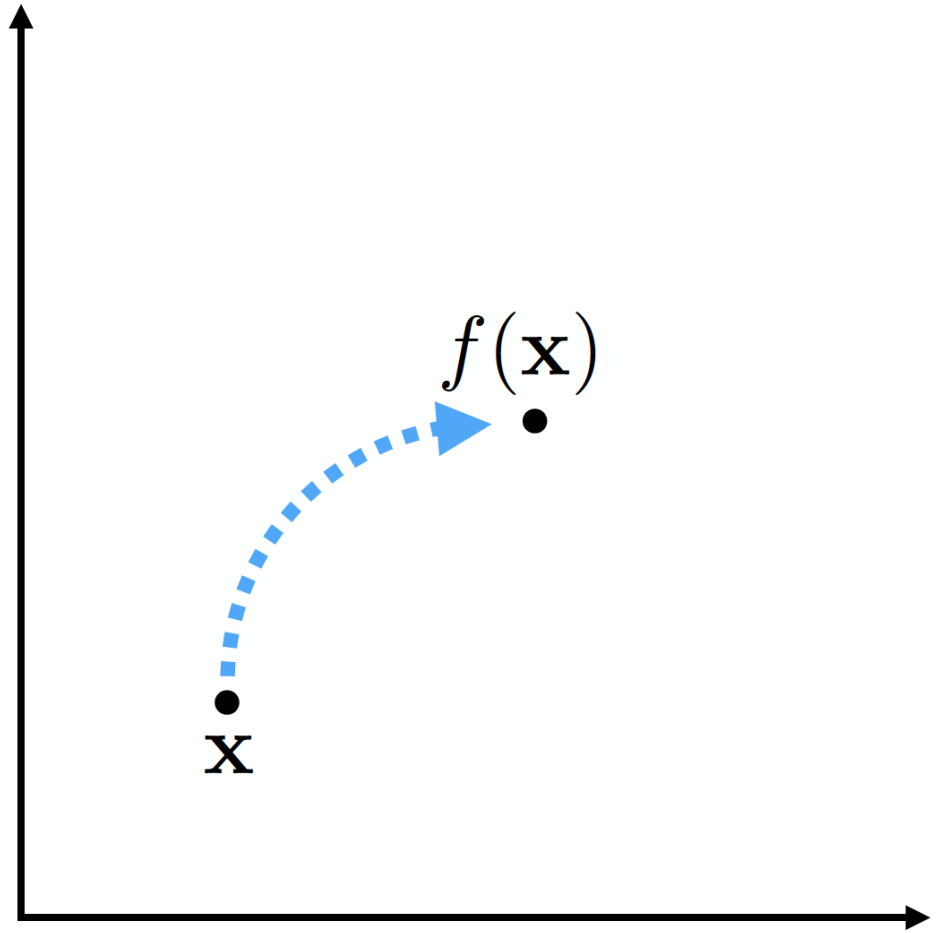
Programmable Pipeline



Transformations

functions

Basic idea: f transforms x to $f(x)$



Linear Transformations

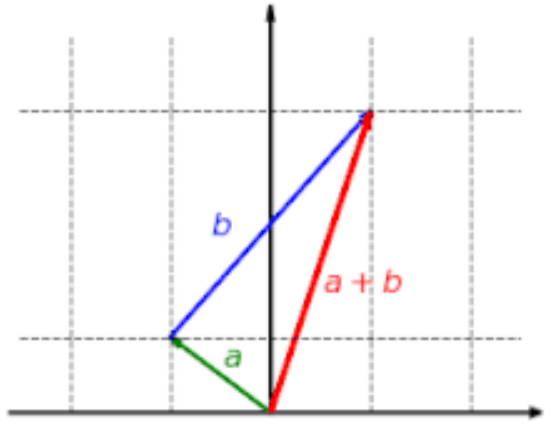


https://youtu.be/kYB8IZa5AuE

What can we do with linear transformations?

- What does linear mean?

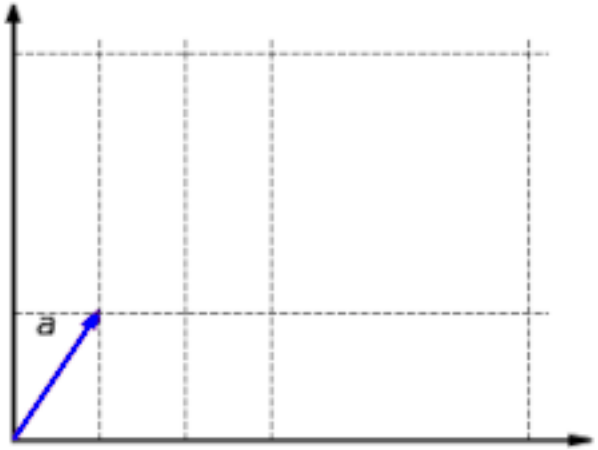
$$f(x + y) = f(x) + f(y)$$



What can we do with linear transformations?

- What does linear mean?

$$f(x+y) = f(x) + f(y)$$
$$f(ax) = af(x)$$



What can we do with linear transformations?

- What does linear mean?

$$f(x+y) = f(x) + f(y)$$
$$f(ax) = af(x)$$

- Cheap to compute
- Composition of linear transformations is linear
- Uniform representation of transformations

Representing transformations

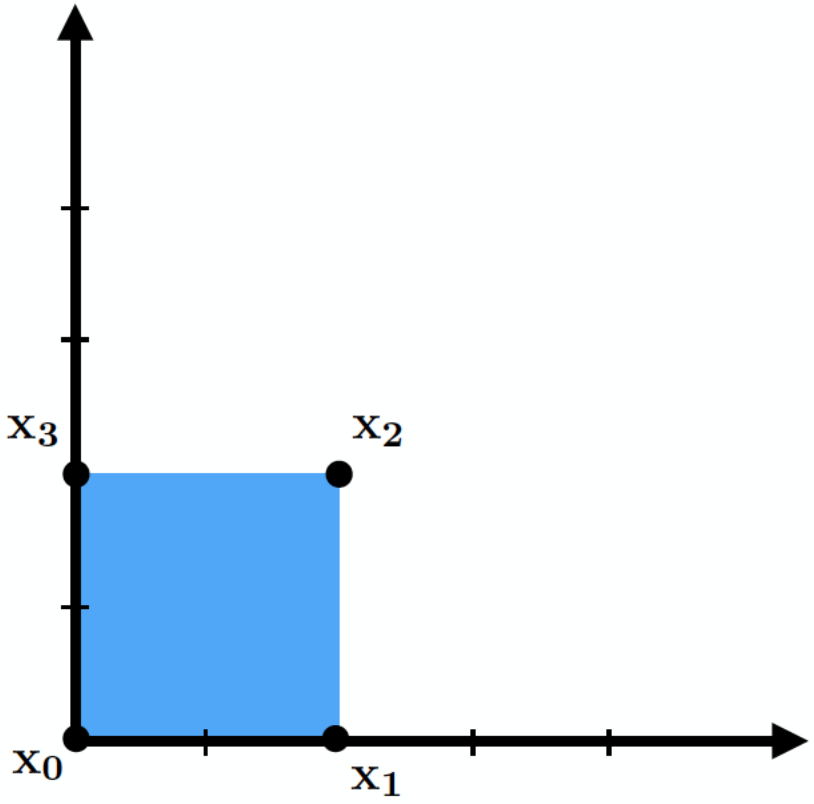
Recap: 2D matrix multiplication

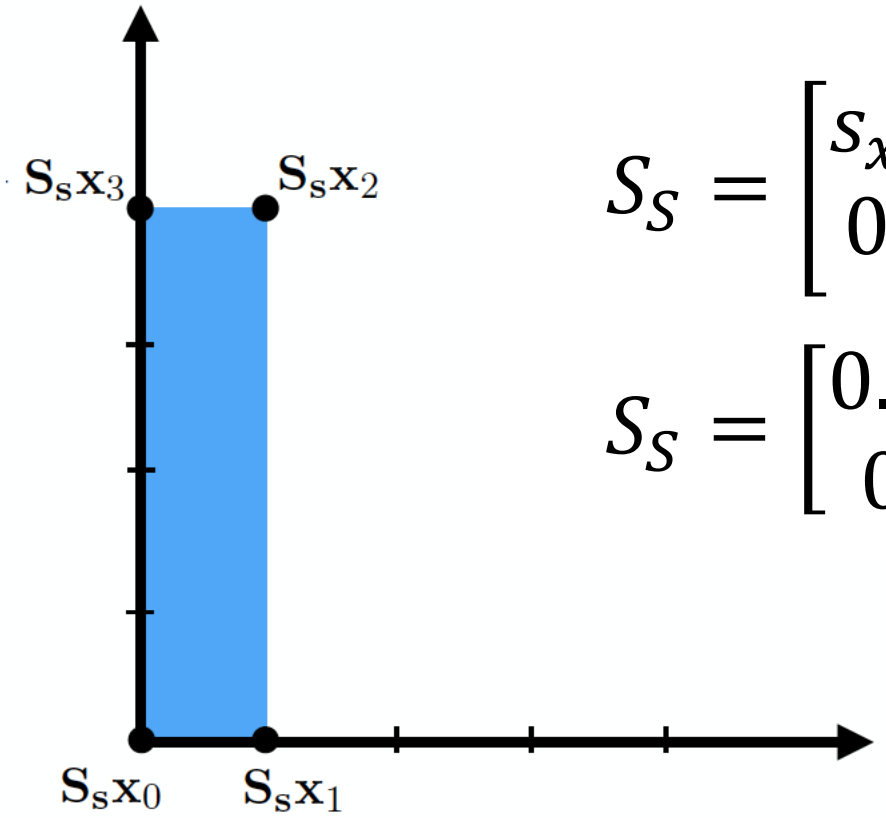
- Matrix multiplication is linear combination of columns
- Encodes a linear map!

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} a \\ b \end{bmatrix} + y \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} ax + & cy \\ bx + & dy \end{bmatrix}$$

Transformations

Non-uniform scale matrix (2d)



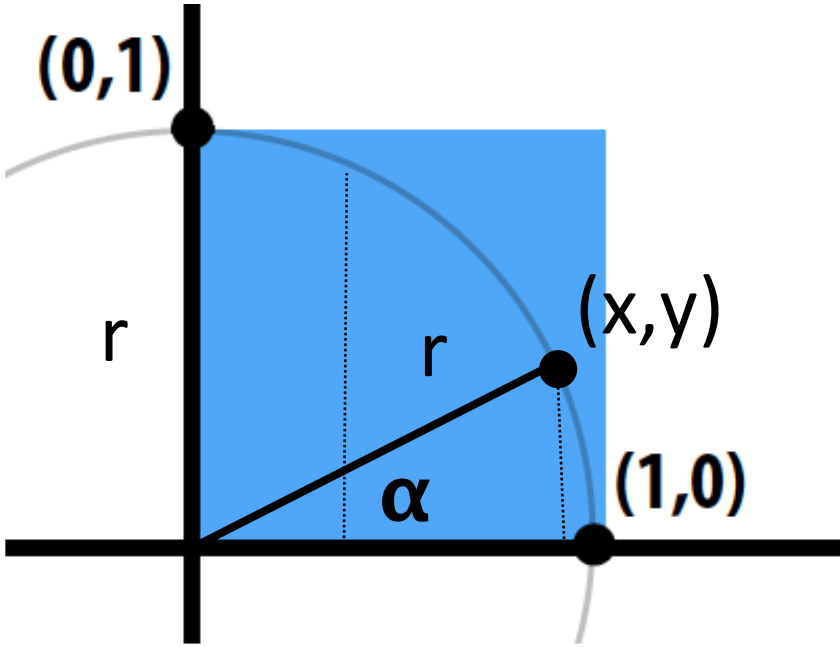


$$S_S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

$$S_S = \begin{bmatrix} 0.5 & 0 \\ 0 & 2 \end{bmatrix}$$

Rotation matrix (2d)

$$x = r \cdot \cos(\alpha)$$
$$y = r \cdot \sin(\alpha)$$

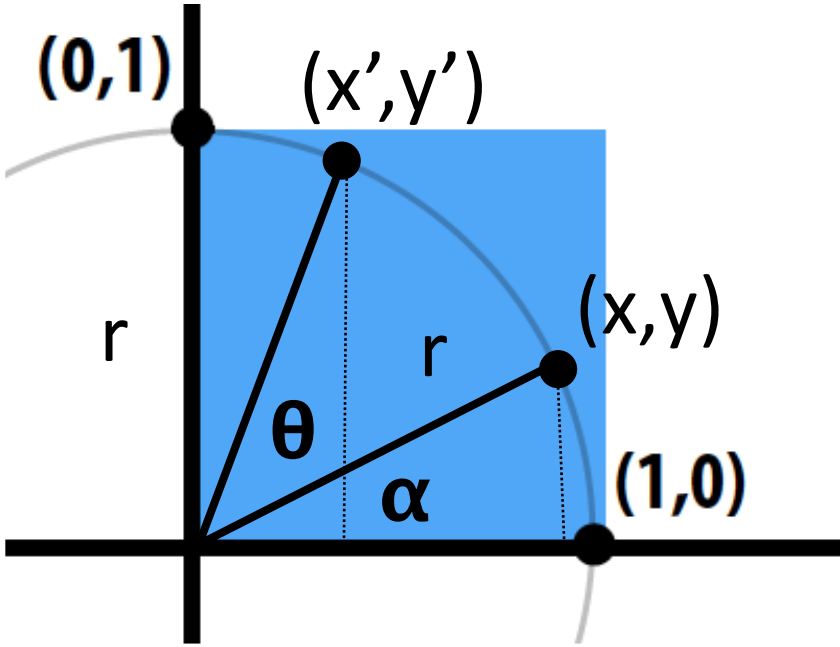


Rotation matrix (2d)

$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

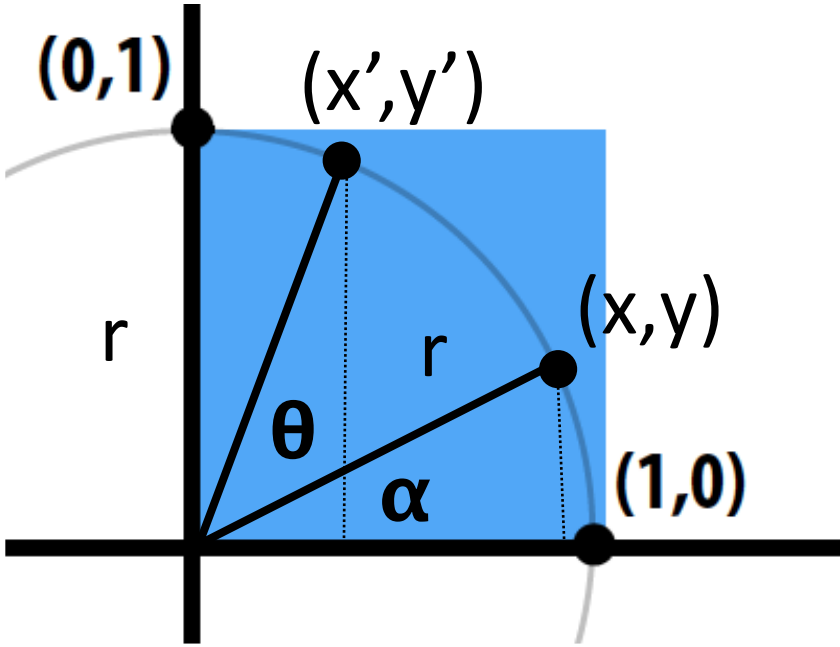
$$x' = r \cdot \cos(\alpha + \theta)$$



Rotation matrix (2d)

$$x = r \cdot \cos(\alpha)$$
$$y = r \cdot \sin(\alpha)$$

$$x' = r \cdot \cos(\alpha + \theta)$$
$$= r \cdot \cos(\alpha) \cdot \cos(\theta) - r \cdot \sin(\alpha) \cdot \sin(\theta)$$

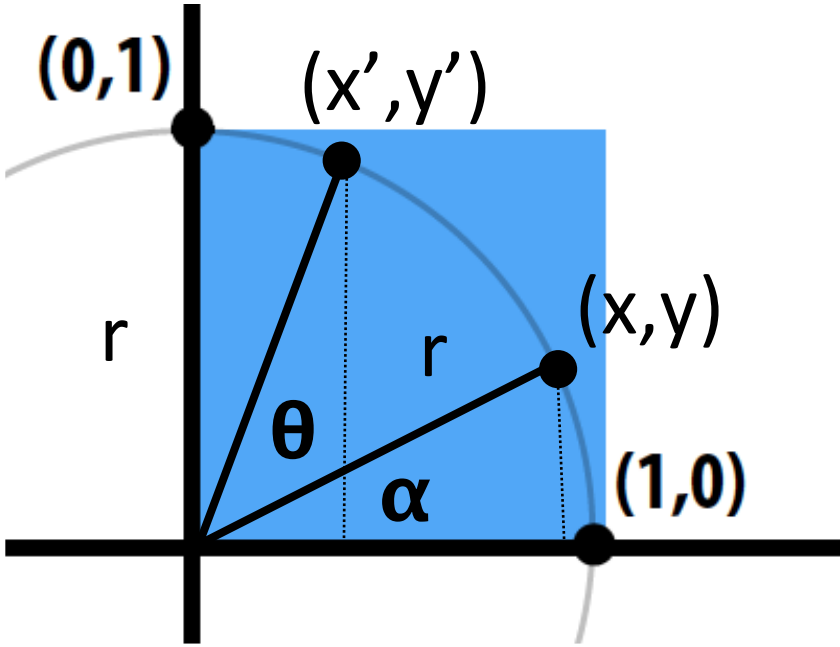


Rotation matrix (2d)

$$x = r \cdot \cos(\alpha)$$
$$y = r \cdot \sin(\alpha)$$

$$x' = r \cdot \cos(\alpha + \theta)$$
$$= r \cdot \cos(\alpha) \cdot \cos(\theta) - r \cdot \sin(\alpha) \cdot \sin(\theta)$$

$$y' = r \cdot \sin(\alpha + \theta)$$
$$= r \cdot \cos(\alpha) \cdot \sin(\theta) + r \cdot \sin(\alpha) \cdot \cos(\theta)$$



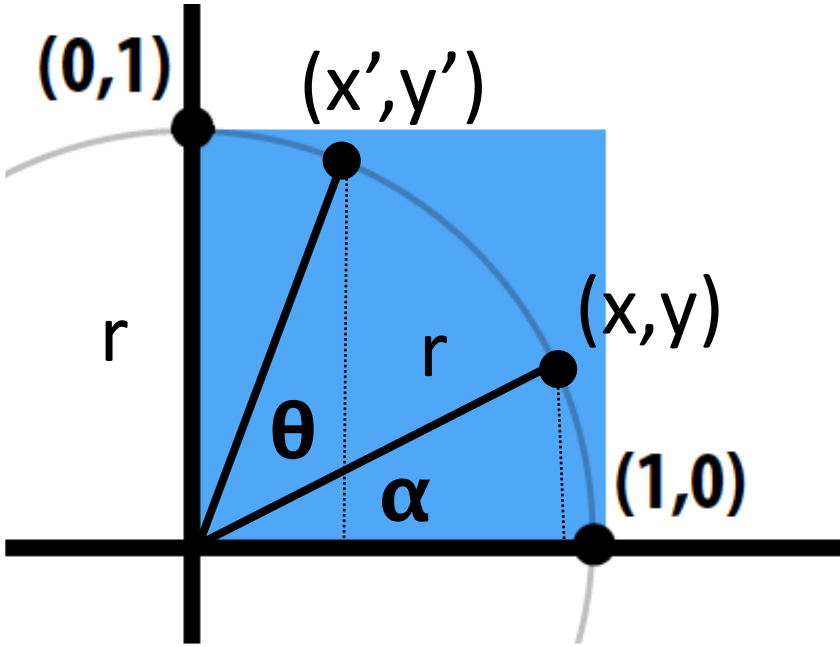
Rotation matrix (2d)

$$x = r \cdot \cos(\alpha)$$
$$y = r \cdot \sin(\alpha)$$

$$x' = r \cdot \cos(\alpha + \theta)$$
$$= r \cdot \cos(\alpha) \cdot \cos(\theta) - r \cdot \sin(\alpha) \cdot \sin(\theta)$$

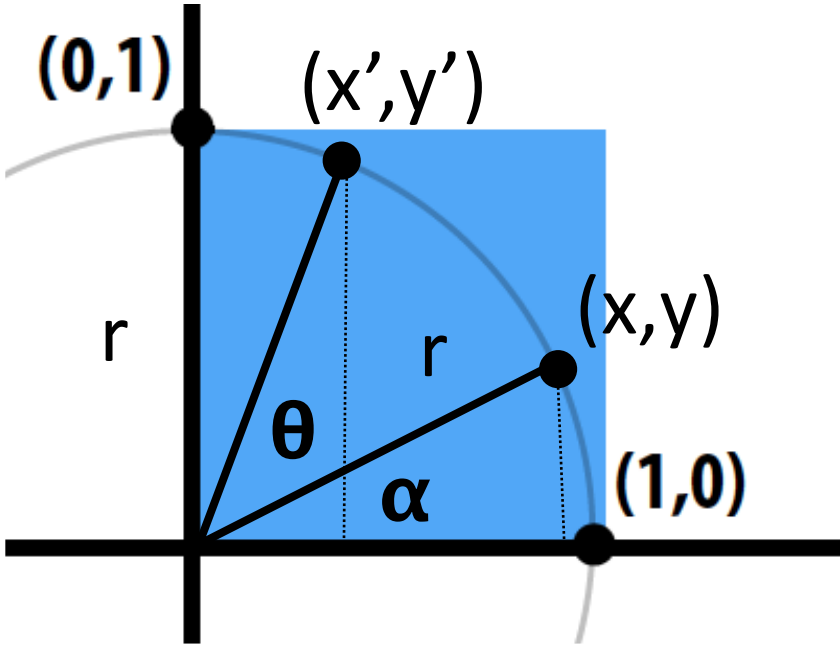
$$y' = r \cdot \sin(\alpha + \theta)$$
$$= r \cdot \cos(\alpha) \cdot \sin(\theta) + r \cdot \sin(\alpha) \cdot \cos(\theta)$$

$$x' = x \cdot \cos(\theta) - y \cdot \sin(\theta)$$
$$y' = x \cdot \sin(\theta) + y \cdot \cos(\theta)$$



Rotation matrix (2d) – part 2

$$
\begin{aligned}
x &= r \cdot \cos(\alpha) \\
y &= r \cdot \sin(\alpha) \\
x' &= r \cdot \cos(\alpha + \theta) \\
&= r \cdot \cos(\alpha) \cdot \cos(\theta) - r \cdot \sin(\alpha) \cdot \sin(\theta) \\
y' &= r \cdot \sin(\alpha + \theta) \\
&= r \cdot \cos(\alpha) \cdot \sin(\theta) + r \cdot \sin(\alpha) \cdot \cos(\theta) \\
x' &= x \cdot \cos(\theta) - y \cdot \sin(\theta) \\
y' &= x \cdot \sin(\theta) + y \cdot \cos(\theta)
\end{aligned}
$$

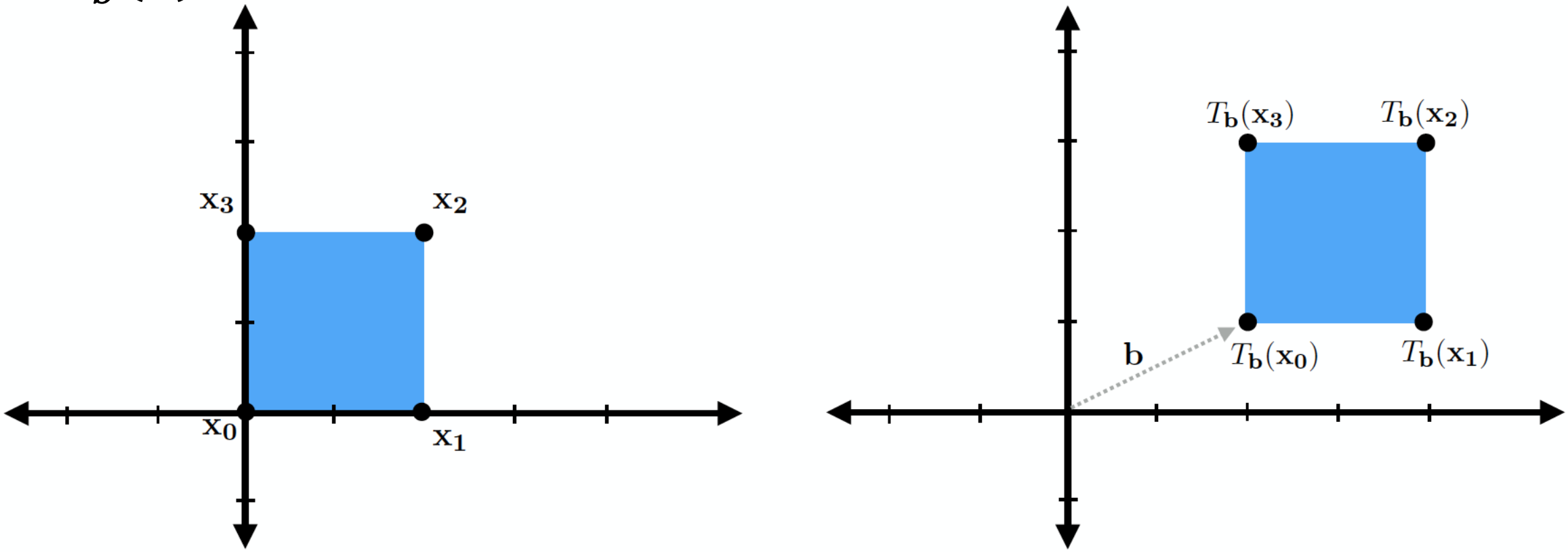


$$
R_\theta = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}
$$

How do we deal with translation?

Translation not represented by a 2x2 matrix

$$T_b(x) = x + b$$



Problem!

Summary

$$S_S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \qquad R_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \qquad T_b(x) = x + b$$

2D homogeneous coordinates

- Represent the 2d points with three values (“homogeneous coordinates”)
- So, the point (x, y) is represented as a 3D-vector $[x \quad y \quad 1]^T$
- And all the transformations are represented as 3x3 matrices that transform these vectors

Translation in 2D homogeneous coordinates

- Translation expressed as 3x3 matrix multiplication:

$$T_b = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix}$$

Translation in 2D homogeneous coordinates

- Translation expressed as 3x3 matrix multiplication:

$$x = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_x \\ x_y \\ 1 \end{bmatrix} = \begin{bmatrix} x_x + b_x \\ x_y + b_y \\ 1 \end{bmatrix}$$

Homogeneous coordinates let us encode translations as linear transformations

Scale & rotation in 2D homogeneous coordinates

- For transformations that are already linear, not much changes:

$$S_S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad R_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Summary

$$\mathrm{P}(x_x, x_y) = \begin{bmatrix} x_x \\ x_y \\ 1 \end{bmatrix}$$

$$S_S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad R_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad T_b = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix}$$

Directions & Points

Point (homogenous coord. = 1):

$$T_b x = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_x \\ x_y \\ 1 \end{bmatrix} = \begin{bmatrix} x_x + b_x \\ x_y + b_y \\ 1 \end{bmatrix}$$

Direction (homogenous coord. = 0):

$$T_b d = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} d_x \\ d_y \\ 0 \end{bmatrix}$$

Composing Transformations

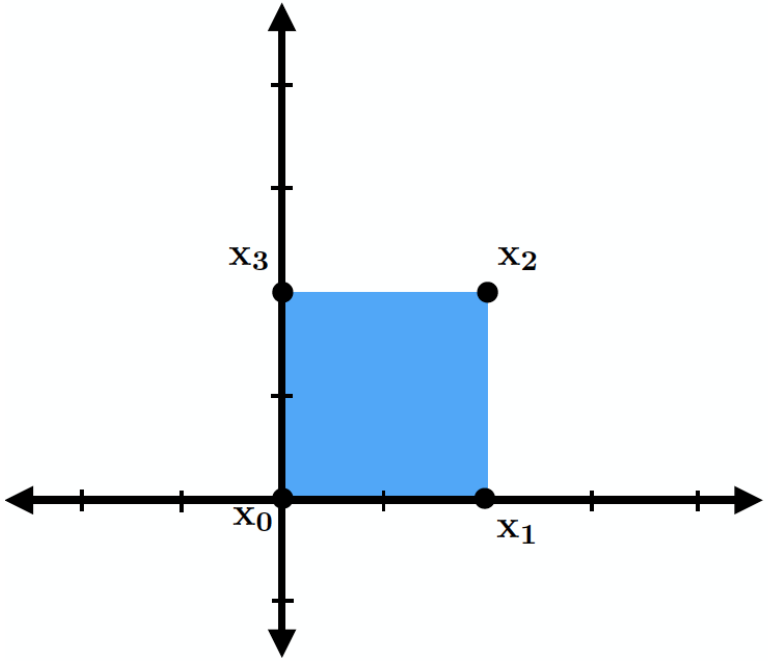
Example

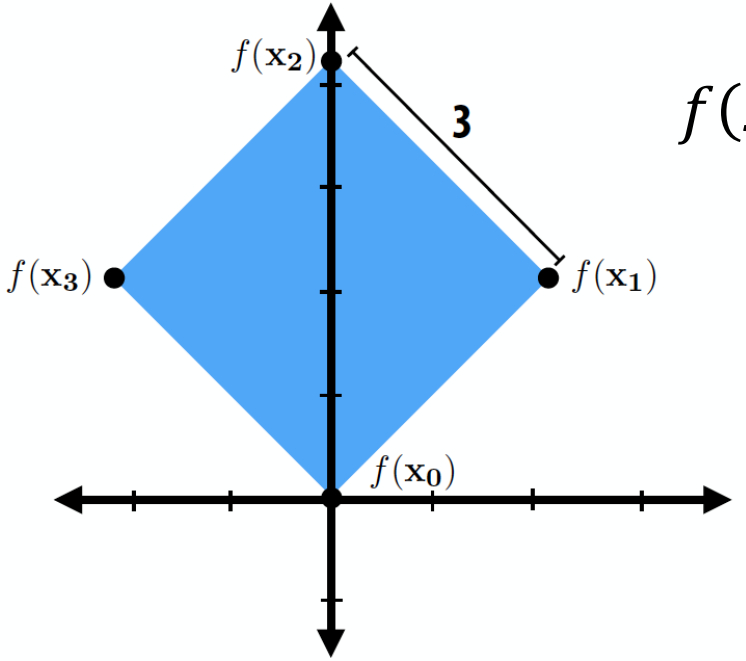
$$\mathrm{P}(2,3) = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

$$S_{(2,4)} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 1 \end{bmatrix} \; R_{45} = \begin{bmatrix} \cos(45) & -\sin(45) & 0 \\ \sin(45) & \cos(45) & 0 \\ 0 & 0 & 1 \end{bmatrix} T_{(-2,-3)} = \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$$

$$M \cdot P = \; S_{(2,4)} \, R_{45} \, T_{(-2,-3)} \cdot P$$

How do we compose linear transformations?





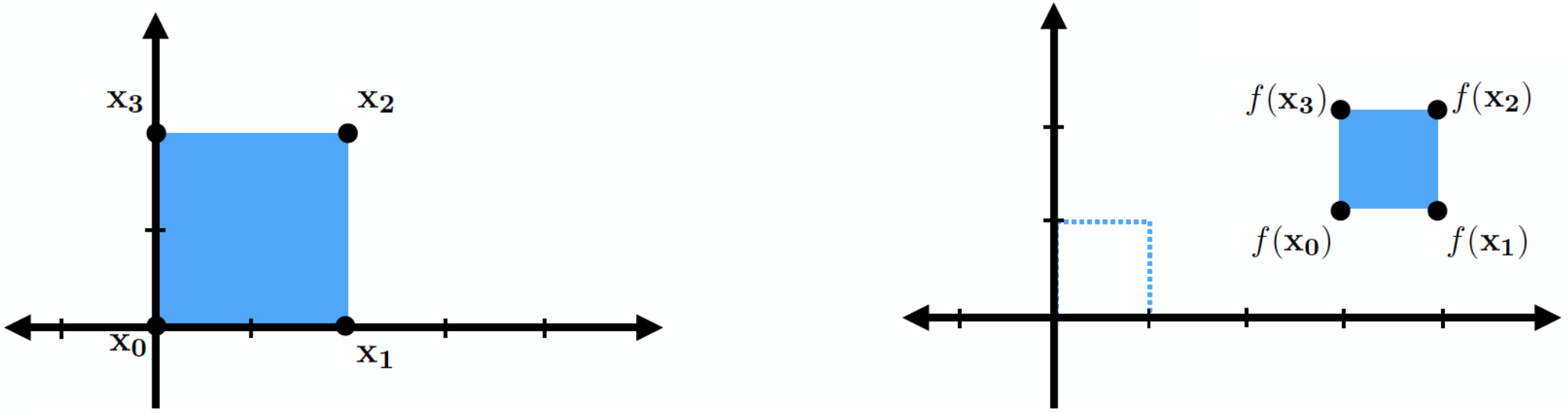
$$f(x) = R_{\pi/4} S_{[1.5,1.5]} x$$

Now we can compose linear transformations via matrix multiplications – this enables a simple, efficient implementation: reduce complex chain of transformations to a single matrix multiplication.

Matrix Multiplication

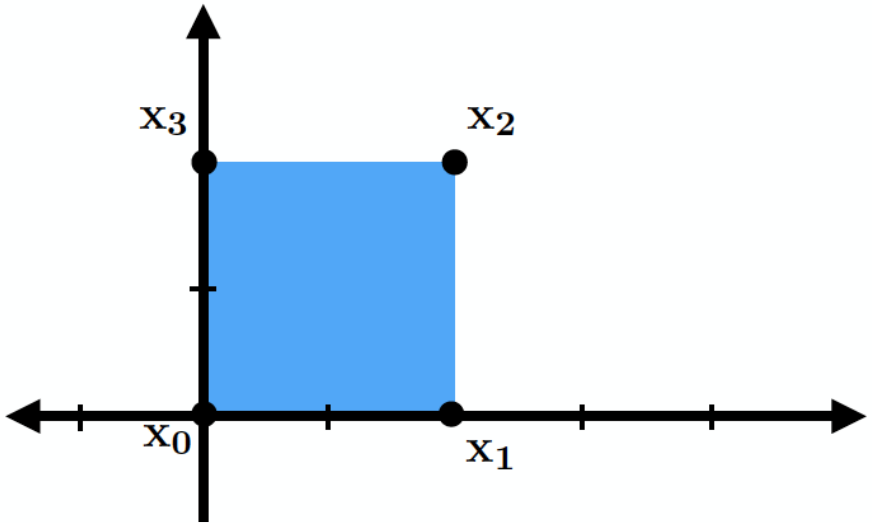


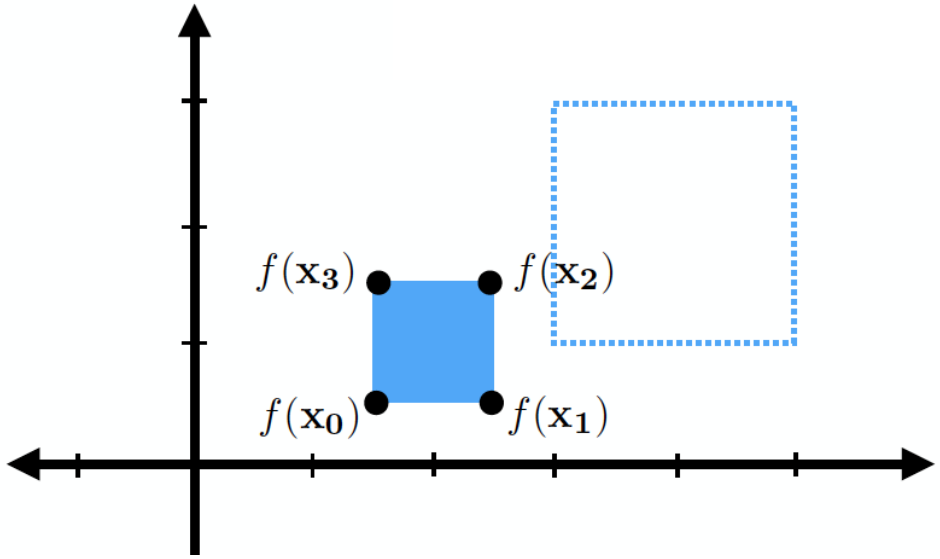
Order of composition matters



- Scale, then translate $f(x) = T_{3,1}S_{[0.5,0.5]}x$

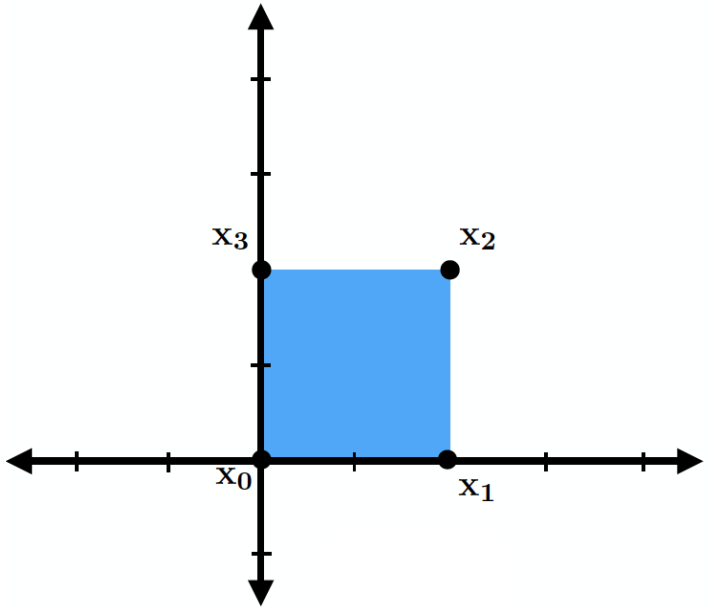
Order of composition matters

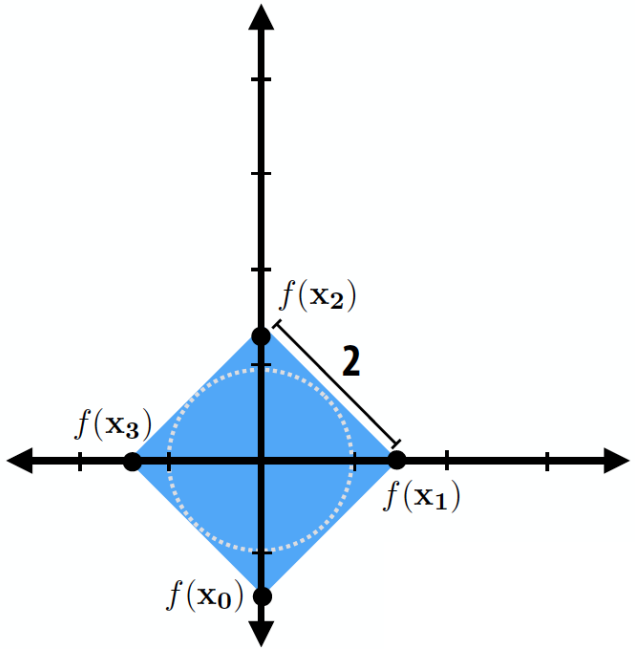




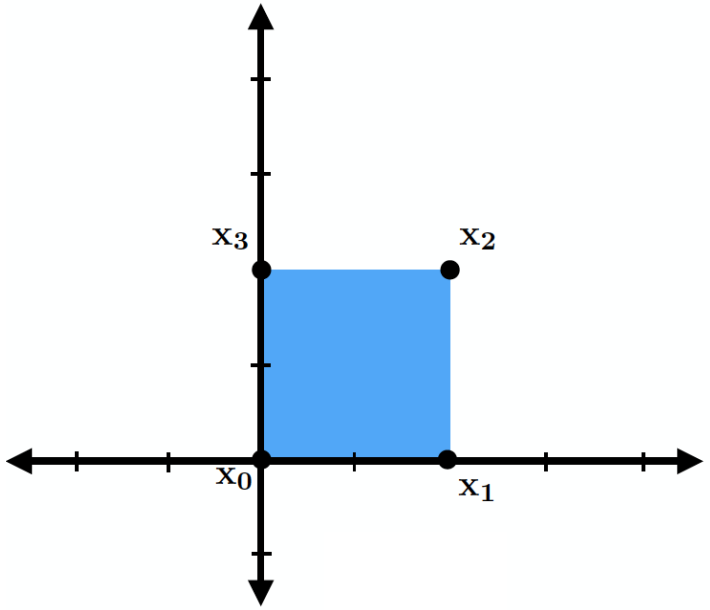
- Translate, then scale $f(x) = S_{[0.5,0.5]}T_{3,1}x$

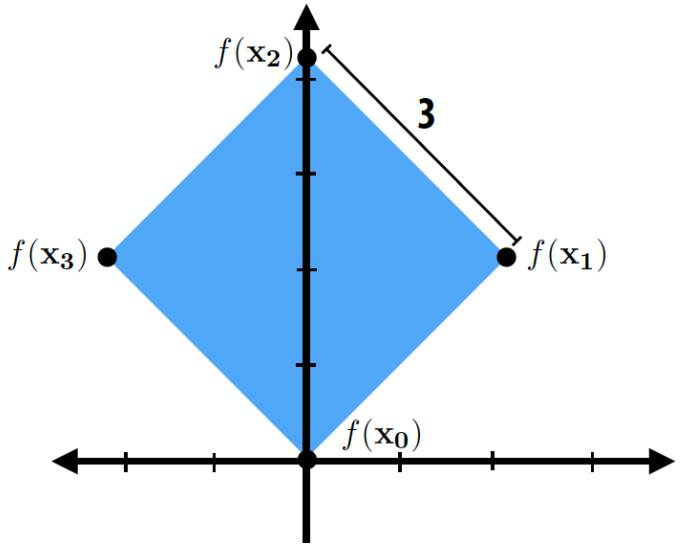
How would you perform this transformation?



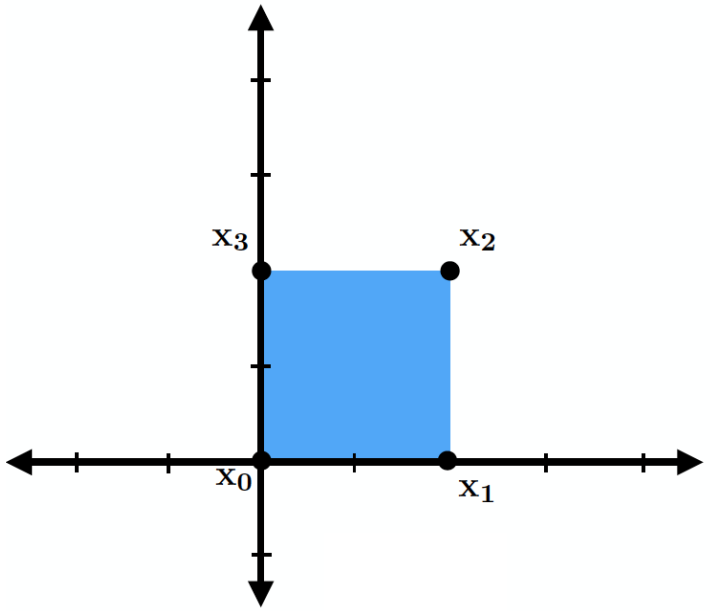


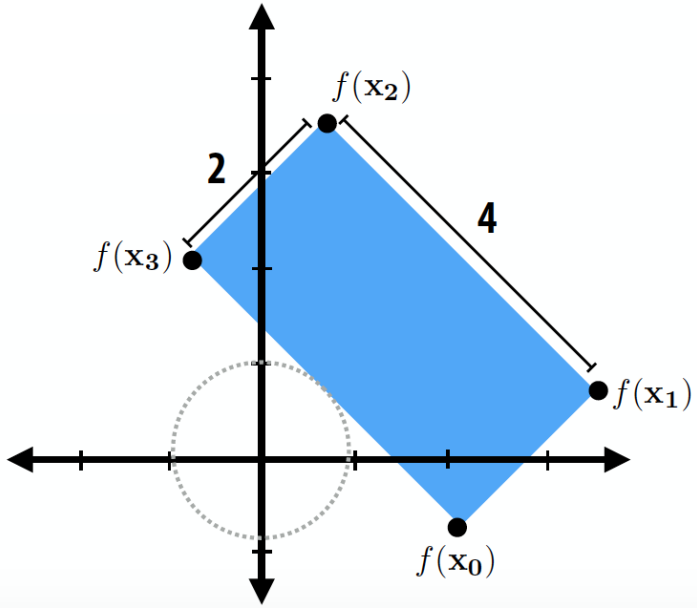
How would you perform this transformation?



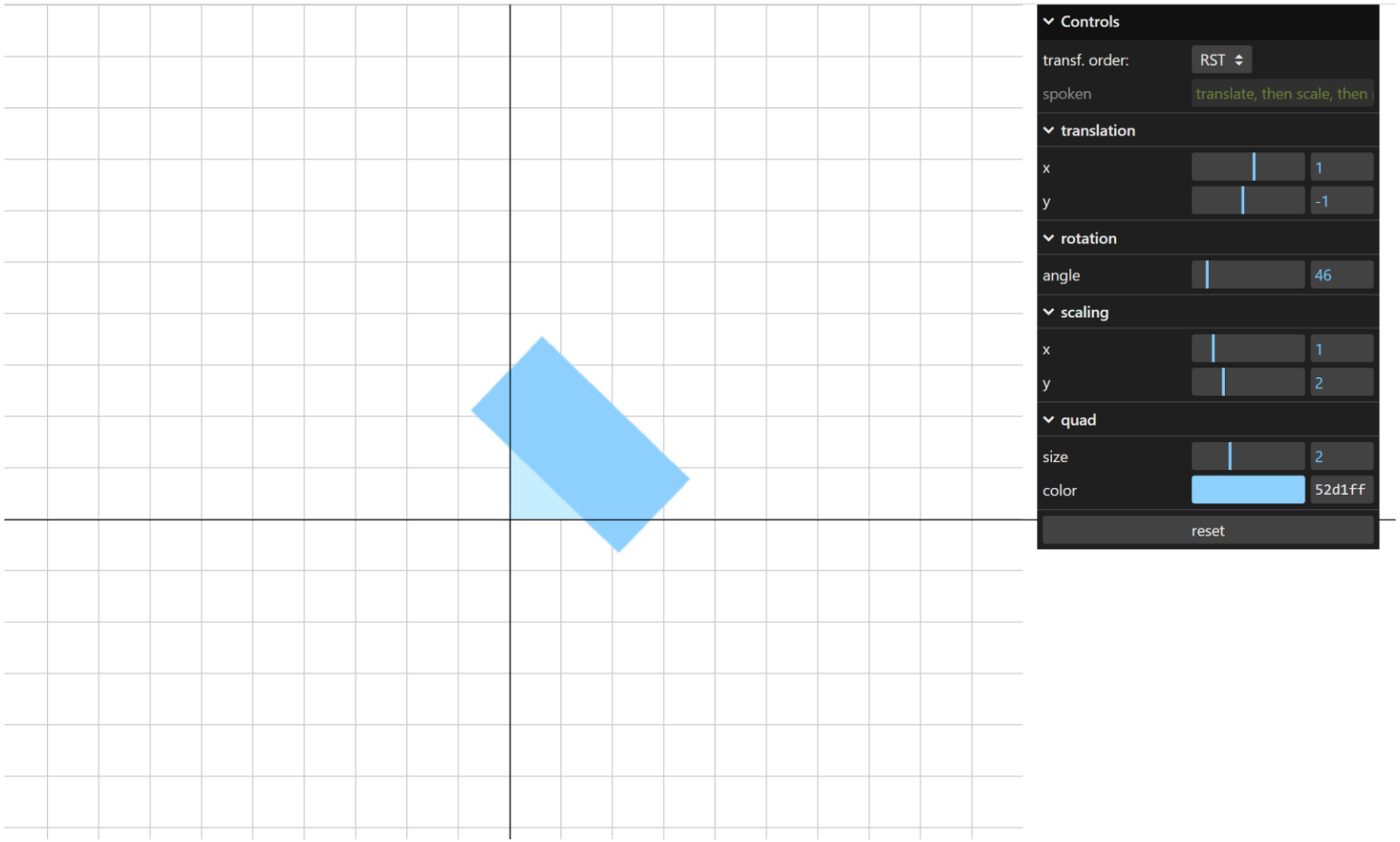


How would you perform this transformation?



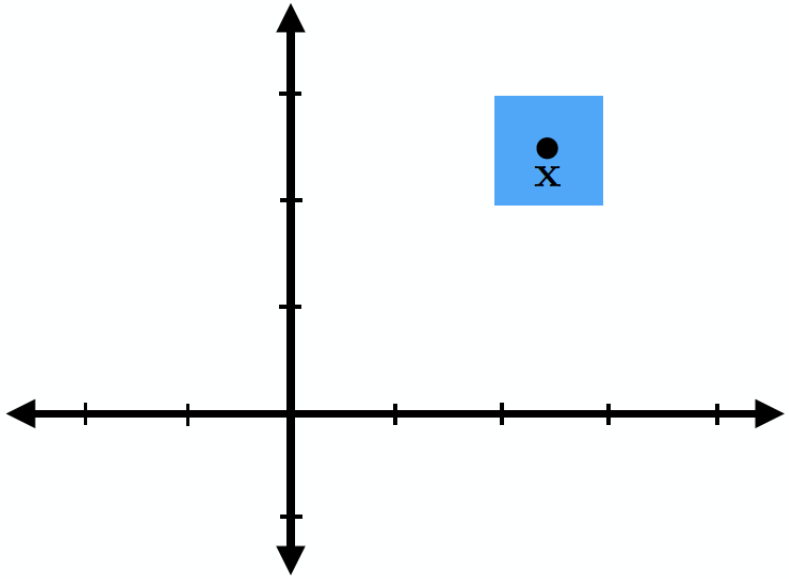


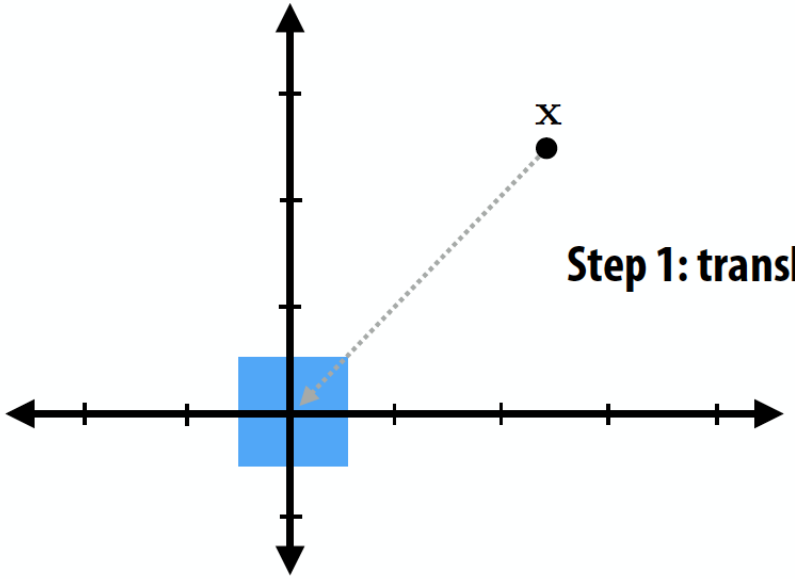
Try it yourself!

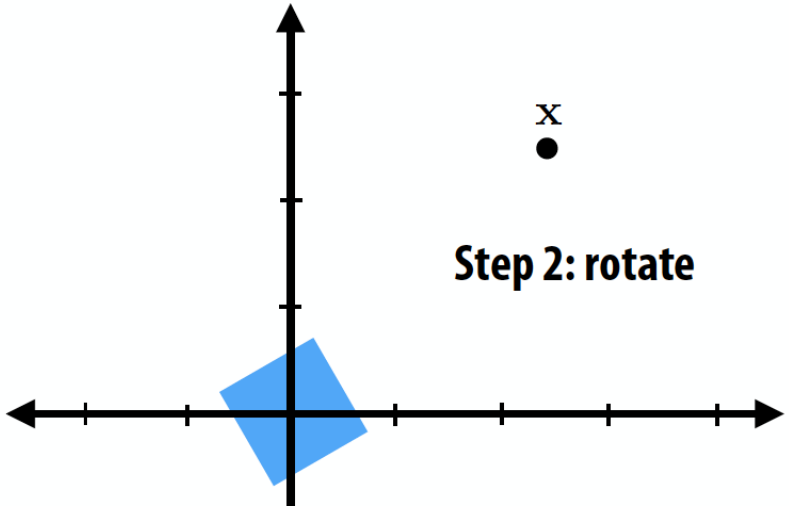


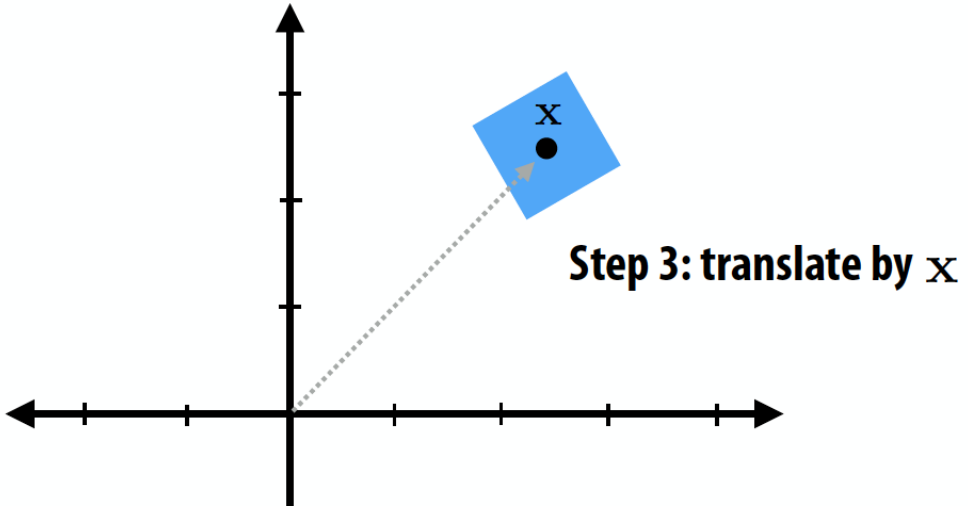
https://schedldave.github.io/gop_playground/transformations2D

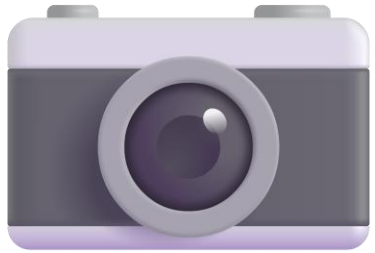
Rotate about a point x







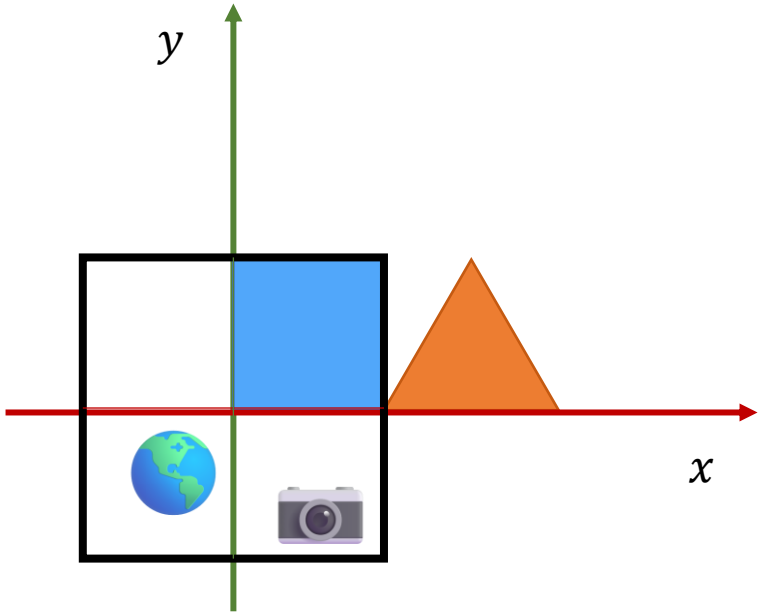




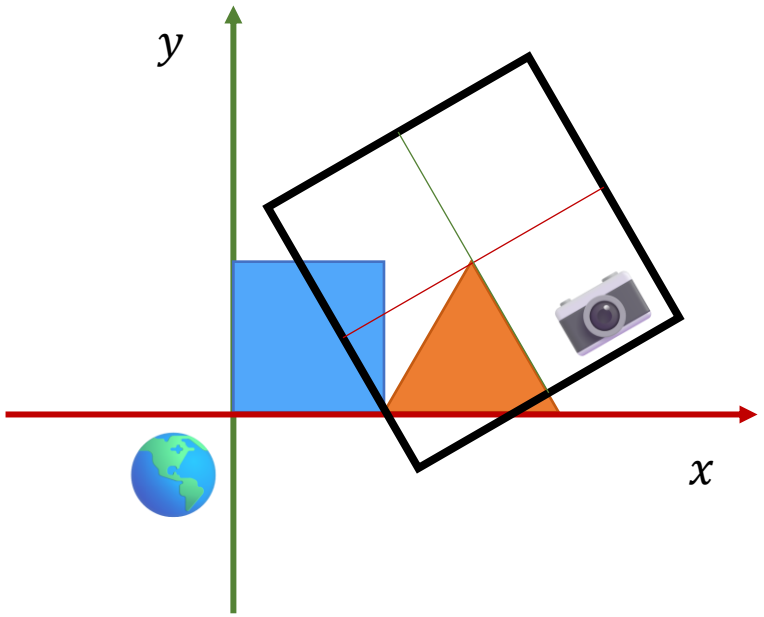
(2D) Camera Transformations

Camera Transformation
📷 ➔ 🌎 (Camera to World)

📷 in origin of our world (🌎)



Camera Transformation
📷 ➔ 🌎 (Camera to World)



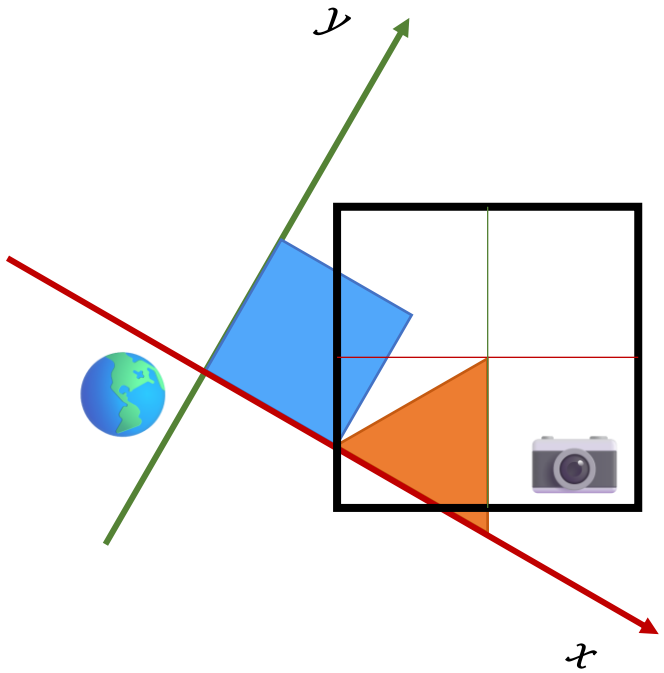
📷 ➔ 🌎

rotate, then translate

$$f(x) = T_{1.5,1} R_{30} x$$

Camera Transformation

🌍 ➔ 📷 (World to Camera)



📷 ➔ 🌍

rotate, then translate

$f(x) = T_{1.5,1} R_{30} x$

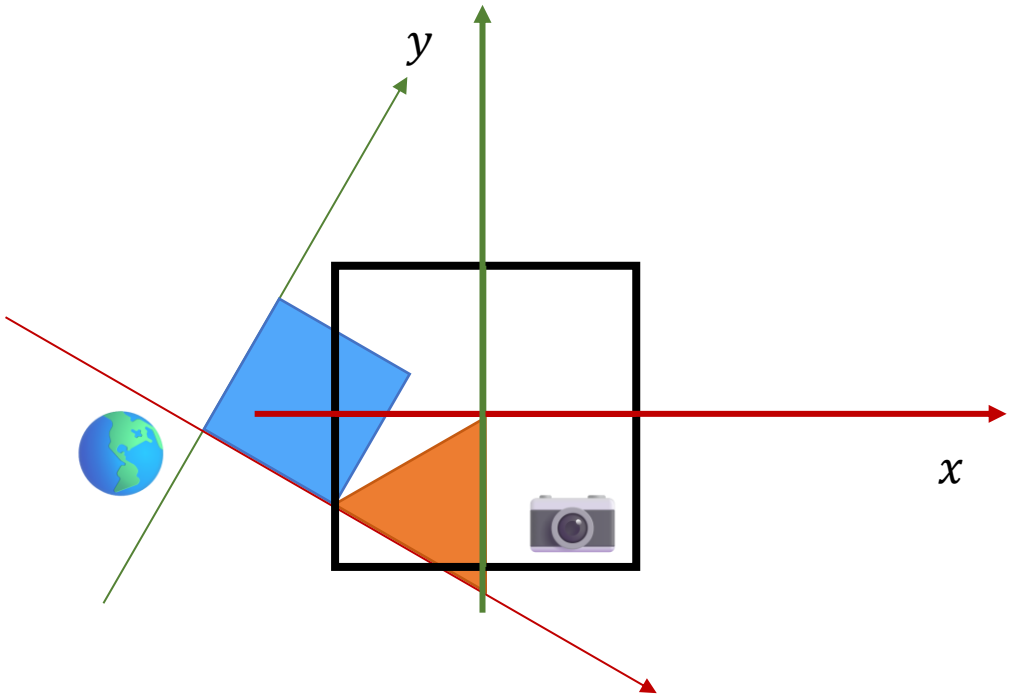
⇕ inverse

🌍 ➔ 📷

-translate, then -rotate

$f^{-1}(x) = R_{-30} T_{-1.5,-1} x$

Camera Transformation
🌎 ➔ 📷 (World to Camera)

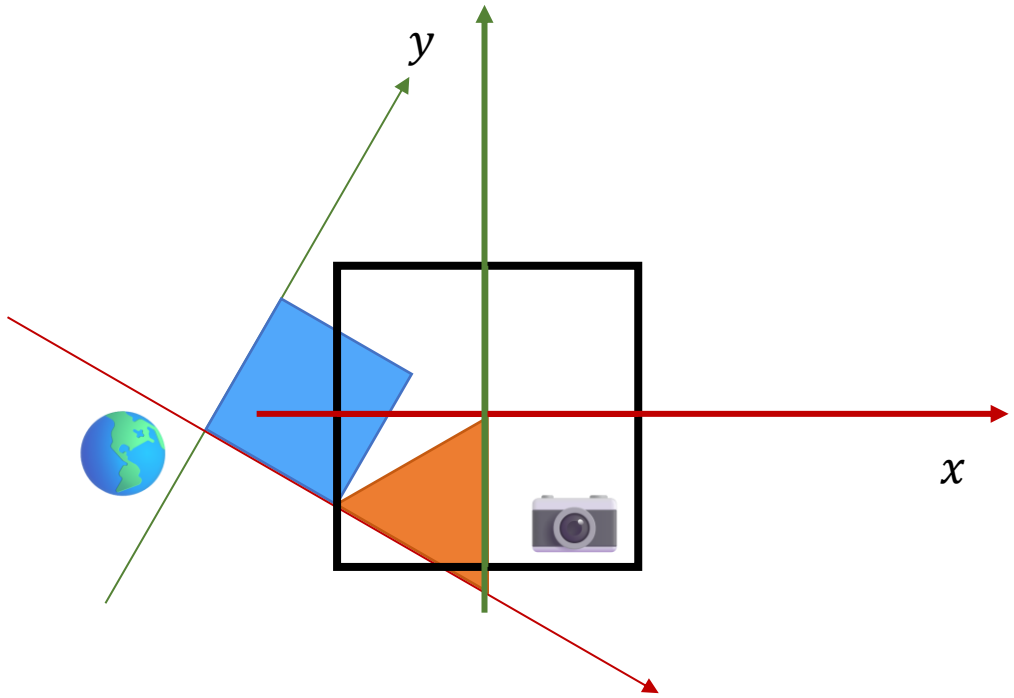


In graphics we use 🌎 ➔ 📷 transformations to transform objects from world to view/camera space!

We move the world to the camera!

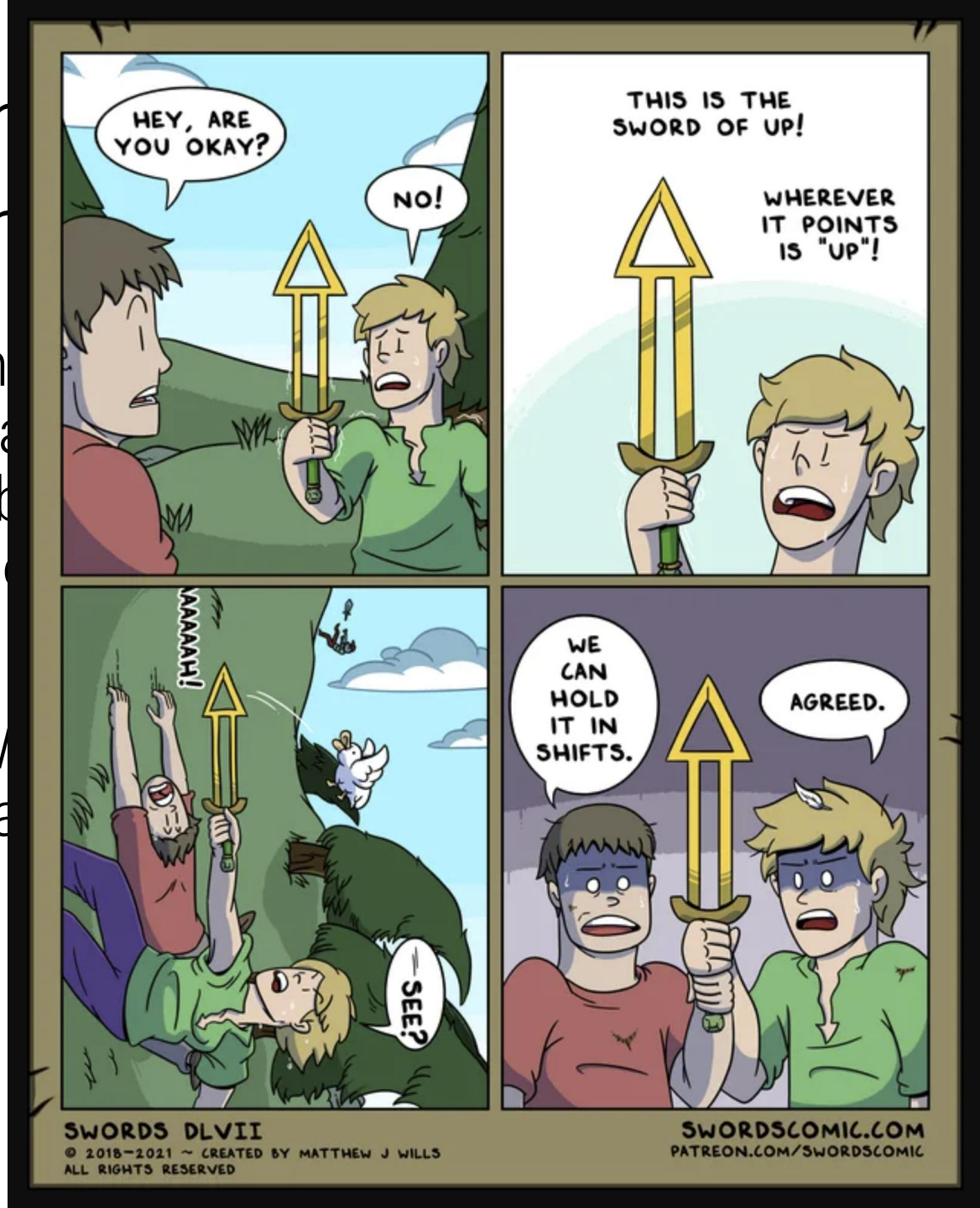
Camera Transformation

🌍 ➔ 📷 (World to Cam

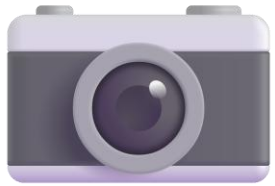


In
tra
ob
vi

W
ca

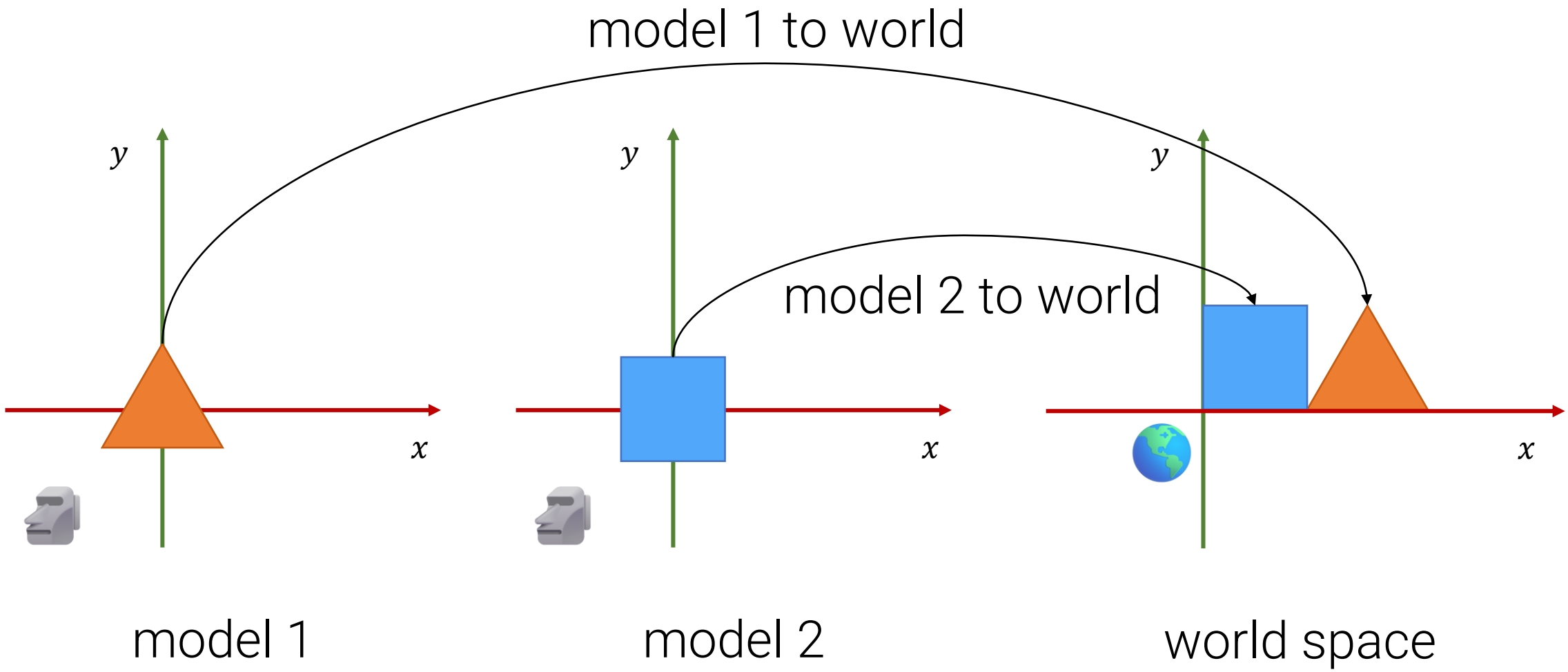


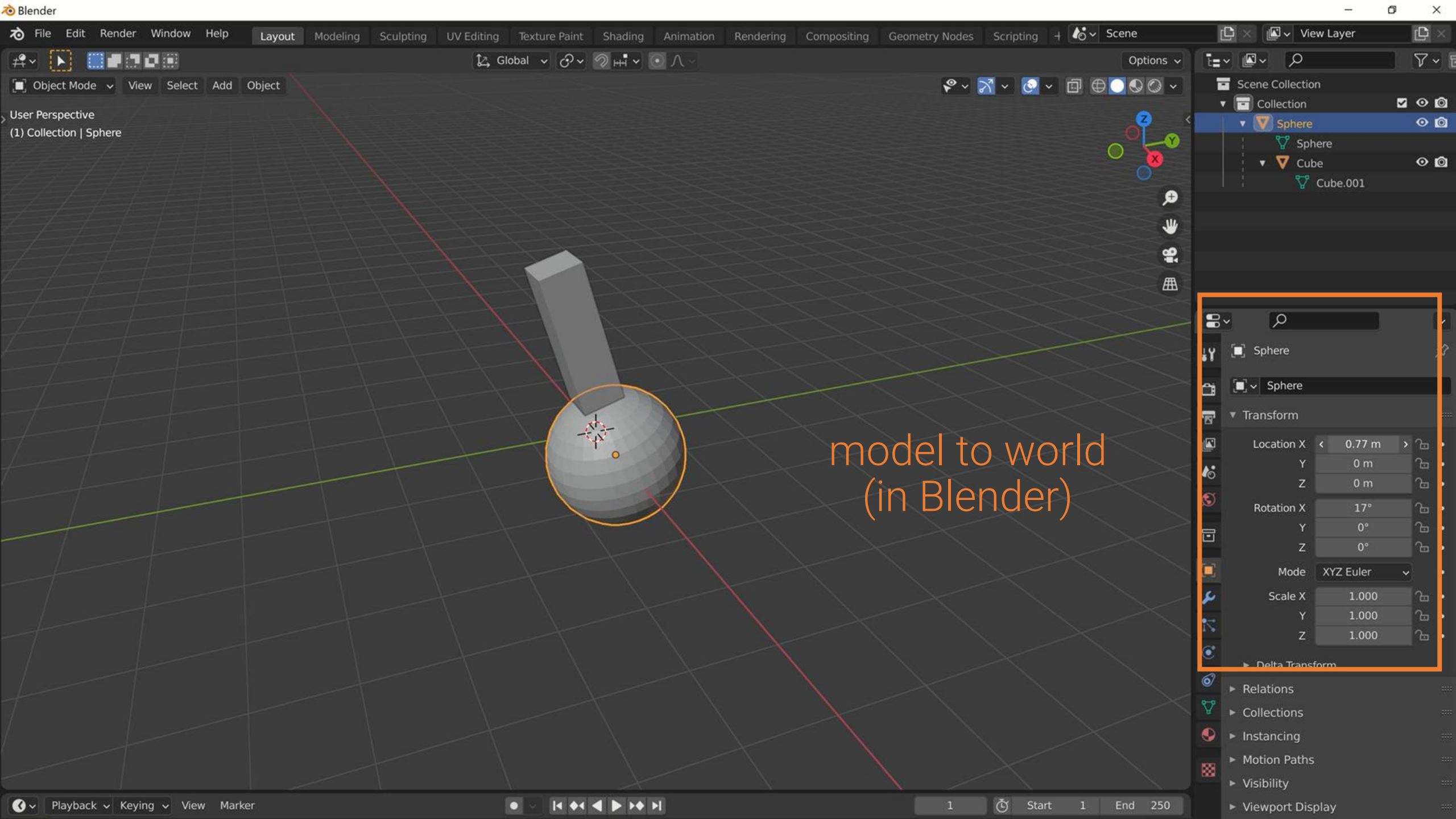




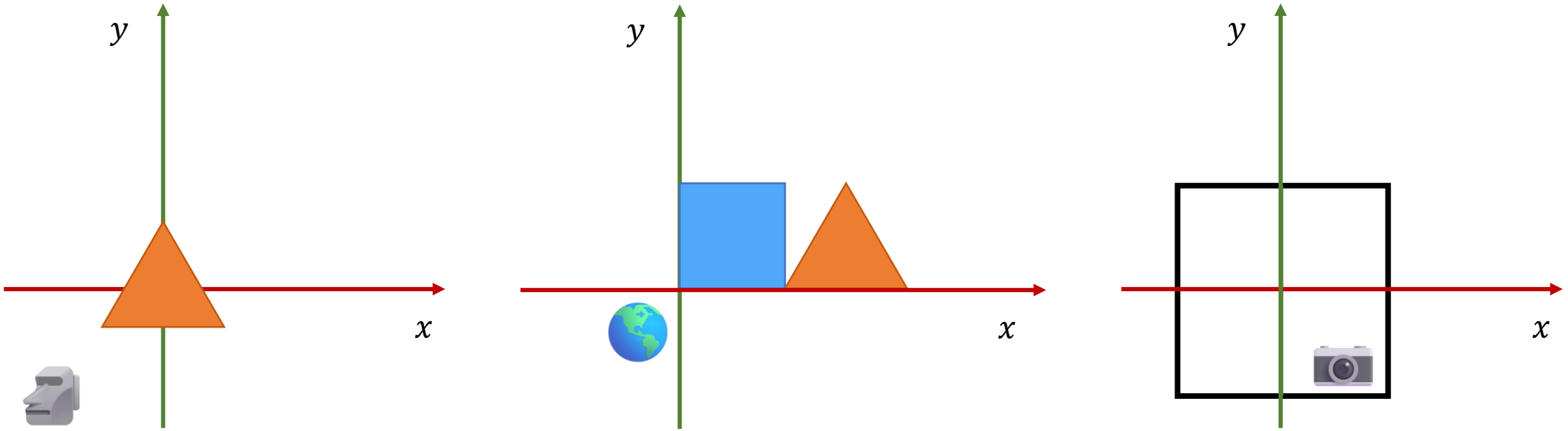
More on (2D) Spaces

Model Space


model 1 to world
model 2 to world
y
x
y
x
y
x
model 1
model 2
world space


Blender
File Edit Render Window Help
Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Scene
View Layer
Global
Options
Object Mode View Select Add Object
User Perspective
(1) Collection | Sphere
Scene Collection
Collection
Sphere
Sphere
Cube
Cube.001
Sphere
Sphere
Transform
Location X 0.77 m
Y 0 m
Z 0 m
Rotation X 17°
Y 0°
Z 0°
Mode XYZ Euler
Scale X 1.000
Y 1.000
Z 1.000
Delta Transform
Relations
Collections
Instancing
Motion Paths
Visibility
Viewport Display
model to world
(in Blender)
Playback Keying View Marker
1
Start 1
End 250

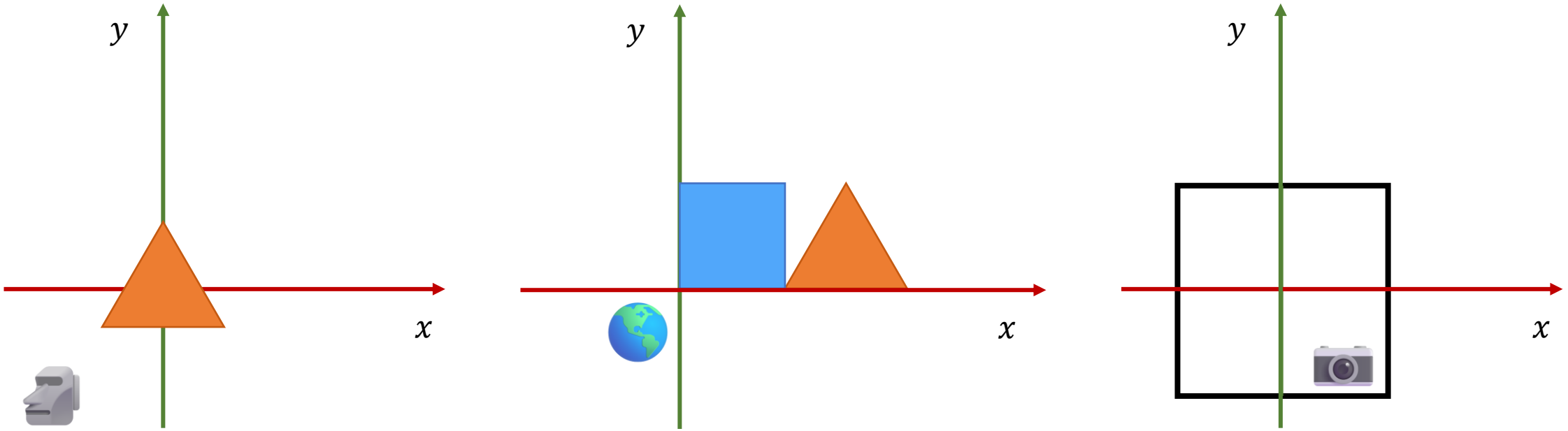
Space Transformations (2D)



🗿 ➔ 🌎
model 1 to world

🌎 ➔ 📷
world to camera/view

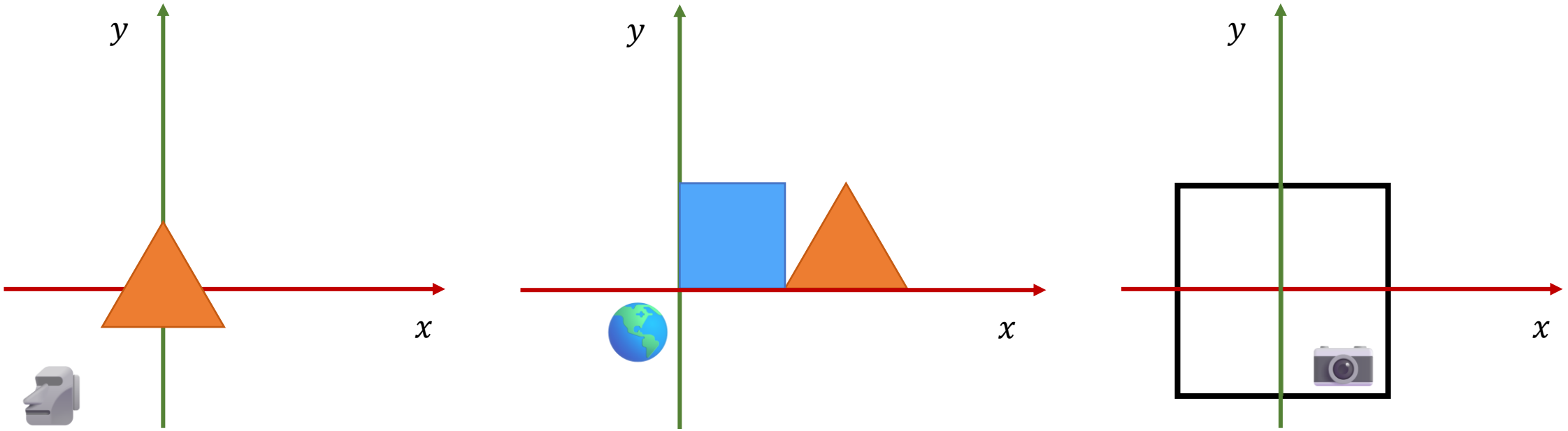
Transformation Matrices (2D)



🗿 ➔ 🌎
model or world matrix

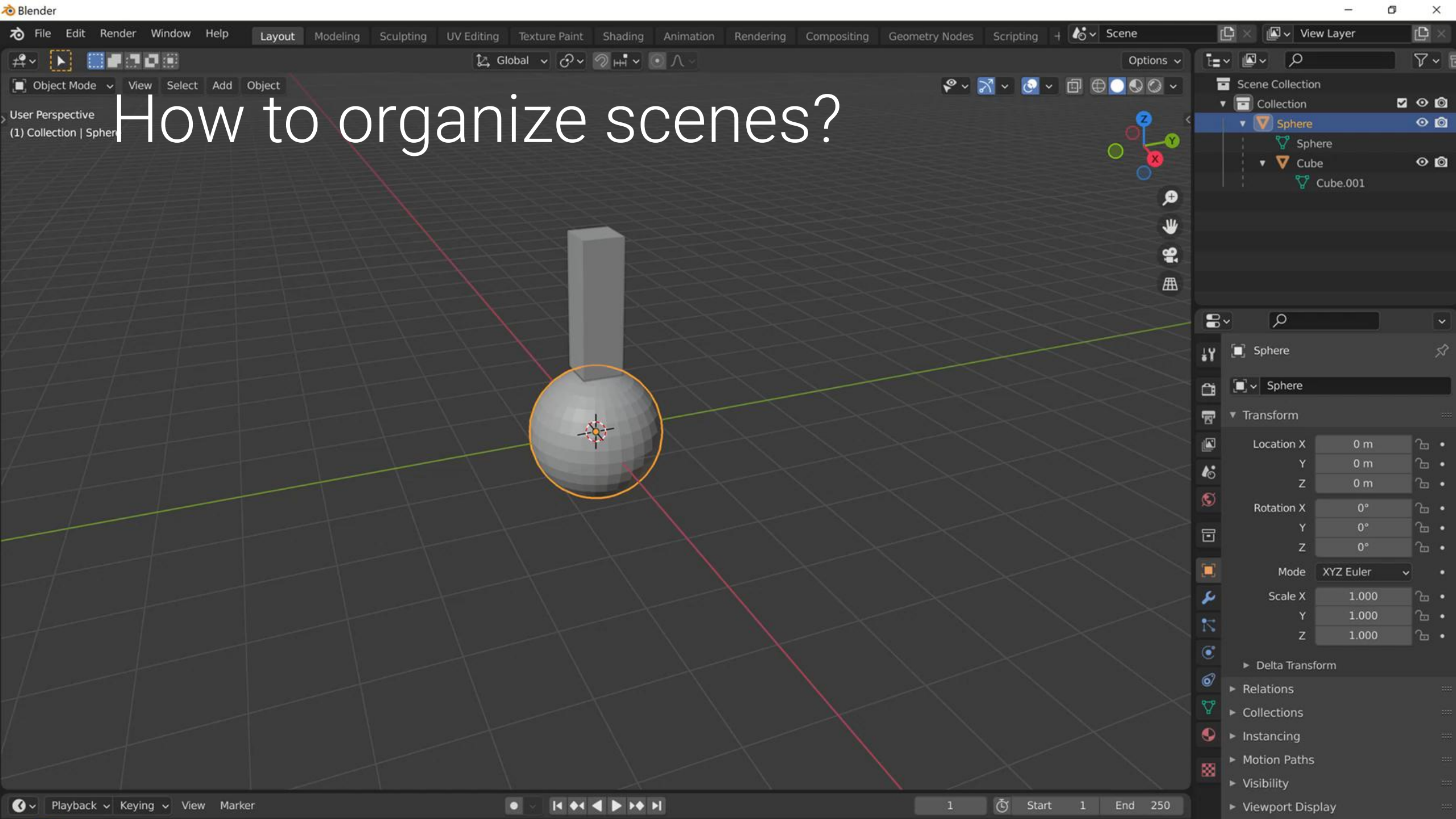
🌎 ➔ 📷
camera/view matrix

Transformation Matrices (2D)



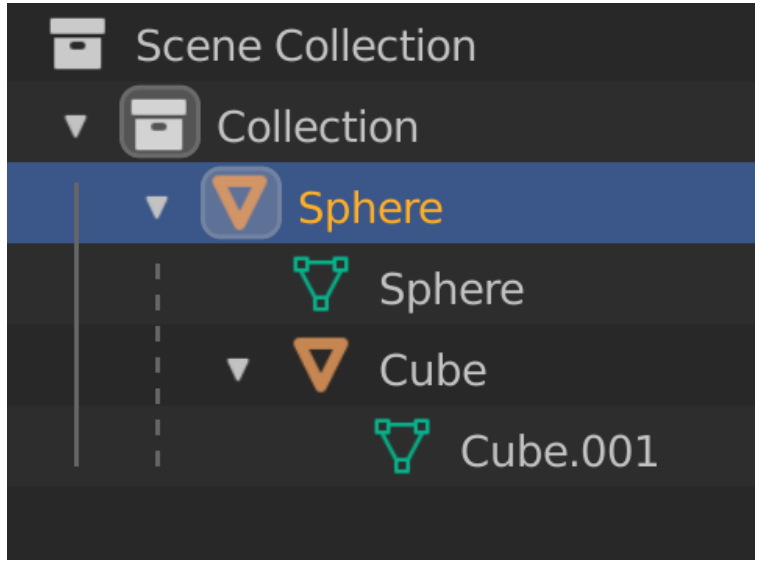
🗿 ➔ 🌎 ➔ 📷

model-view matrix


Blender
File Edit Render Window Help
Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Scene
View Layer
Global
Options
Object Mode View Select Add Object
User Perspective
(1) Collection | Sphere
How to organize scenes?
Scene Collection
Collection
Sphere
Sphere
Cube
Cube.001
Sphere
Sphere
Transform
Location X 0 m
Y 0 m
Z 0 m
Rotation X 0°
Y 0°
Z 0°
Mode XYZ Euler
Scale X 1.000
Y 1.000
Z 1.000
Delta Transform
Relations
Collections
Instancing
Motion Paths
Visibility
Viewport Display
Playback Keying View Marker
1
Start 1
End 250

Scenegraph

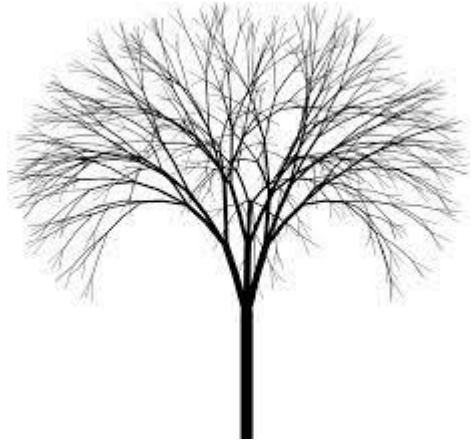
- Hierarchical (tree) structure for organizing (3D) scenes
- Objects in the scene are once in graph
- A graph can be traversed recursively



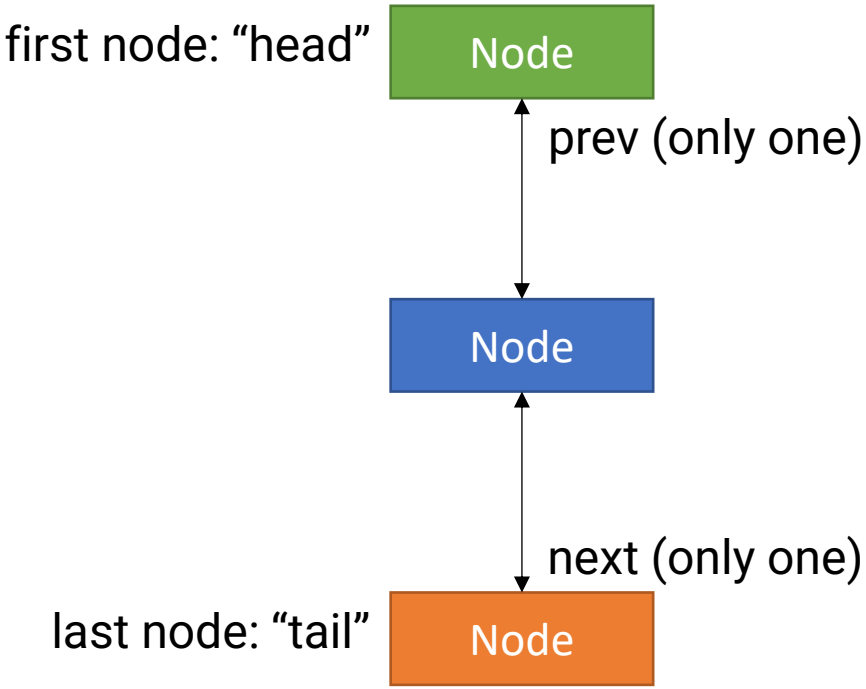
Advantages:

- Propagation of properties (transformations: objects move together)
- Unique point of access
- Reusable components / Abstraction (OOP)

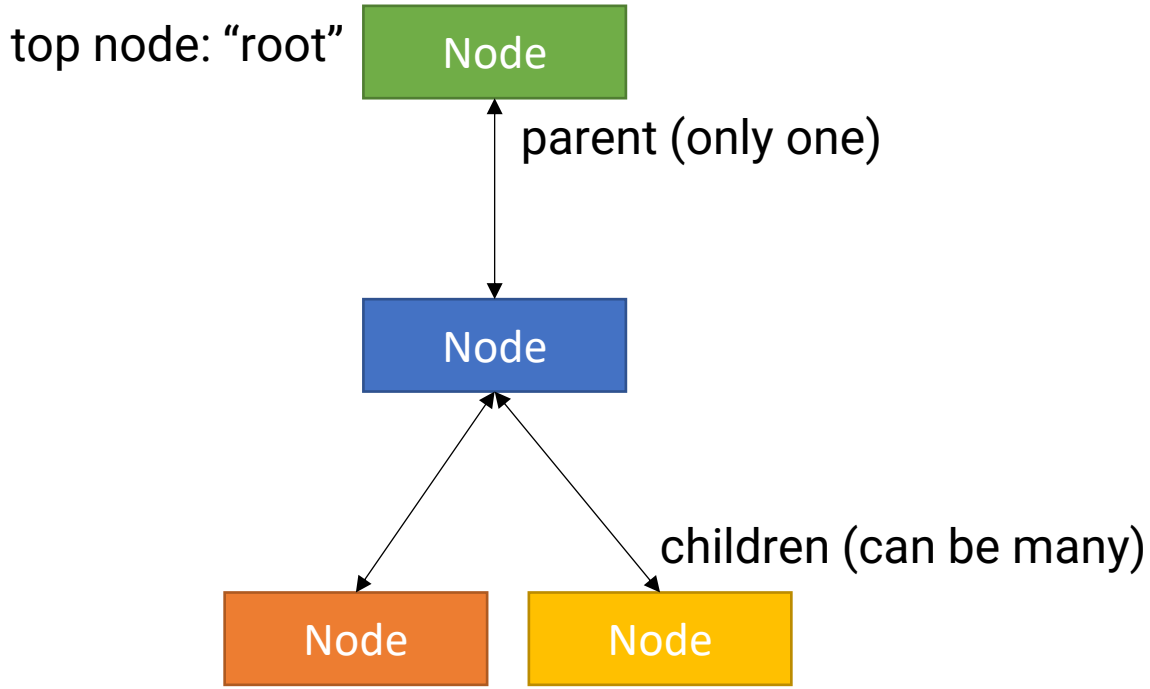
Linked-List vs. Tree



(Double) Linked-List

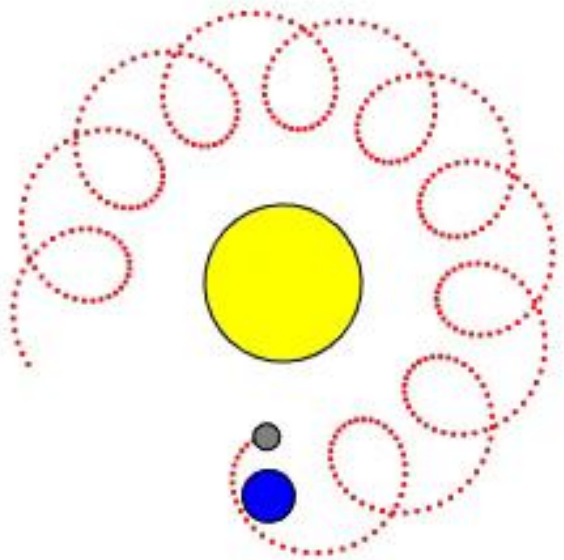


Tree



Scenegraph Example

- Sun (root; parent of earth and other planets)
  - Earth (child of sun)
    - Moon (child of earth)
  - Mars (child of sun)
  - …

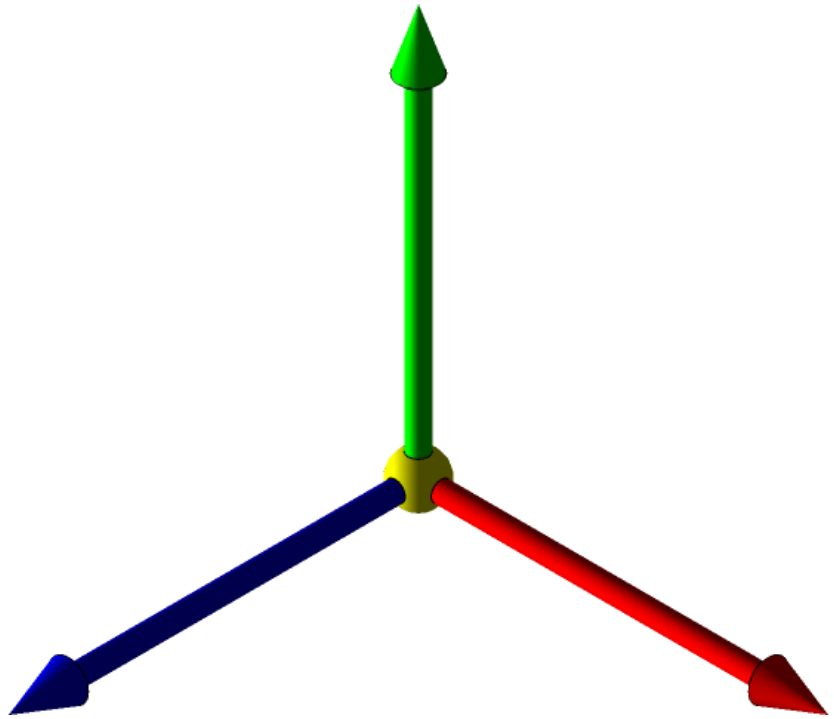


- Getting transformations for the Moon:

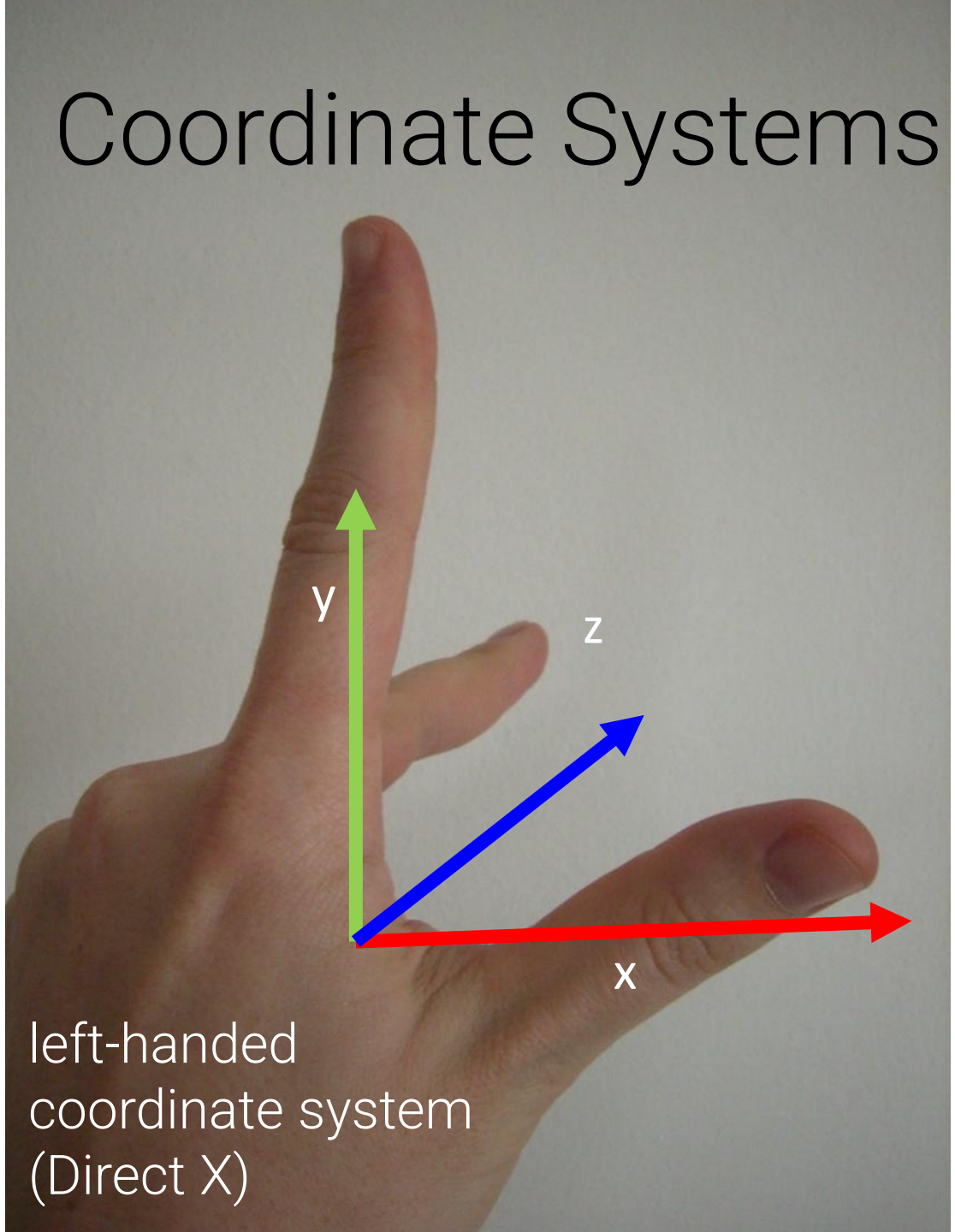
  worldMatrixForMoon = starMatrix * planetMatrix * moonMatrix;

https://webglfundamentals.org/webgl/lessons/webgl-scene-graph.html

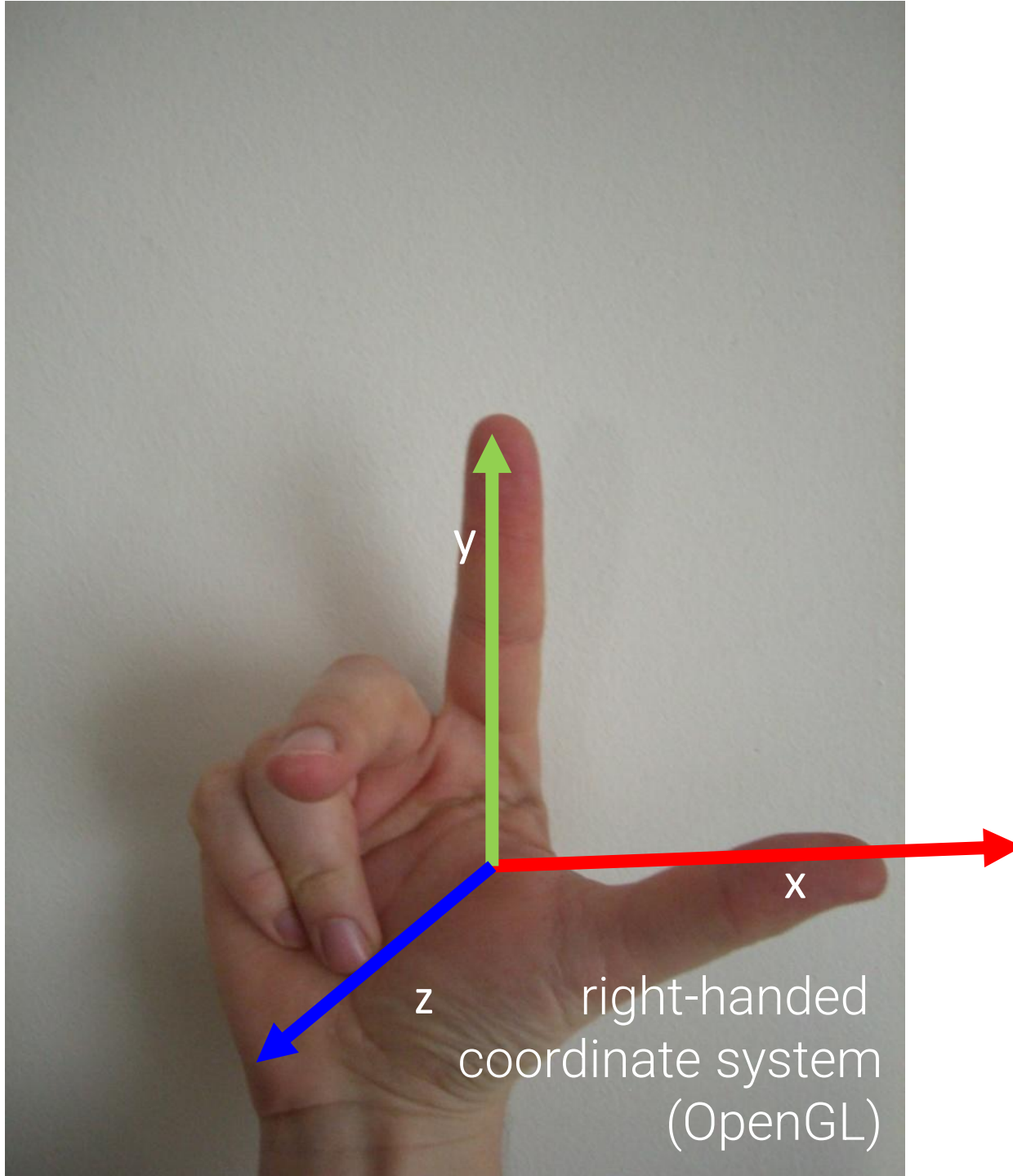
3D Transformations



Coordinate Systems

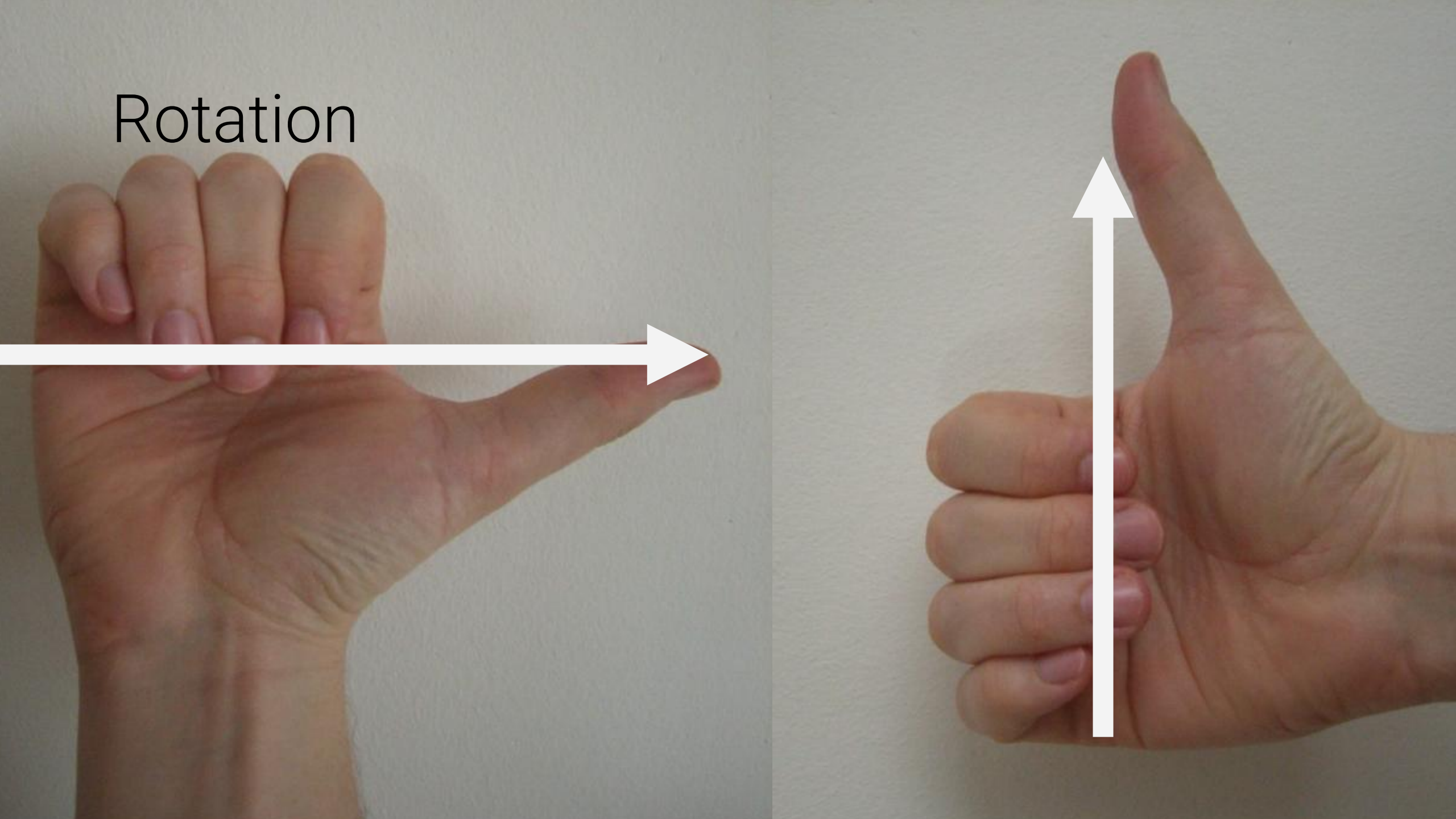


left-handed
coordinate system
(Direct X)



right-handed
coordinate system
(OpenGL)

Rotation



4x4 Matrices for Translation and Scale

$$\mathrm{P}(x_x, x_y, x_z) = \begin{bmatrix} x_x \\ x_y \\ x_z \\ 1 \end{bmatrix}$$

$$S_S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_b = \begin{bmatrix} 1 & 0 & 0 & b_x \\ 0 & 1 & 0 & b_y \\ 0 & 0 & 1 & b_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D homogeneous coordinates

- Represent the 3D points with 4 values ("homogeneous coordinates")
- So, the point (x, y, z) is represented as 4D-vector $[x \quad y \quad z \quad 1]^T$
- All the transformations are represented as 4x4 matrices

3D Transformations



https://youtu.be/rHLEWRxRGiM

4x4 Rotation Matrices

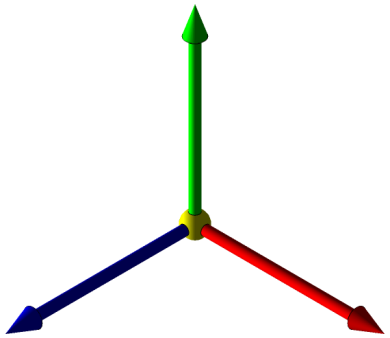
$$\mathrm{P}(x_x, x_y, x_z) = \begin{bmatrix} x_x \\ x_y \\ x_z \\ 1 \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

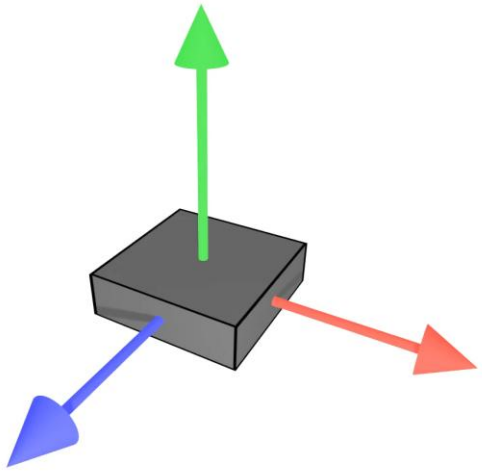
$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

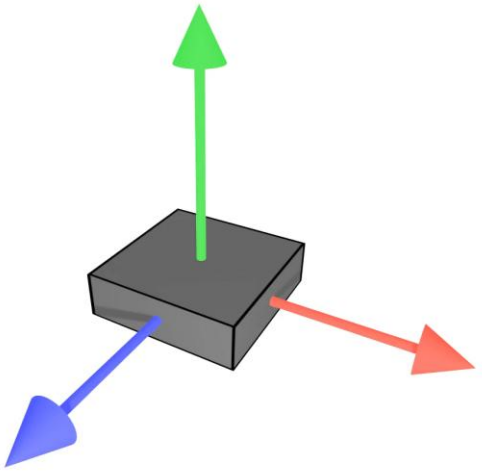
4x4 Rotation Matrices



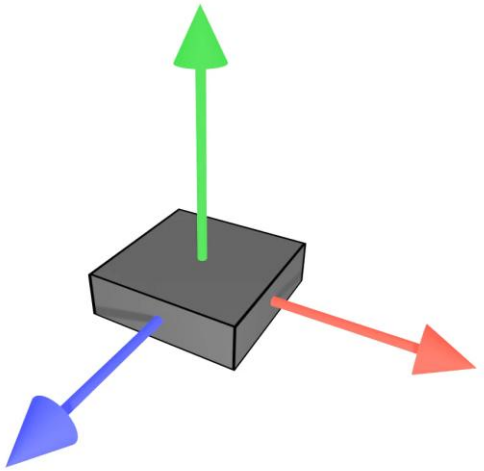
$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



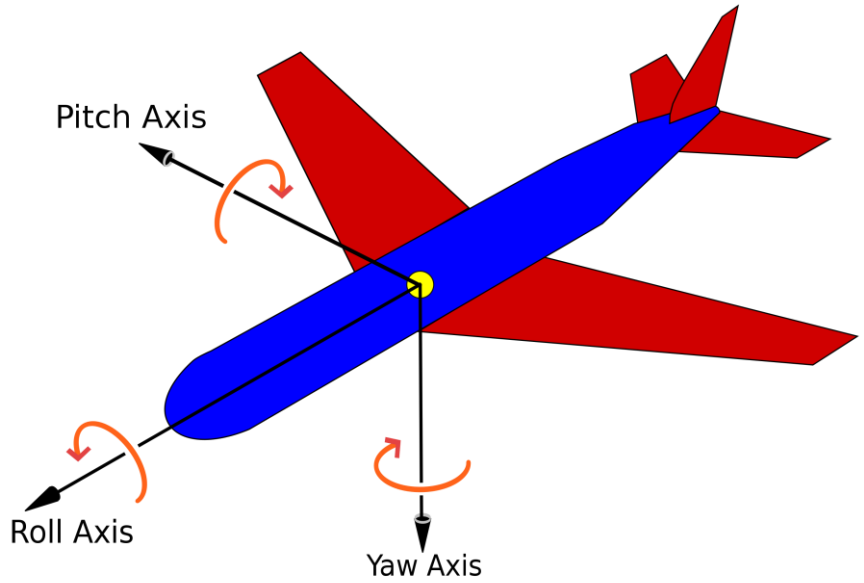
$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

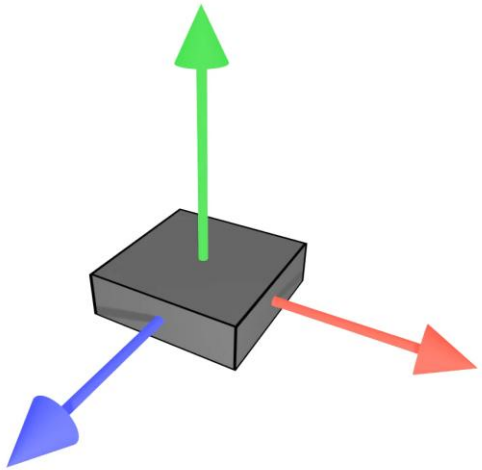


$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

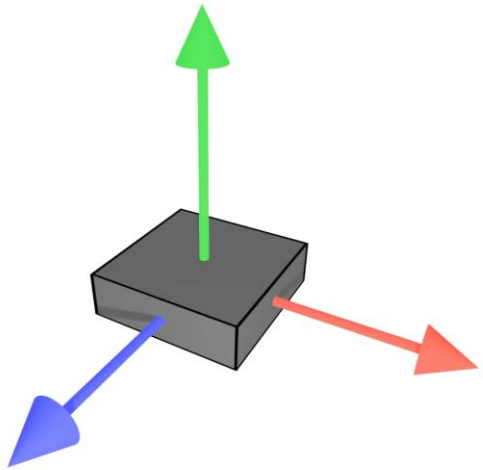


Roll, Pitch, Yaw

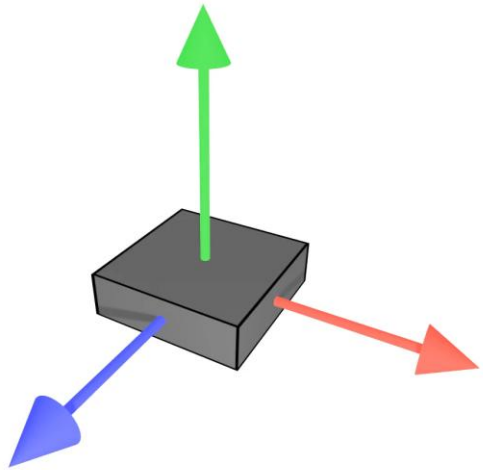




Pitch (x)



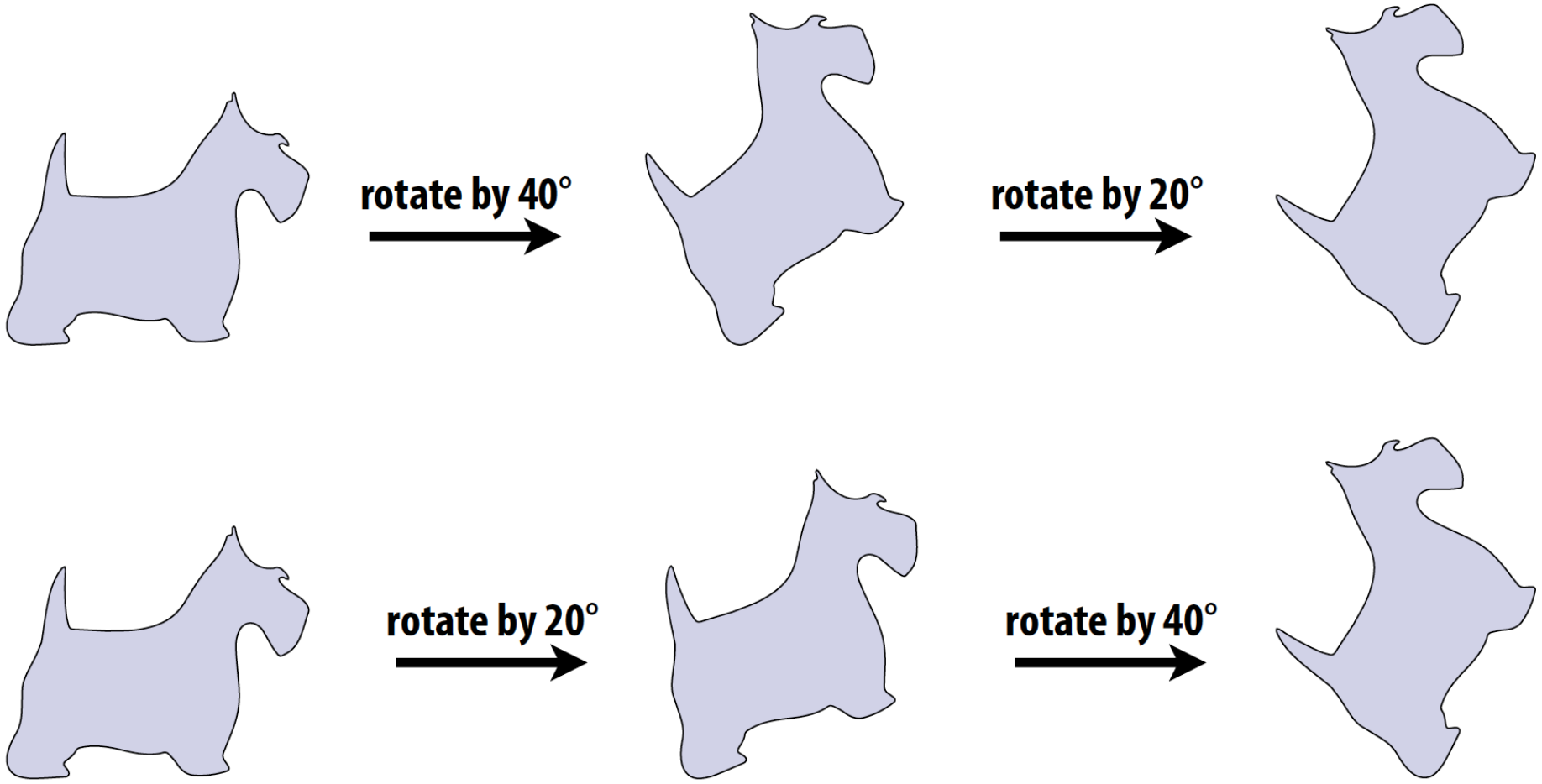
Yaw (y)



Roll (z)

Commutativity of 2D rotation

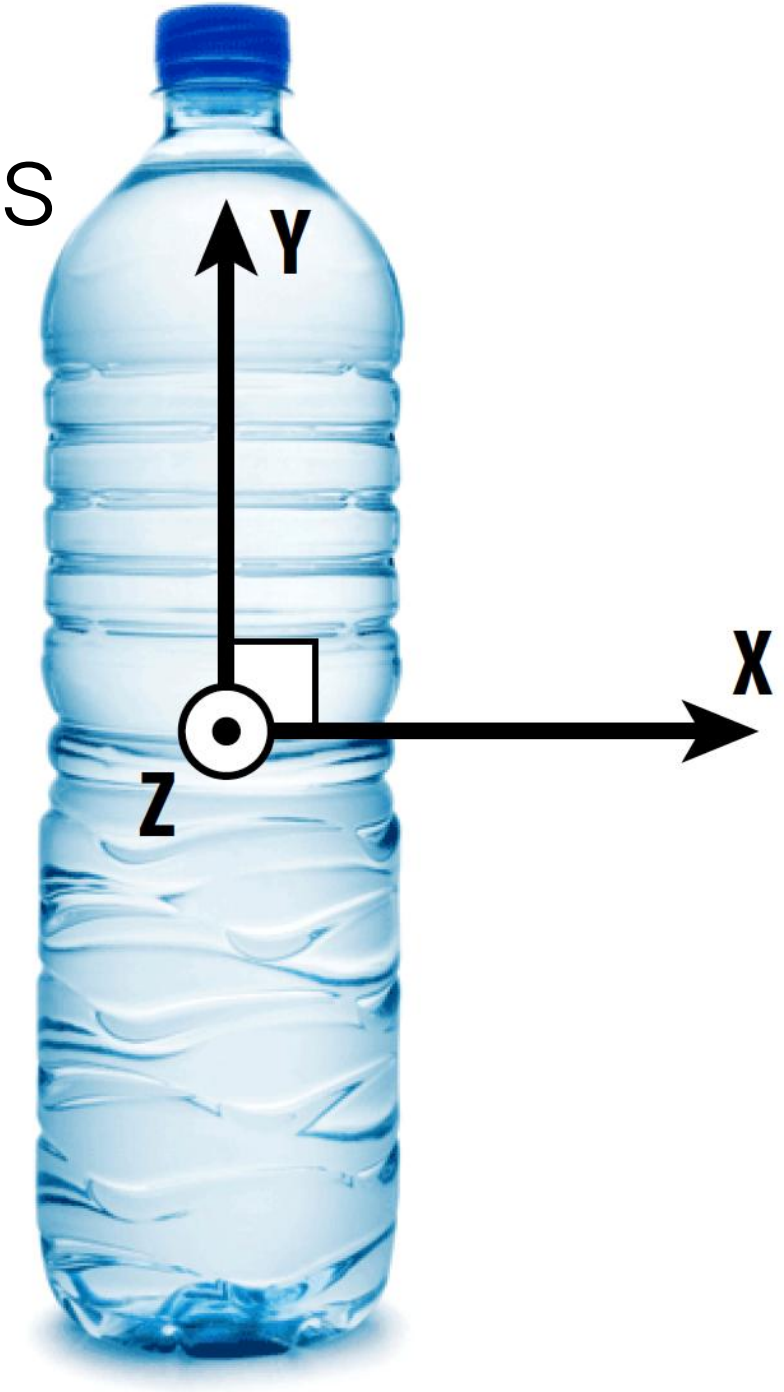
- In 2D, order of rotations doesn't matter:



Commutativity of 3D rotations

- What about in 3D?
  - Rotate 90° around Y, then 90° around Z, then 90° around X
  - Rotate 90° around Z, then 90° around Y, then 90° around X

- We need to be careful about the order in which we apply matrices!



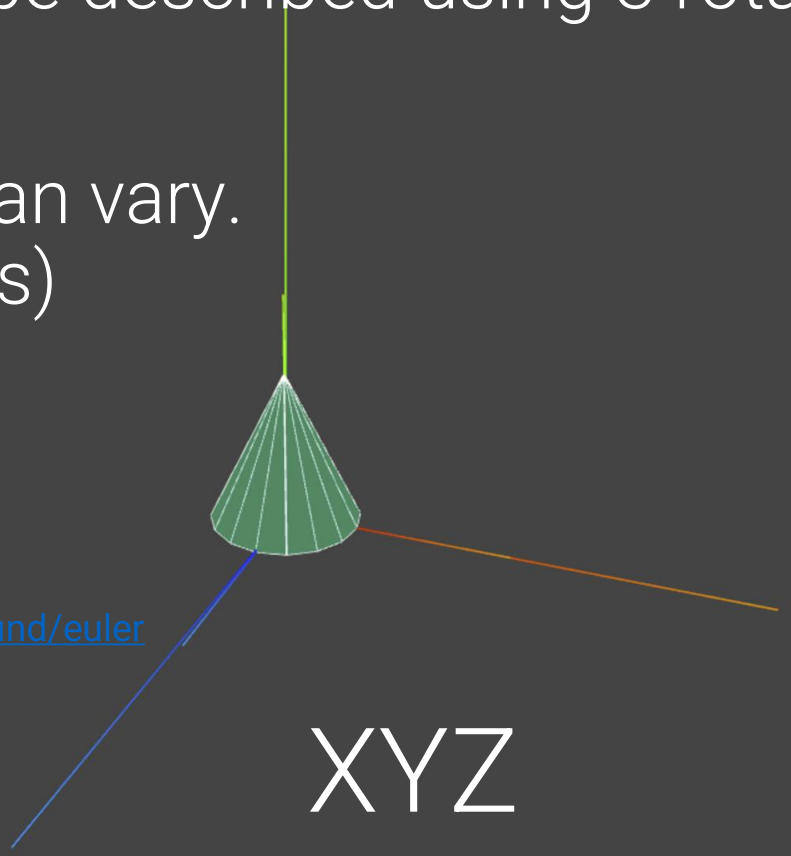
Euler Angles

- Euler’s Theorem, 1776:

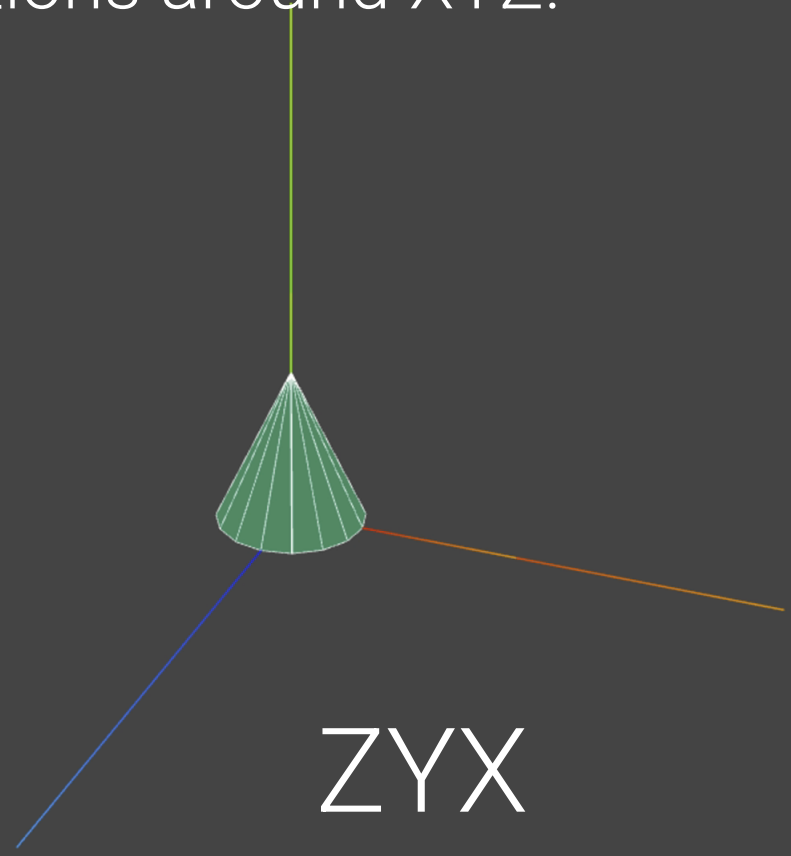
  „Any rotation can be described using 3 rotations around XYZ.“

- The rotation order can vary.
  (example: 45° all axis)

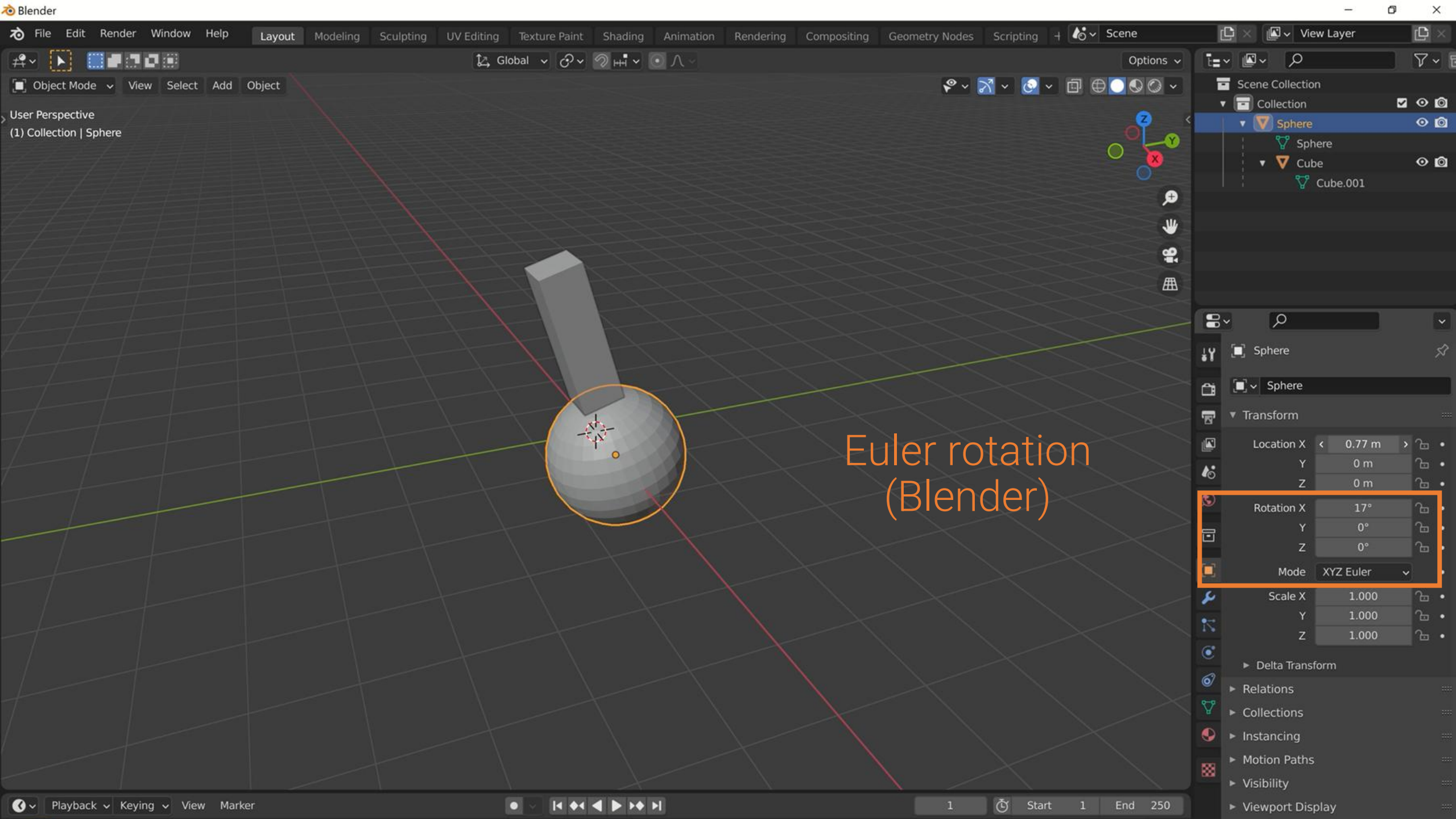
Try it yourself:
https://schedldave.github.io/gop_playground/euler



XYZ

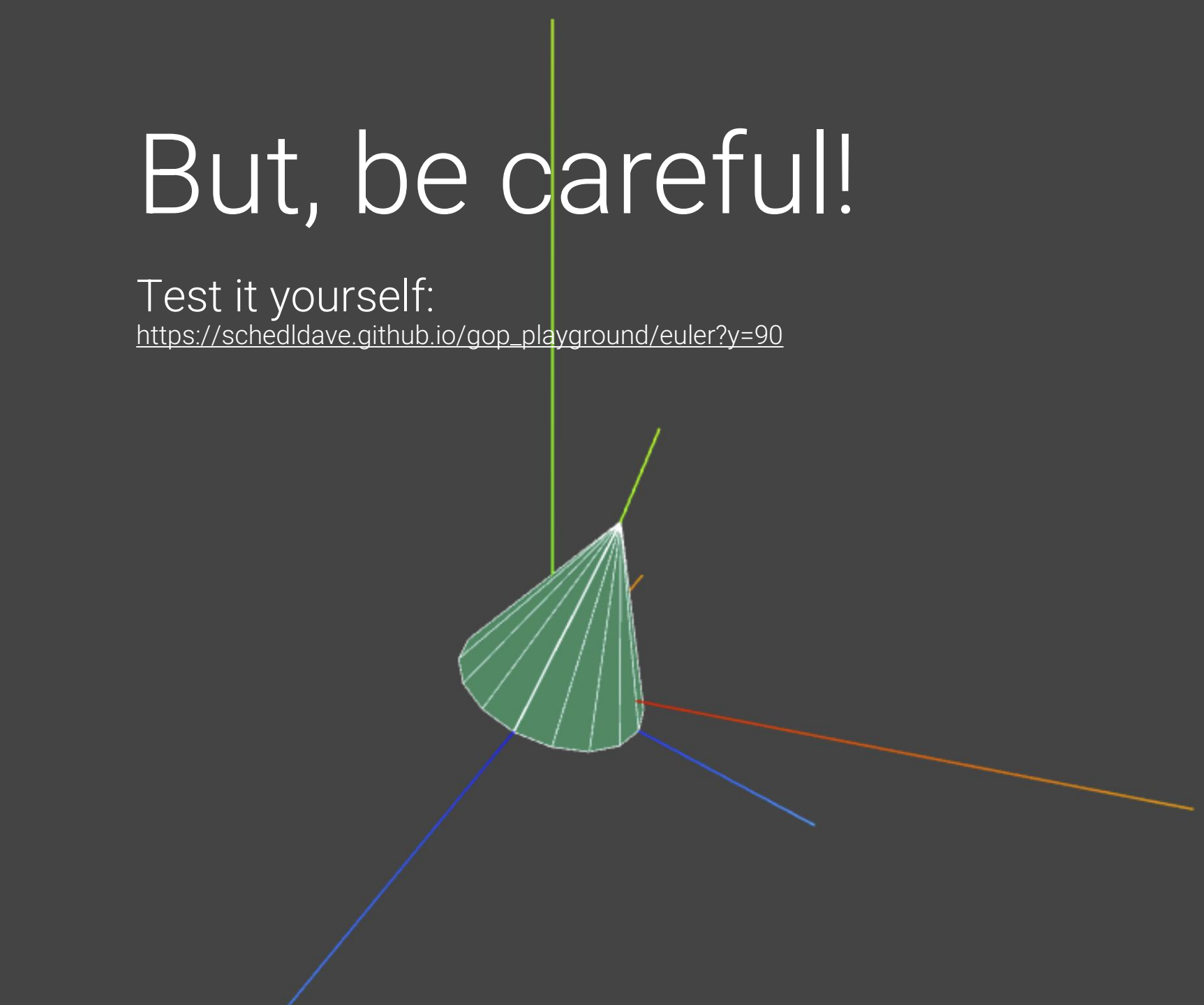


ZYX


Blender
File Edit Render Window Help
Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Scene
View Layer
Global
Options
Object Mode View Select Add Object
User Perspective
(1) Collection | Sphere
Scene Collection
Collection
Sphere
Sphere
Cube
Cube.001
Sphere
Sphere
Transform
Location X 0.77 m
Y 0 m
Z 0 m
Rotation X 17°
Y 0°
Z 0°
Mode XYZ Euler
Scale X 1.000
Y 1.000
Z 1.000
Delta Transform
Relations
Collections
Instancing
Motion Paths
Visibility
Viewport Display
Euler rotation
(Blender)
Playback Keying View Marker
1
Start 1
End 250

But, be careful!

Test it yourself:
https://schedldave.github.io/gop_playground/euler?y=90



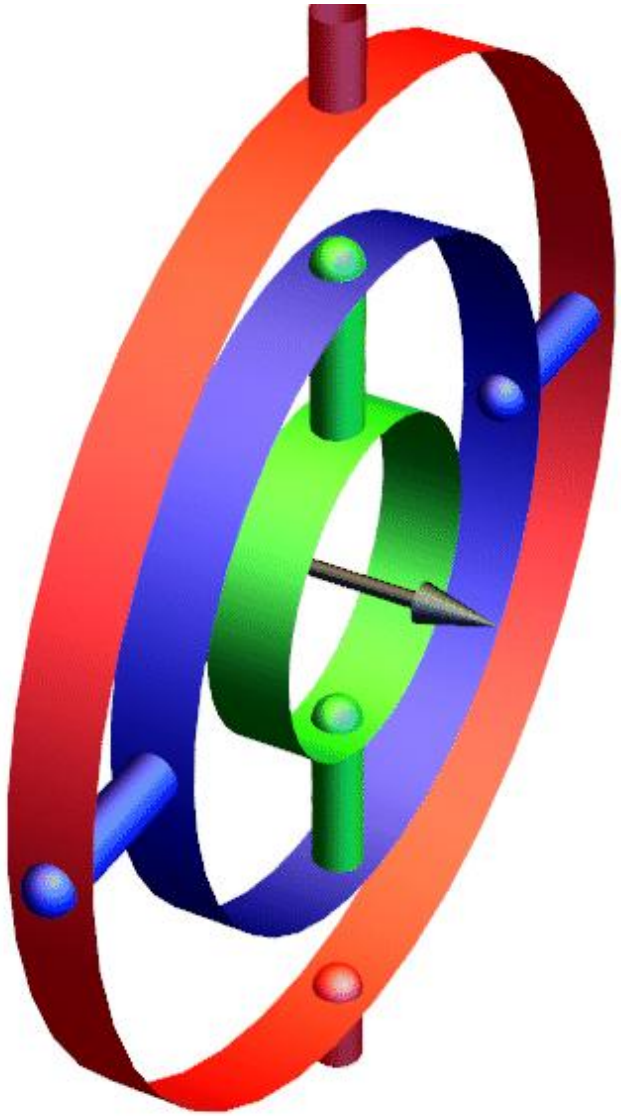
Controls

x	85
y	90
z	65
order	ZYX
Use Global Axis (reverse)	
animate	
t	3

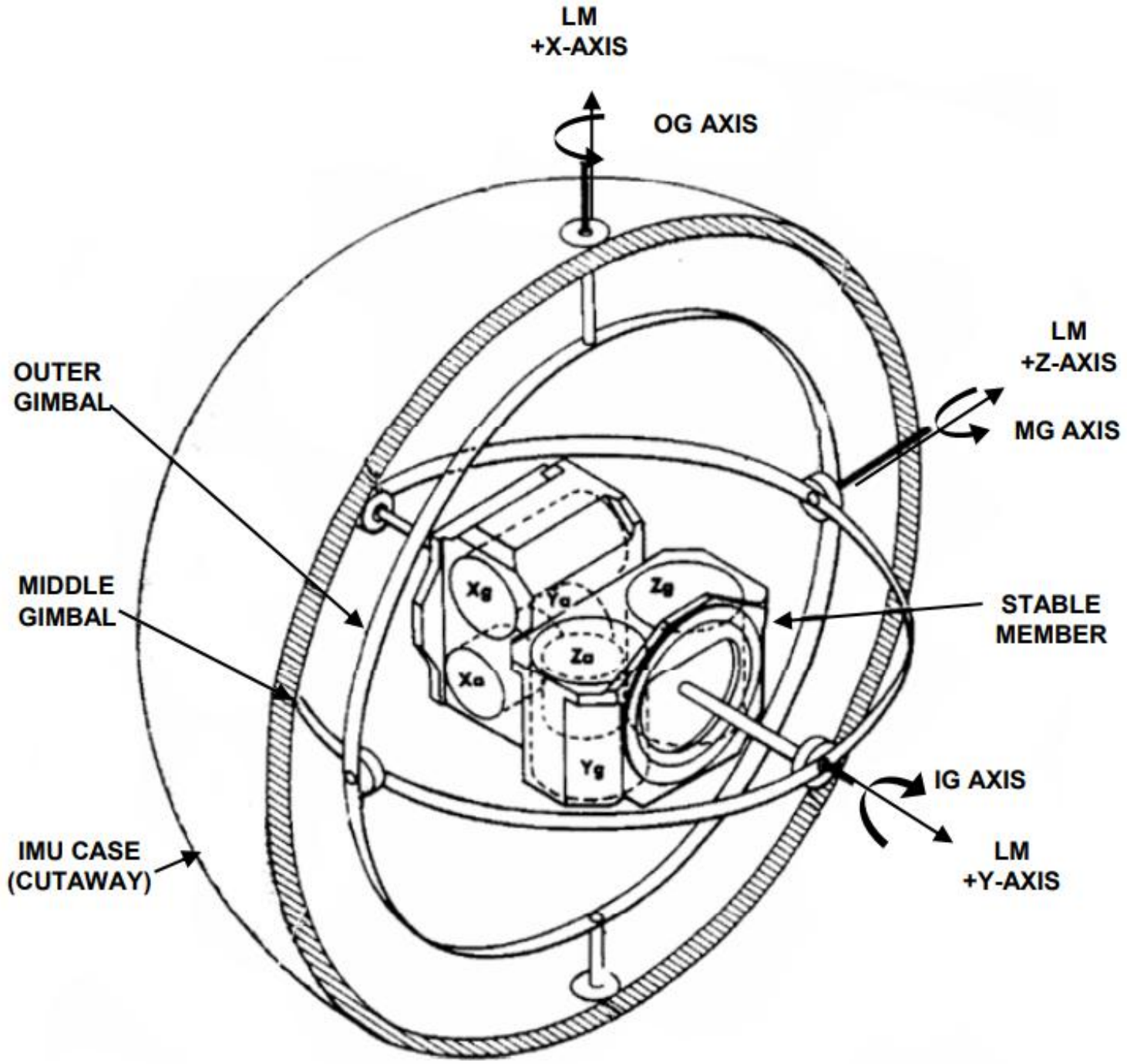
Gimbal Lock

- In some cases the rotation in one axis will be mapped onto another rotation axis.
  (e.g. 90° rotation of middle axis)
- It may become impossible to rotate an object in a desired axis.
- This is called **Gimbal lock**.

Test it yourself:
https://schedldave.github.io/gop_playground/euler?y=90

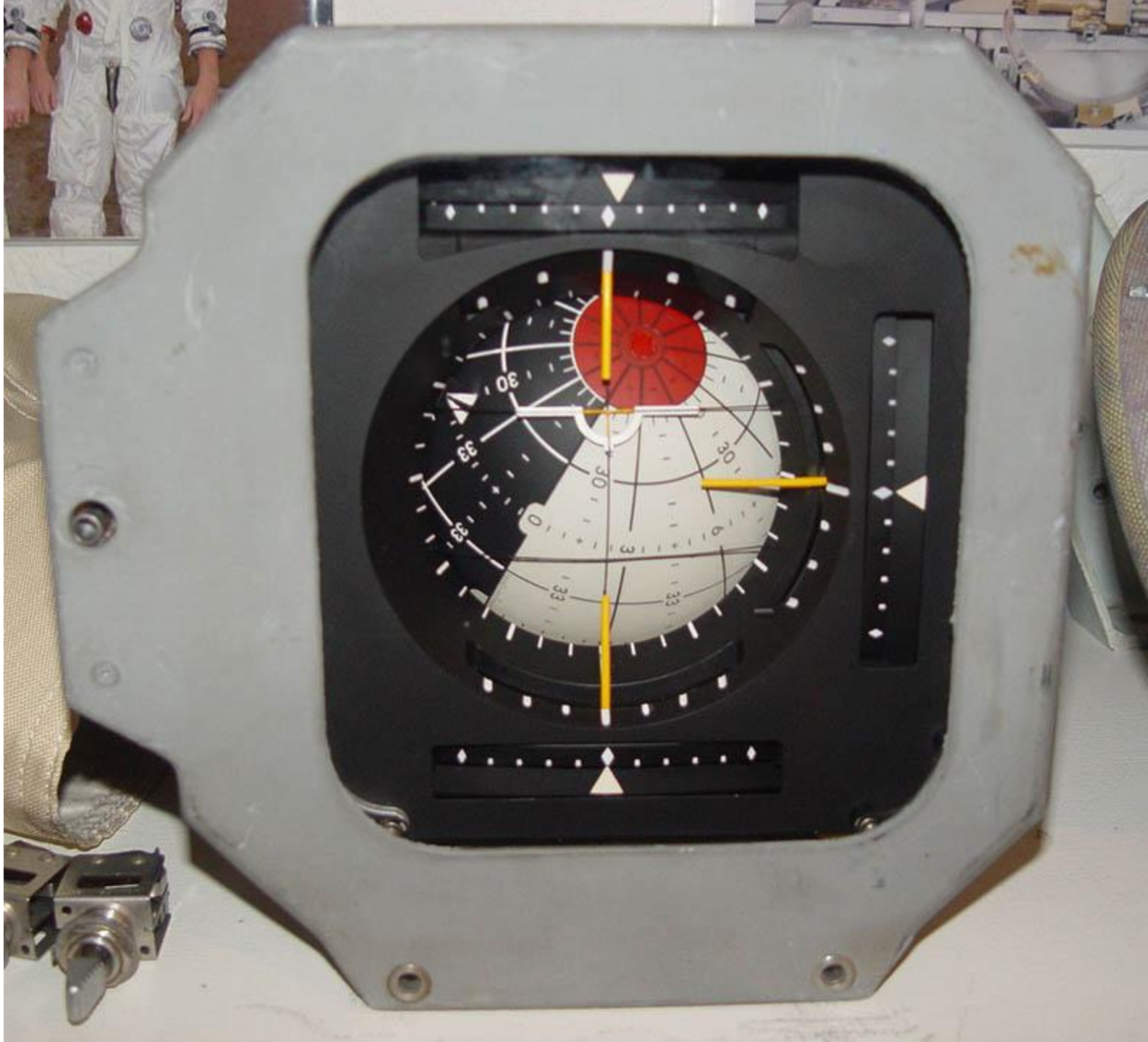


Apollo 11



https://apollo11space.com/apollo-and-gimbal-lock/

Apollo 11



https://apollo11space.com/apollo-and-gimbal-lock/

Quaternions

Quaternion - The Math Background in 2D

Complex Numbers:

$x^2 + 1 = 0$

$x^2 = i^2 = -1$

A complex point (2D)

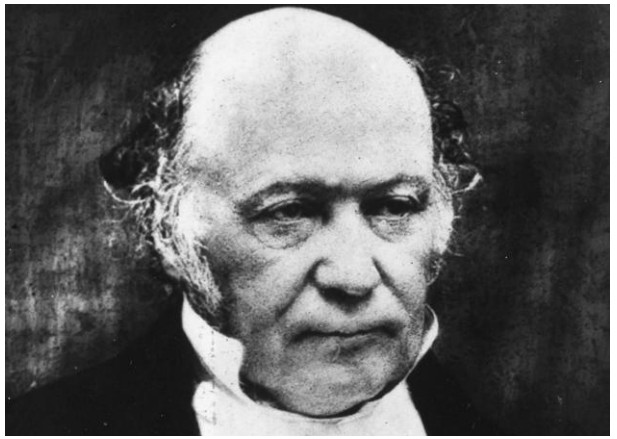
$p = 2 + i$

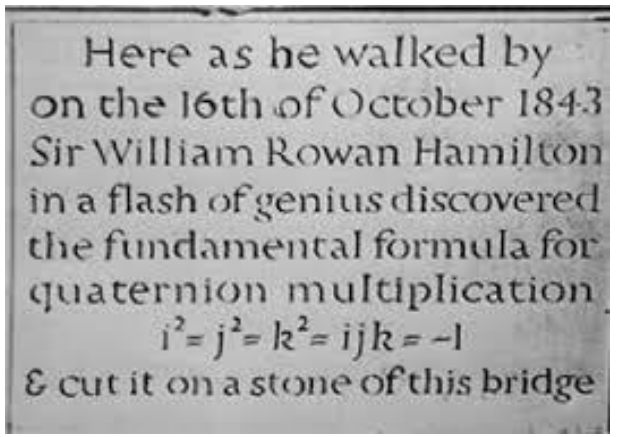
More general:

$p = a + b\,i$, where $a = 2, b = 1$

a is Real, and b is Imaginary

Sir William Rowan Hamilton



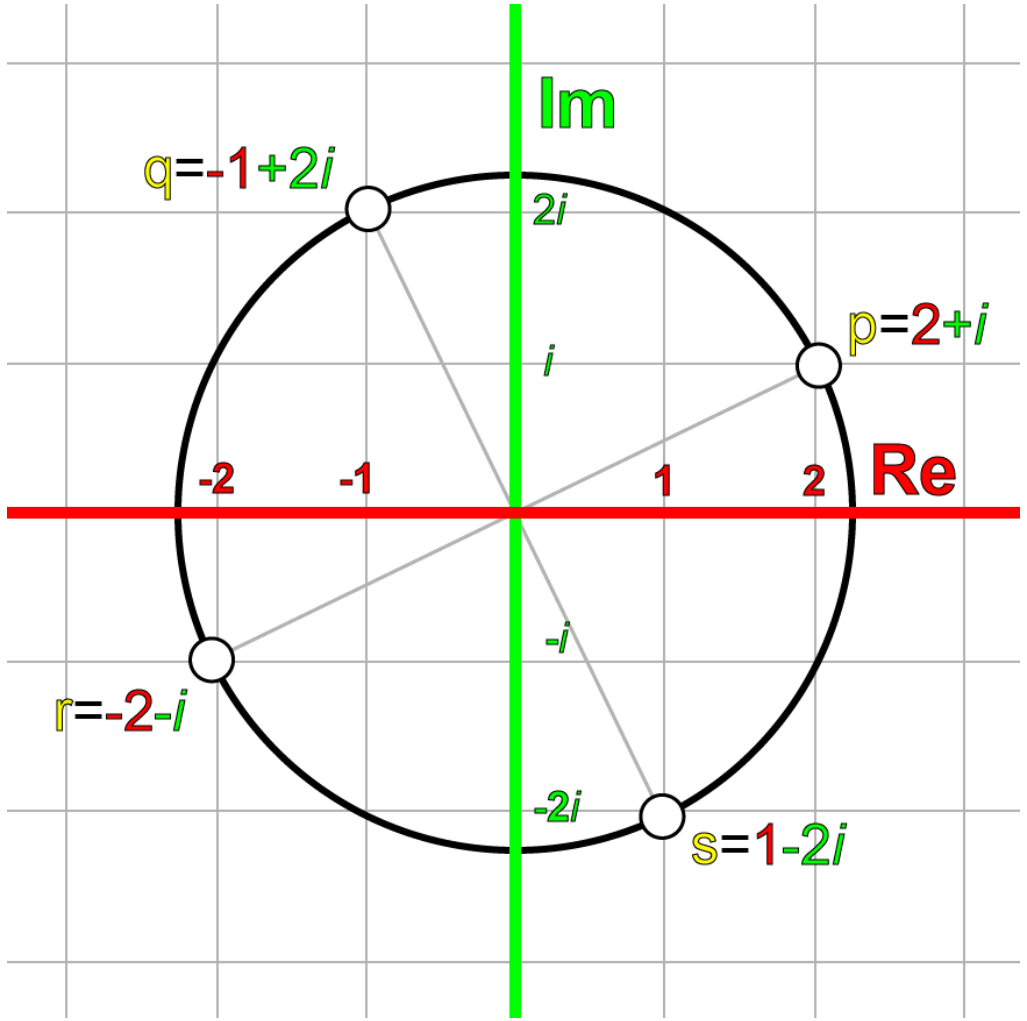


A point on a complex plane (2D)

90° rotations!

$$
\begin{aligned}
p &= 2+i \\
q &= pi \\
&= (2+i)i \\
&= 2i+i^2 \\
&= -1+2i
\end{aligned}
\qquad
\begin{aligned}
r &= -2-i \\
s &= ri
\end{aligned}
$$

$$
\begin{aligned}
q &= -1+2i \\
r &= qi \\
&= (-1+2i)i \\
&= -i+2i^2 \\
&= -2-i
\end{aligned}
\qquad
\begin{aligned}
s &= 1-2i \\
t &= si
\end{aligned}
$$



https://www.3dgep.com/understanding-quaternions/

Rotor in 2D

Arbitrary rotations (rotor):

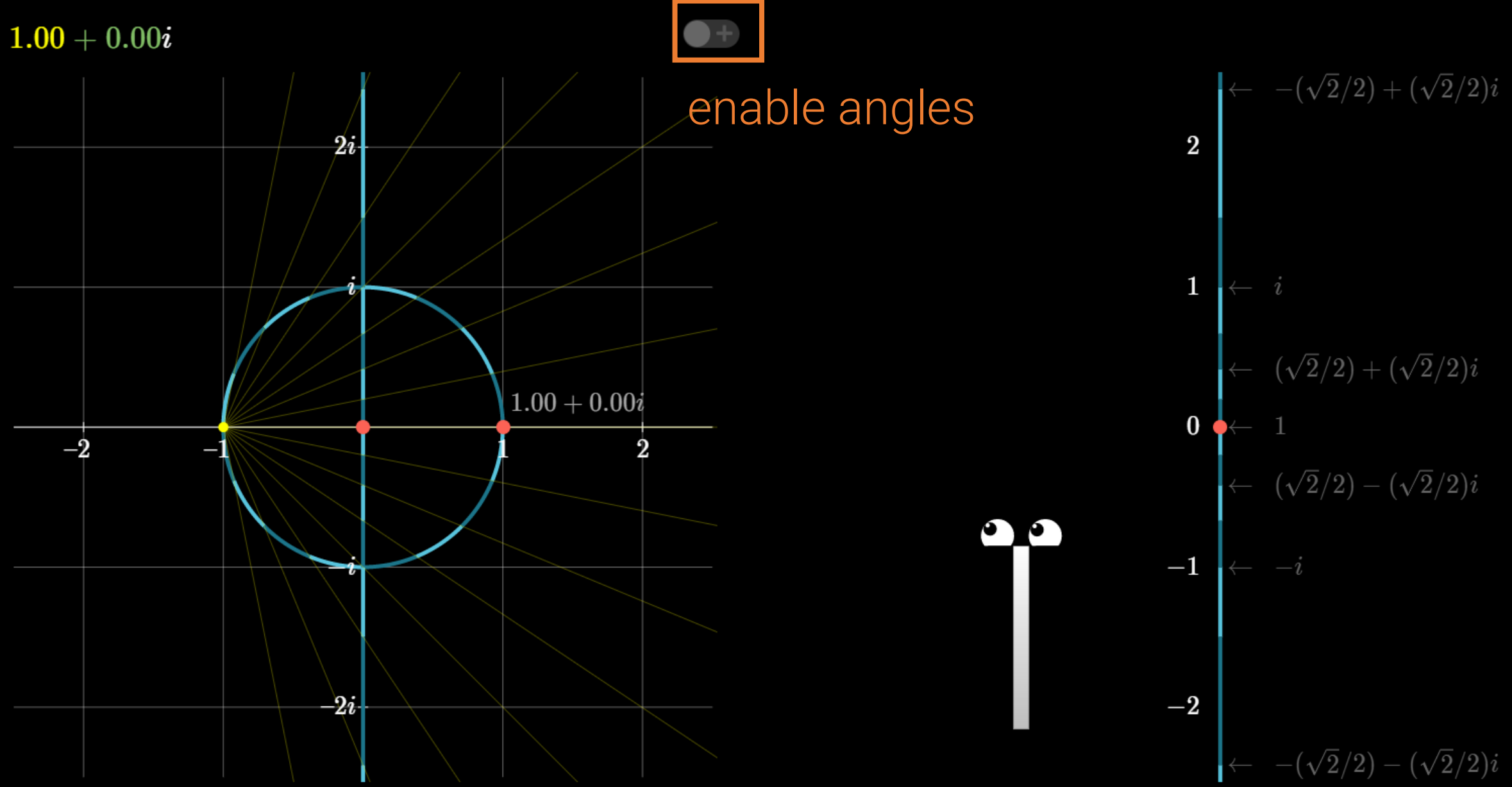
$$q = \cos\theta + i\sin\theta$$

Rotating a point:

$$\begin{aligned} p &= a + bi \\ q &= \cos\theta + i\sin\theta \\ pq &= (a + bi)(\cos\theta + i\sin\theta) \\ a' + b'i &= a\cos\theta - b\sin\theta + (a\sin\theta + b\cos\theta)i \end{aligned}$$

Remember our 2D rotation matrix!

$$R_\theta = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



https://eater.net/quaternions/video/intro

Conjugate and Complex Length

Conjugate

$$
\begin{aligned}
z &= (a+bi) \\
z^* &= (a-bi) \\
zz^* &= (a+bi)(a-bi) \\
&= a^2 - abi + abi + b^2 \\
&= a^2 + b^2
\end{aligned}
$$

Absolute (length)

$$
\begin{aligned}
z &= (a+bi) \\
|z| &= \sqrt{zz^*} \\
&= \sqrt{(a+bi)(a-bi)} \\
&= \sqrt{a^2+b^2}
\end{aligned}
$$

Rotor (unit length)

$$|z| = 1$$

Quaternion – Now 3D!

Quaternion:

$$q = a + bi + cj + dk \qquad a, b, c, d \in R$$

Real ($\in R$): a, b, c, d

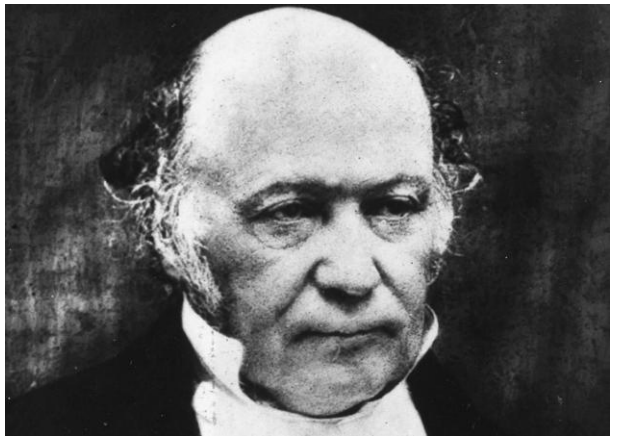
Imaginary: i, j, k

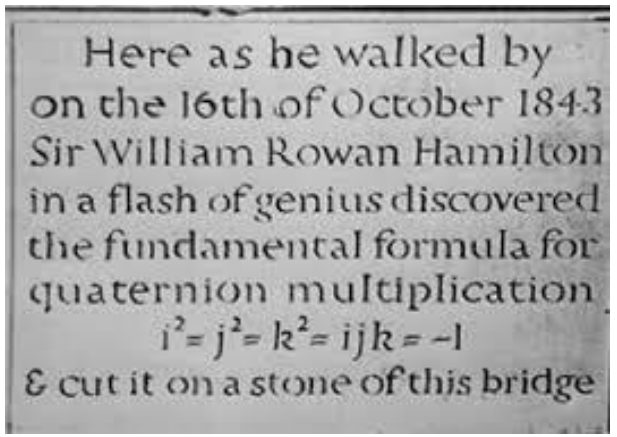
Some of Hamilton's expressions:

$$i^2 = j^2 = k^2 = ijk = -1$$

$$ij = k, \qquad jk = i, \qquad ki = j$$
$$ji = -k, \qquad kj = -i, \qquad ik = -j$$

Sir William Rowan Hamilton



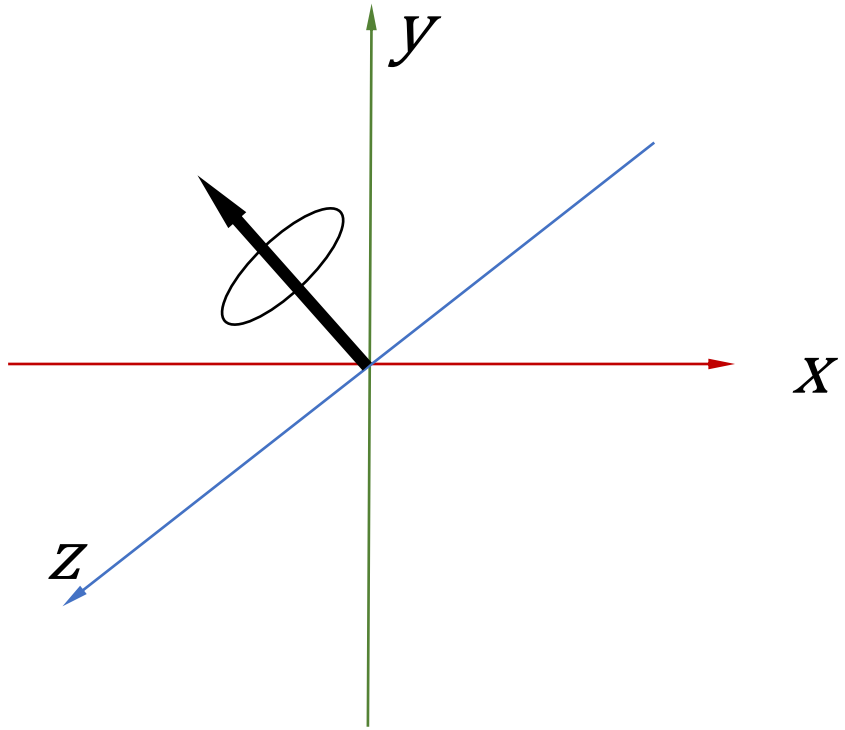


Quaternions – as we use them in graphics

$$q_1 = (a, b, c, d)$$

scalar

vector



Unit Norm

$$a^2 + b^2 + c^2 + d^2 = 1$$

Ken Shoemake. 1985. Animating rotation with quaternion curves. In Proceedings of the 12th annual conference on Computer graphics and interactive techniques (SIGGRAPH '85). Association for Computing Machinery, New York, NY, USA, 245–254. DOI: https://doi.org/10.1145/325334.325242

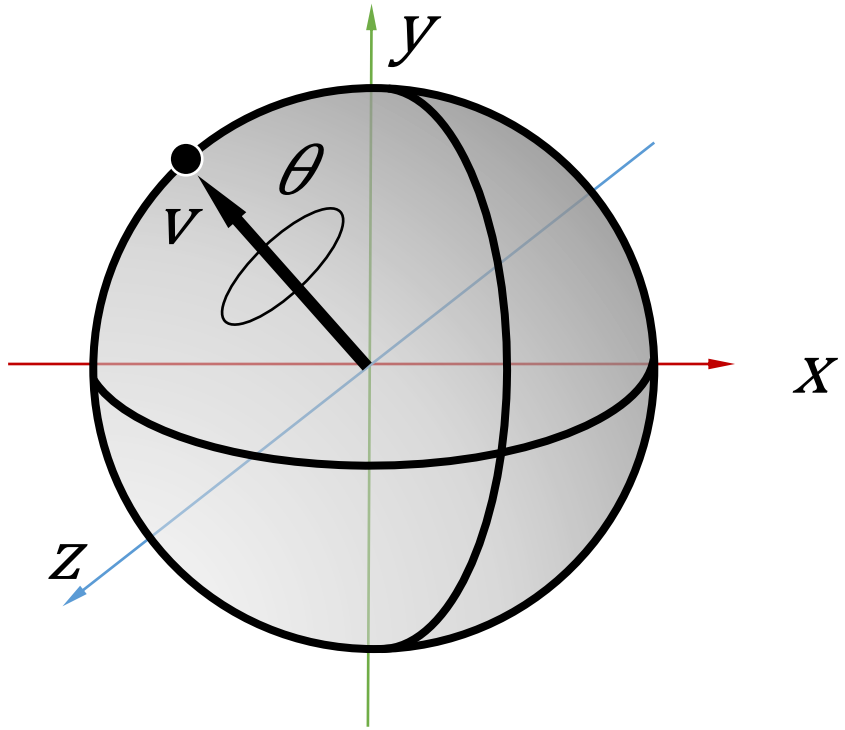
Quaternions – Geometric interpretation

$$\boldsymbol{v} = (v_1, v_2, v_3)$$

$$q_1 = (\theta, \boldsymbol{v})$$

$$= \left(\cos\left(\frac{\theta}{2}\right), v_1 \sin\left(\frac{\theta}{2}\right), v_2 \sin\left(\frac{\theta}{2}\right), v_3 \sin\left(\frac{\theta}{2}\right)\right)$$

$$= (\quad a, \qquad b, \qquad c, \qquad d \quad)$$



$$a^2 + b^2 + c^2 + d^2 = 1$$

$$\cos^2\left(\frac{\theta}{2}\right) + \sin^2\left(\frac{\theta}{2}\right)\left(v_1^2 + v_2^2 + v_3^2\right) = 1$$

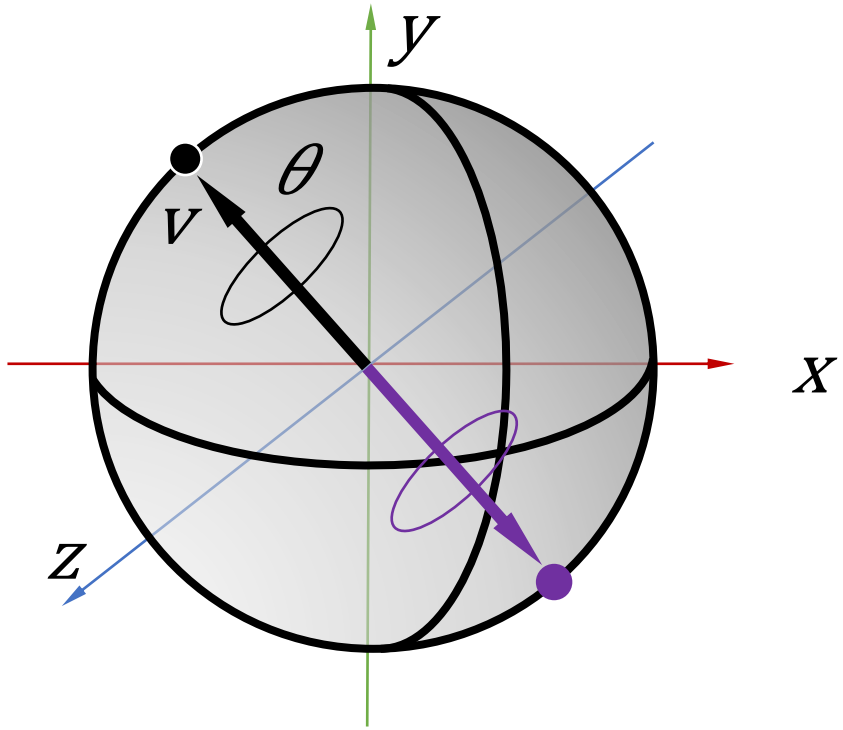
$$|\boldsymbol{v}| = \left(v_1^2 + v_2^2 + v_3^2\right) = 1 \qquad \cos^2\left(\frac{\theta}{2}\right) + \sin^2\left(\frac{\theta}{2}\right) \cdot 1 = 1$$

Quaternions – Relations

$$q_1 = (a, b, c, d) \quad \Leftrightarrow \quad q_2 = (-a, -b, -c, -d)$$

$\Updownarrow$ conjugate $\qquad\qquad$ $\Updownarrow$ conjugate

$$q_3 = (a, -b, -c, -d) \quad \Leftrightarrow \quad q_4 = (-a, b, c, d)$$



For unit quaternions: conjugate is inverse

$$q_1 q_1^{-1} = (1, 0, 0, 0) = 1$$

Quaternion - Algebra

$q_1 = (a, b, c, d)$ $\quad$ $q_2 = (e, f, g, h)$

$q_1 q_2 = ?$

$$q_1 q_2 = (ae - bf - cg - dh),$$
$$(af + be + ch - dg),$$
$$(ag - bh + ce + df),$$
$$(ah + bg - cf + de)$$

Quaternion - Multiplication

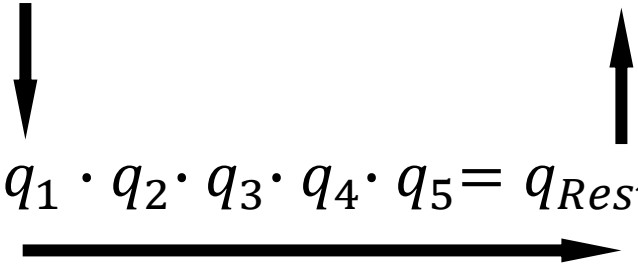
not commutative

$q_1 = (a, b, c, d)$ $q_2 = (e, f, g, h)$

$q_1 q_2 \neq q_2 q_1$

Rotation matrices to quaternions:

$R_1 \cdot R_2 \cdot R_3 \cdot R_4 \cdot R_5 = R_{Result}$



$q_1 \cdot q_2 \cdot q_3 \cdot q_4 \cdot q_5 = q_{Result}$

Quaternion – Rotating a vertex/point

3D points are pure quaternions (no real part):

$$p = (0, x, y, z)$$

The rotation with an arbitrary unit-length rotor q is then:

$$p' = qpq^{-1}$$

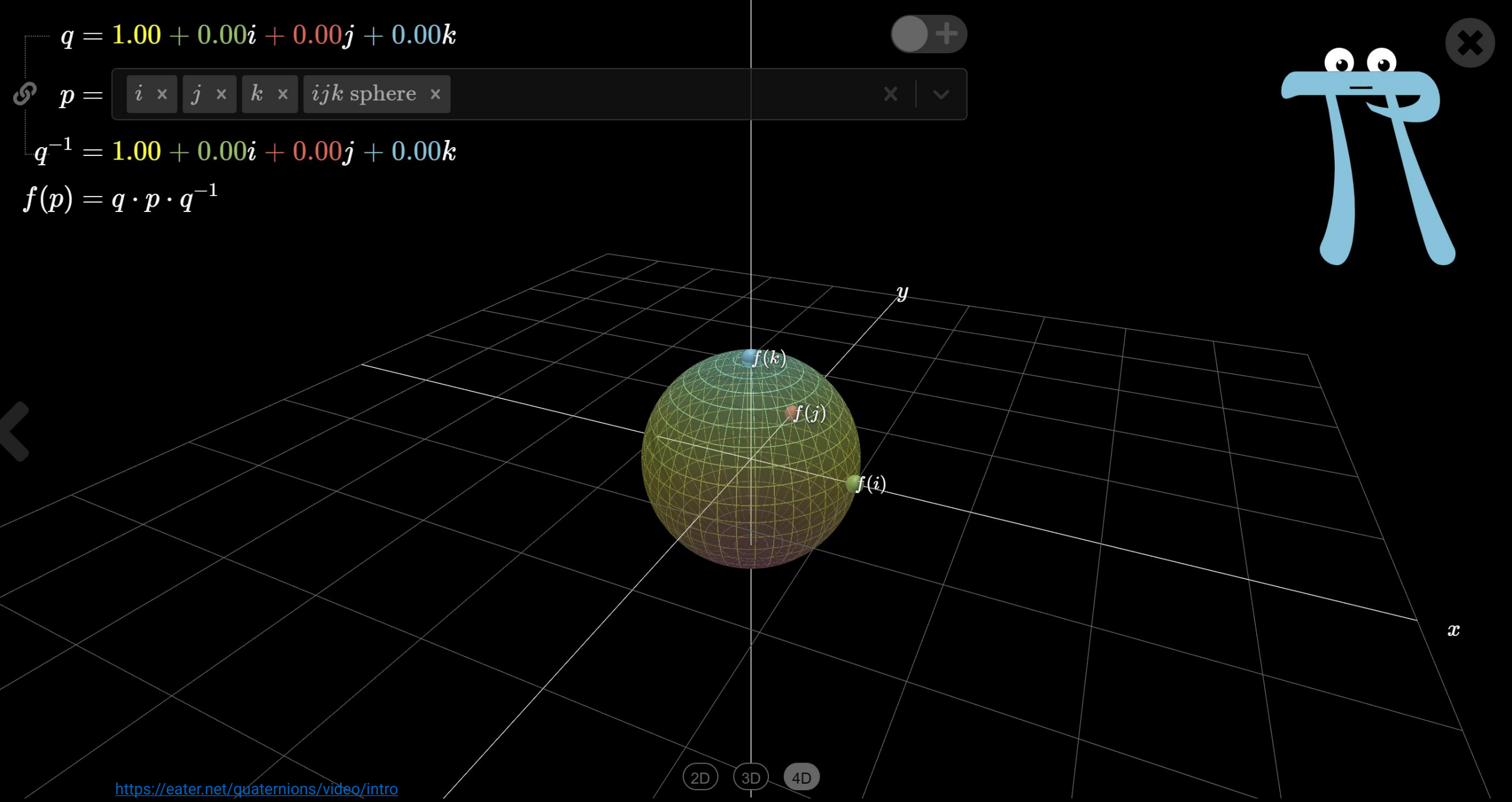
Yes, rotor and inverse rotor!

$q = 1.00 + 0.00i + 0.00j + 0.00k$

$p =$ i × j × k × ijk sphere ×

$q^{-1} = 1.00 + 0.00i + 0.00j + 0.00k$

$f(p) = q \cdot p \cdot q^{-1}$



https://eater.net/quaternions/video/intro

Quaternions – Conclusion

- No Gimbal Lock
- Easy to interpolate (SLERP and SQUAD)
- Typically, faster than matrices (multiplication, inverse, …)
- Most Engines (including three.js) use them internally.

- ☹ Can be cumbersome to understand/interpret!

Quaternion Resources

Interactive Demo:

- https://eater.net/quaternions

Websites/Articles with more Math details:

- https://www.haroldserrano.com/blog/quaternions-in-computer-graphics
- https://www.3dgep.com/understanding-quaternions/
- https://web.mit.edu/2.998/www/QuaternionReport1.pdf



3D Camera transformation

📷

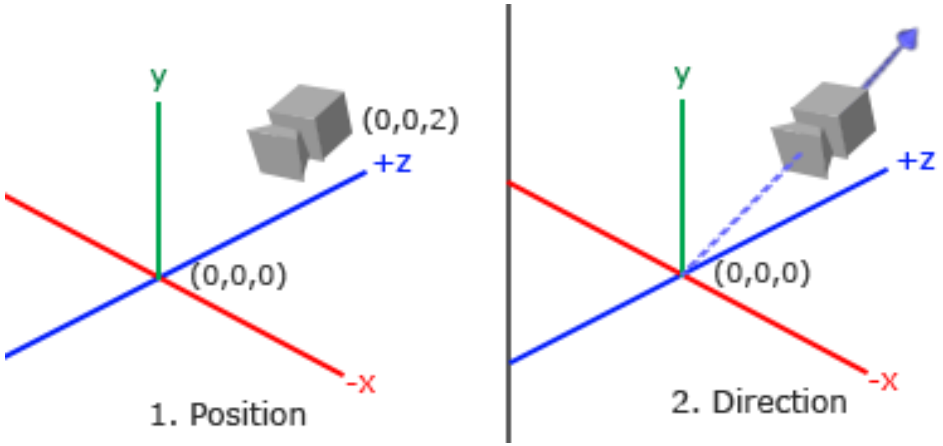
Camera Space

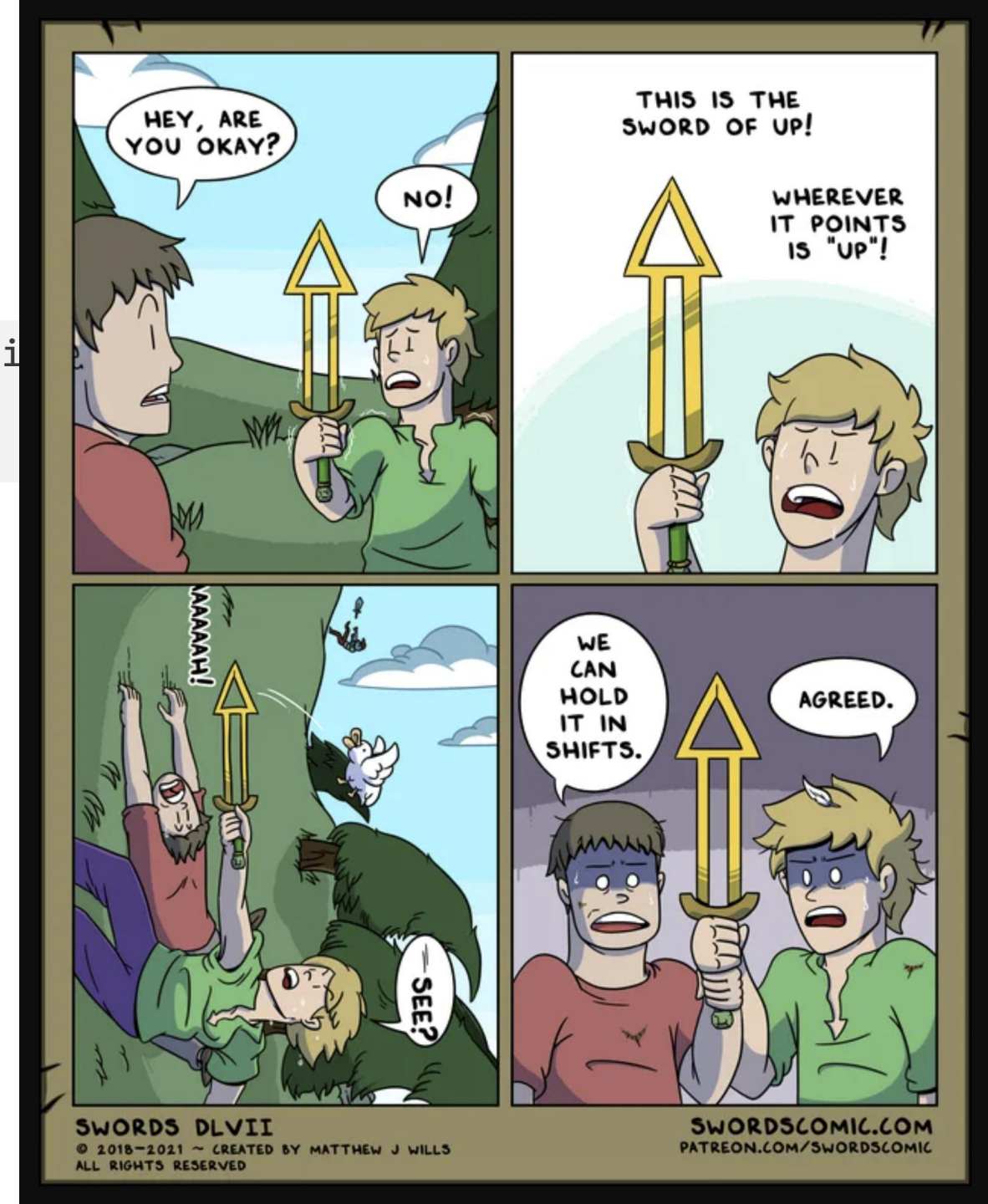
- It’s all relative to the camera… (like in 2D)
- Moving camera is equivalent to moving every object in the world towards a stationary camera
- How to look at something?



Camera LookAt

```
const eye = new Vector3(0,1,2); // camera posi
const target = new Vector3(0,0,0);
const up = new Vector3(0,1,0);
```

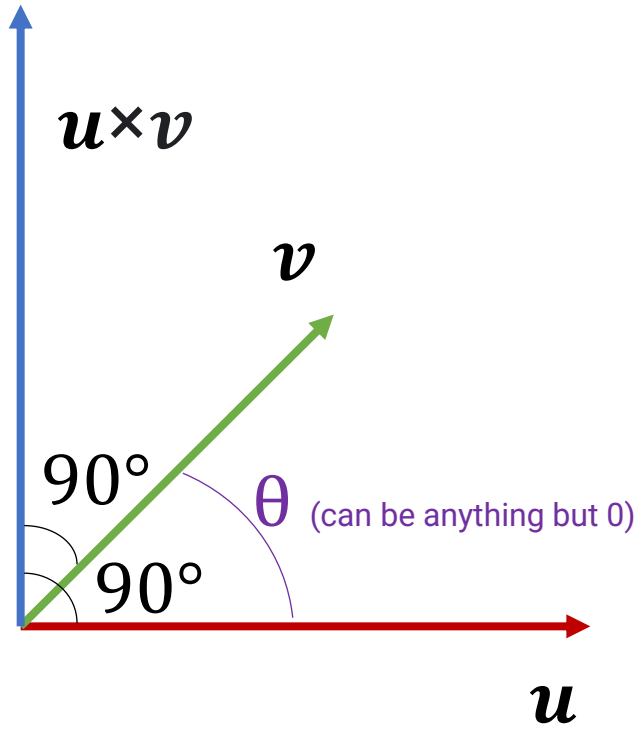




Cross product (×) of 2 Vectors

$$\boldsymbol{u} = \begin{bmatrix} a \\ b \\ c \end{bmatrix} \quad \boldsymbol{v} = \begin{bmatrix} d \\ e \\ f \end{bmatrix}$$

$$\boldsymbol{u}\times\boldsymbol{v} = \begin{bmatrix} bf - ce \\ cd - af \\ ae - bd \end{bmatrix}$$

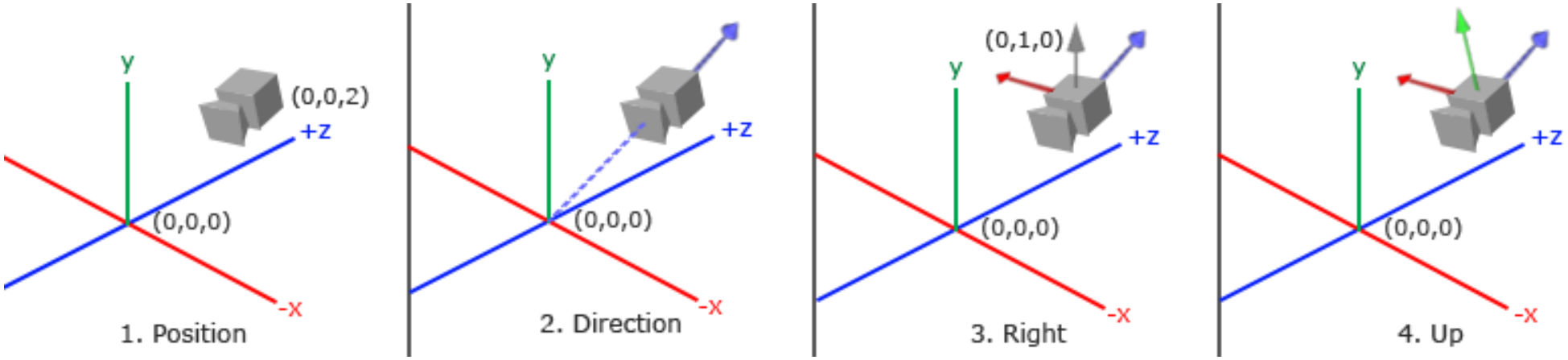


Camera LookAt

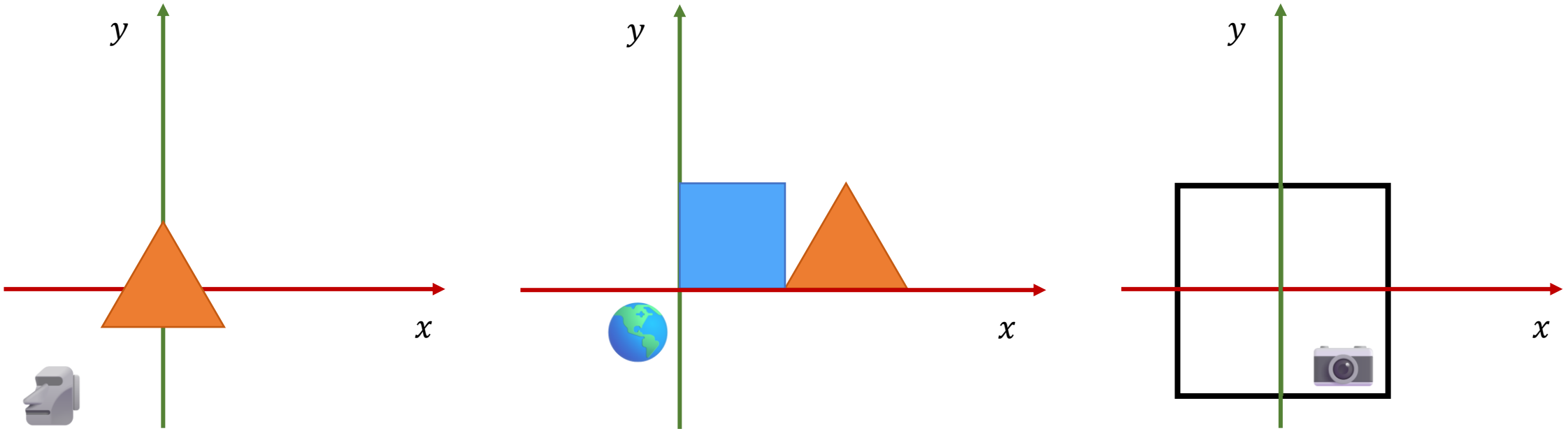
For further details watch:
https://youtu.be/G6skrOtJtbM

```
const eye = new Vector3(0,1,2); // camera position
const target = new Vector3(0,0,0);
const up = new Vector3(0,1,0);

const z = eye – target;
const x = up x z; // cross product
const y = z x x; // cross product
```



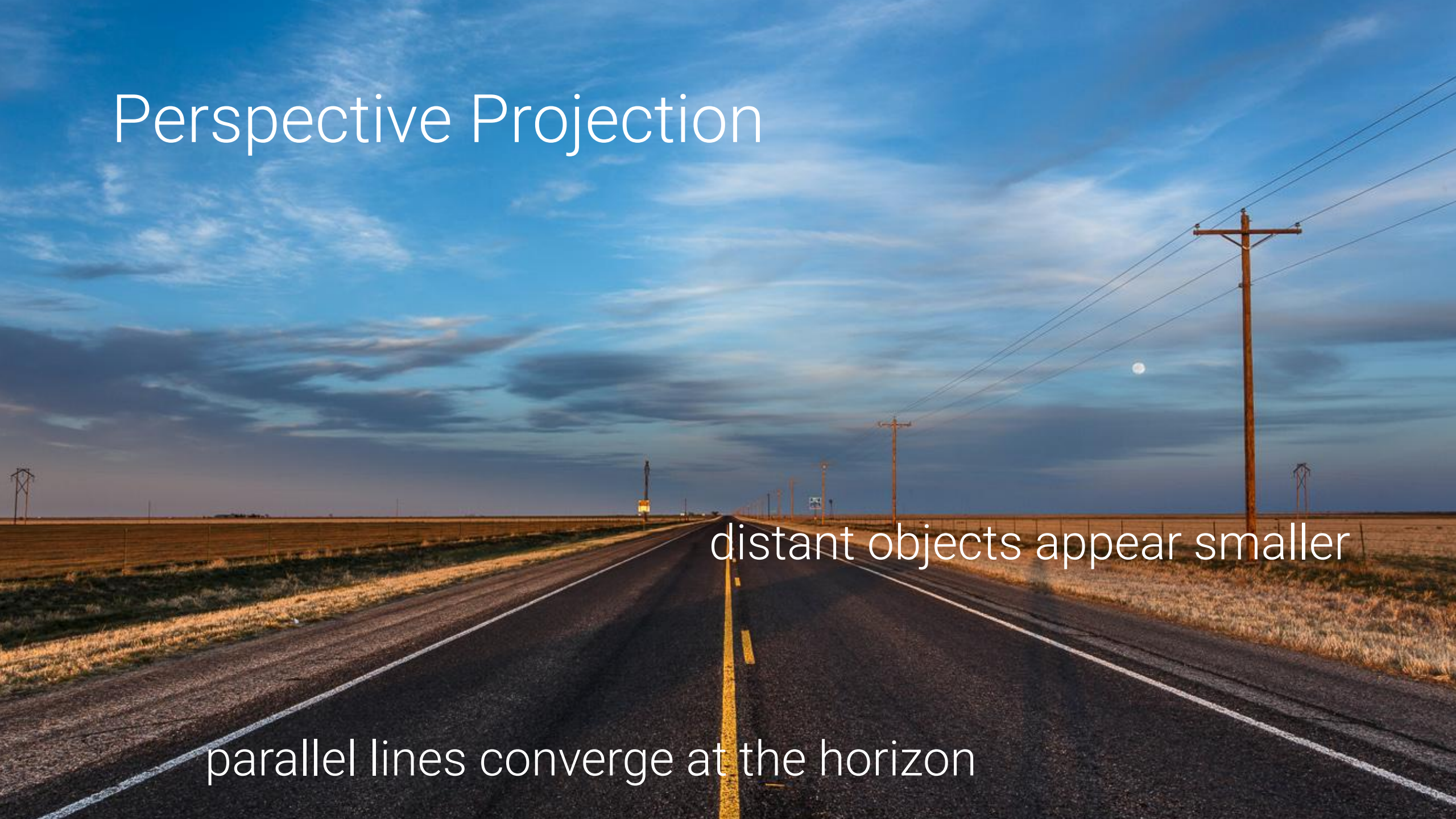
Transformation so far (like 2D)



🗿 ➔ 🌎 ➔ 📷

model-view matrix

Perspective Projection



distant objects appear smaller

parallel lines converge at the horizon

Early painting

- Incorrect perspective
- Carolingian painting from the 8-9th century





Giotto 1290

Evolution toward correct perspective



Ambrogio Lorenzetti
Annunciation, 1344



Brunelleschi, elevation of Santo Spirito,
1434-83, Florence



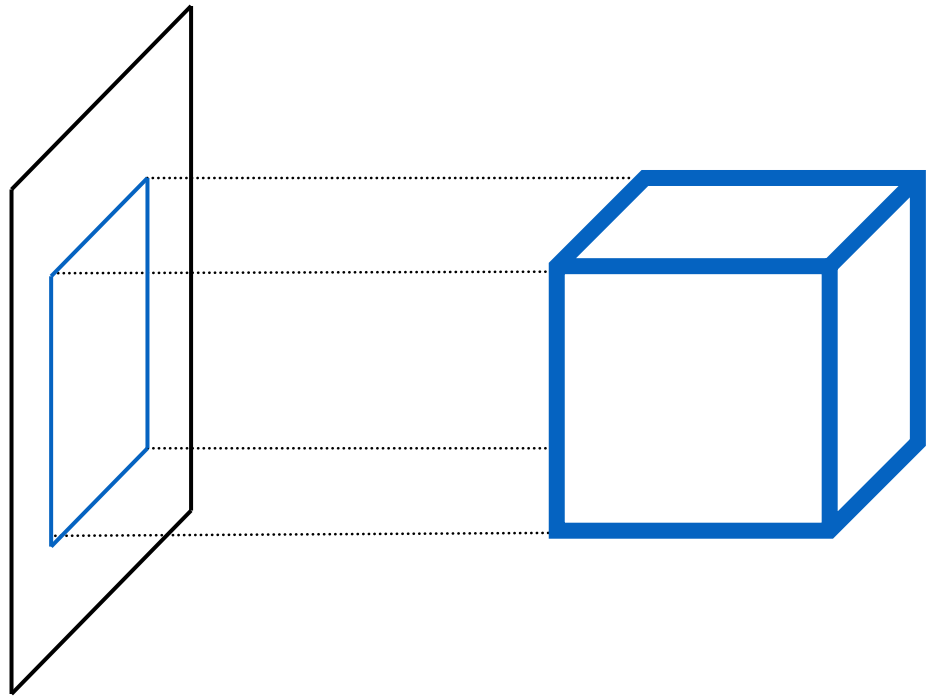
Delivery of the Keys (Sistine Chapel), Perugino, 1482

Rejection of perspective in CG



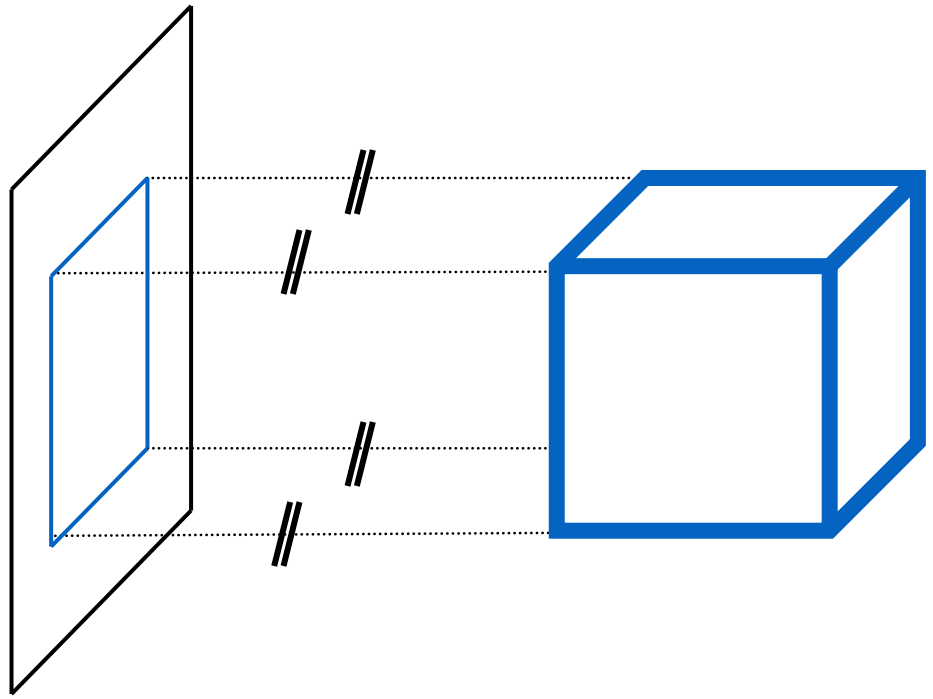
Parallel projection

- Projection of the coordinates from $\mathfrak{R}^3$ to $\mathfrak{R}^2$



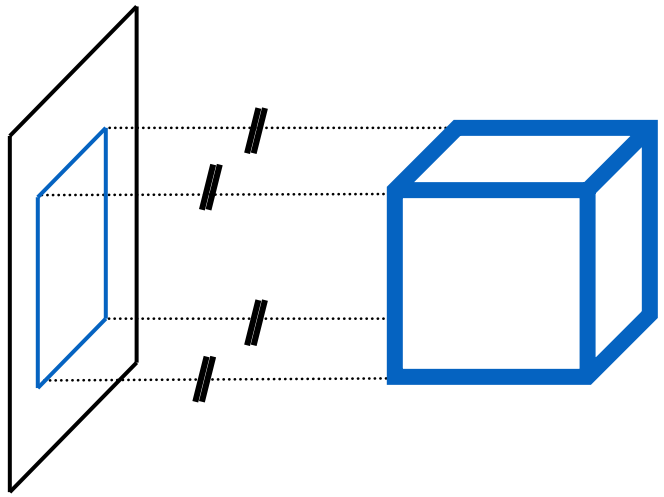
Parallel projection

- Projection rays are parallel

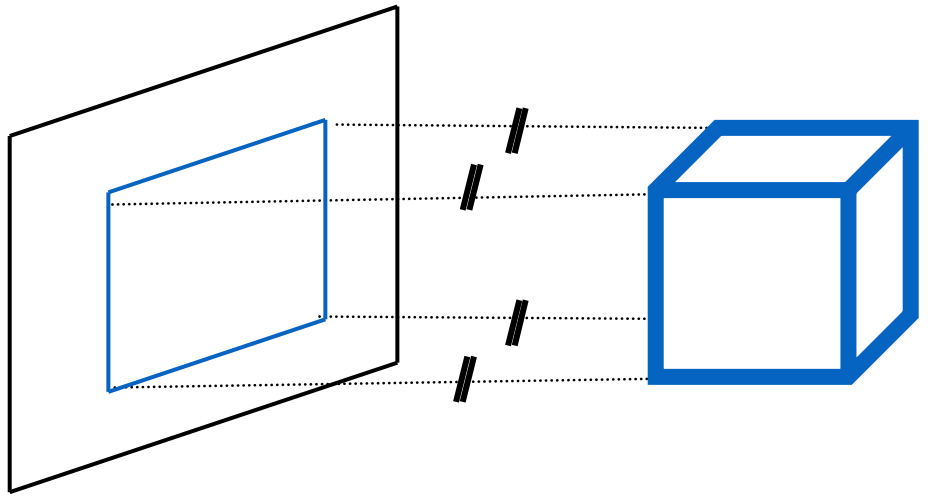


Parallel projection 2

Orthogonal

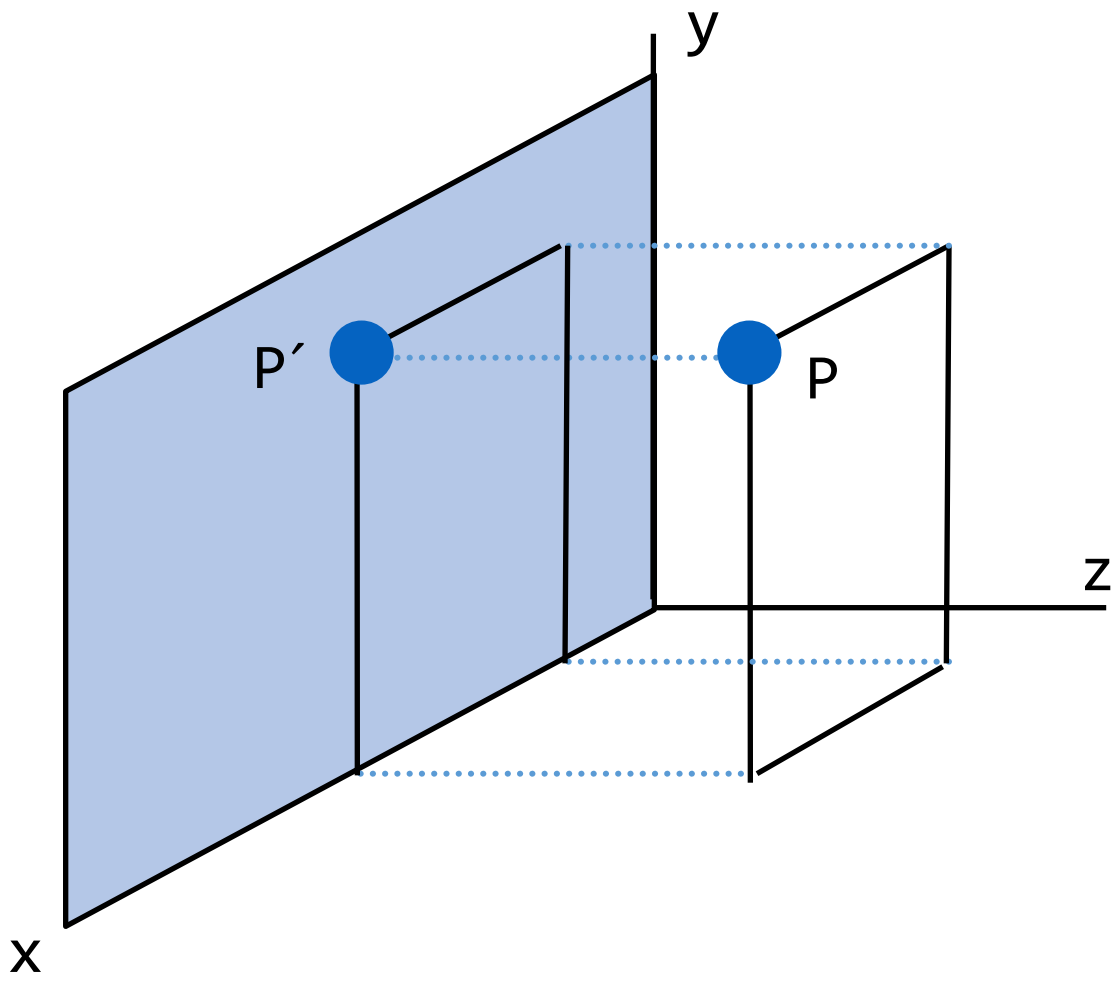


Oblique projection („Schrägriss“)



Orthogonal Projection

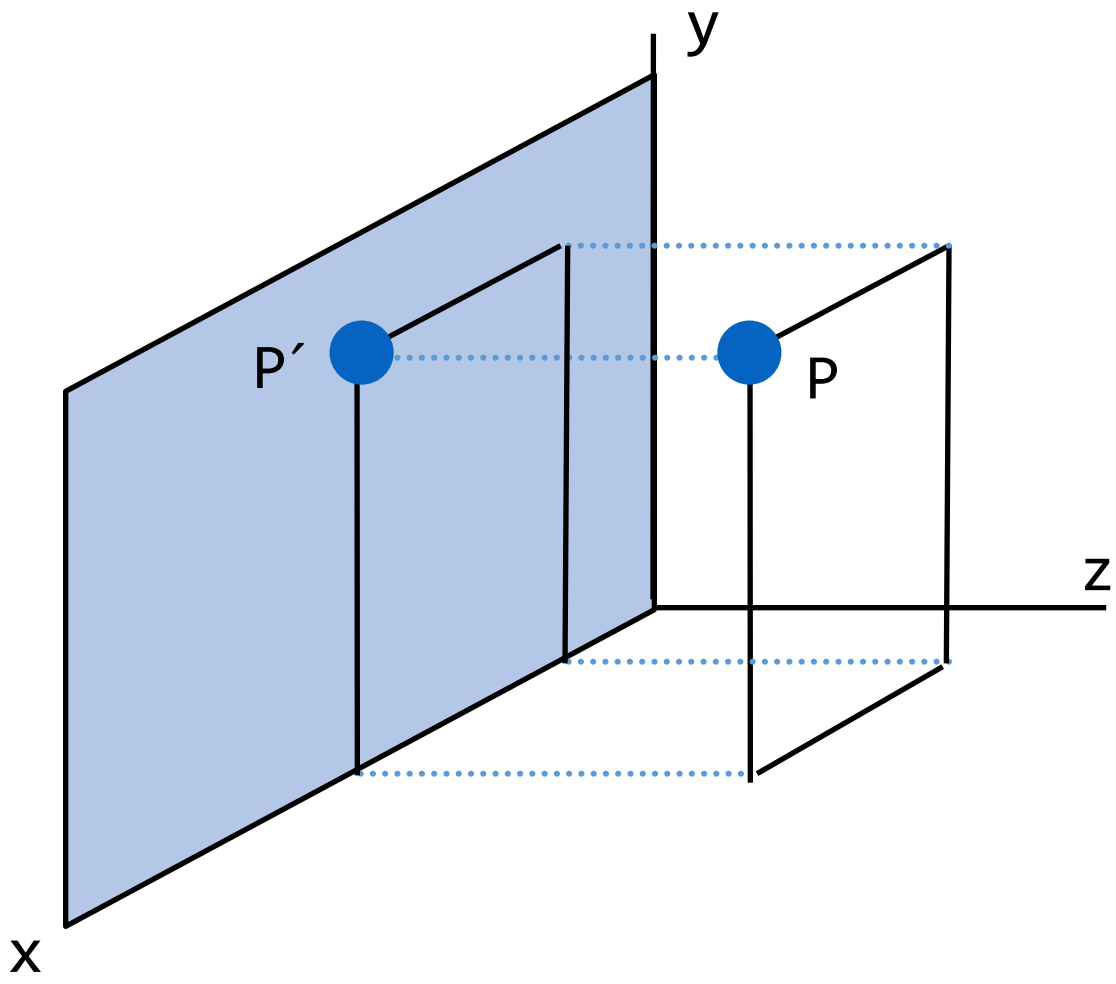
$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix}$$



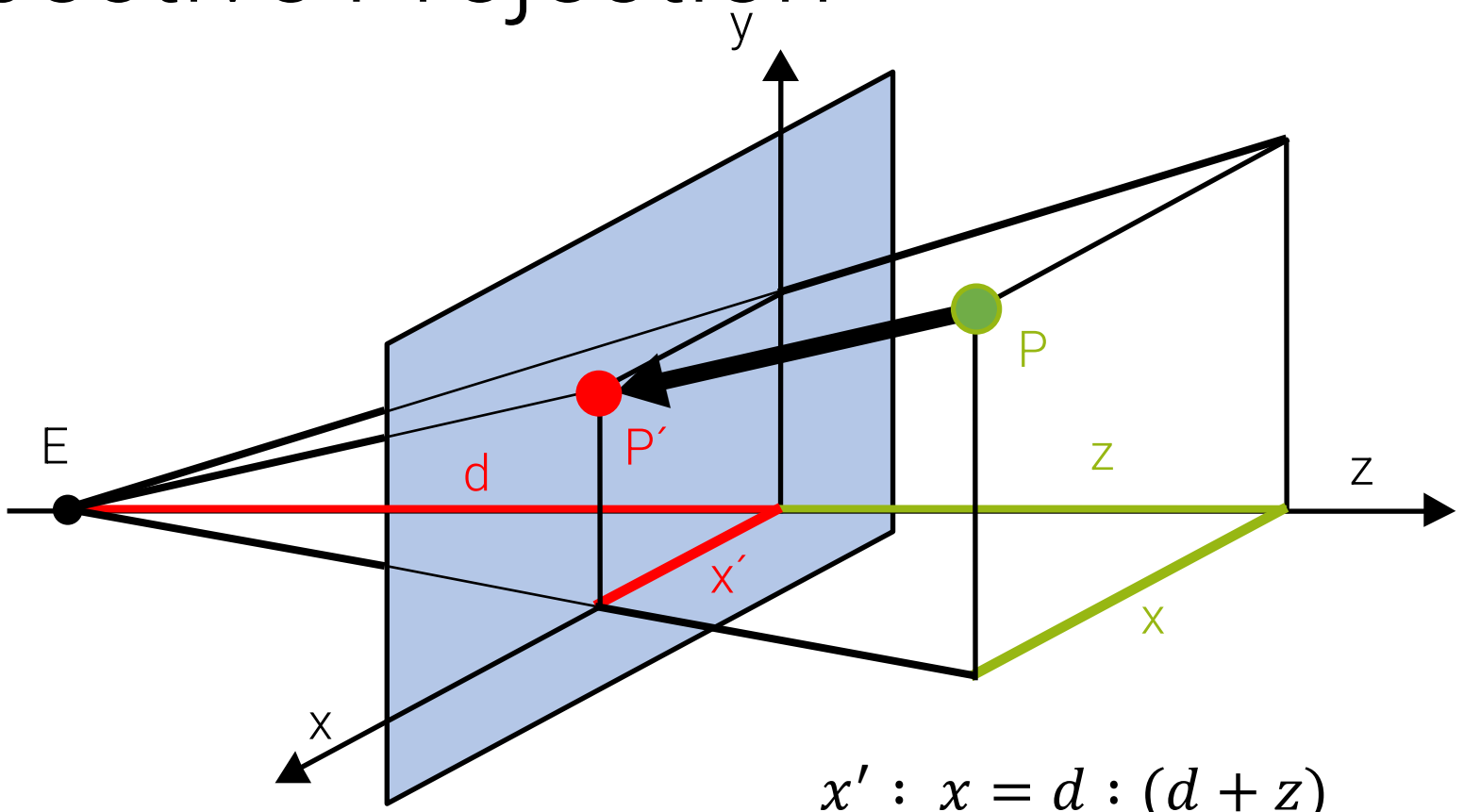
Orthogonal Projection - 2

$$M \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Perspective Projection



$$x' : x = d : (d + z)$$

$$x' = \frac{xd}{z + d} \qquad y' = \frac{yd}{z + d} \qquad z' = 0$$

Perspective Projection 2

$$x' = \frac{xd}{z+d} = \frac{x}{1+(\frac{z}{d})} \qquad y' = \frac{yd}{z+d} = \frac{y}{1+(\frac{z}{d})} \qquad z' = 0$$

$$\begin{bmatrix} x' \\ y' \\ z' = 0 \\ 1 \end{bmatrix} = M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$M_P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1/d & 1 \end{bmatrix}$$

$$M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ \frac{z}{d} + 1 \end{bmatrix} \text{ (homogenous division)} \rightarrow \begin{bmatrix} \frac{x}{1+(\frac{z}{d})} \\ \frac{y}{1+(\frac{z}{d})} \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 0 \\ 1 \end{bmatrix}$$

In practice: Perspective Projection

$$x' = \frac{xd}{z+d} = \frac{x}{1+(\frac{z}{d})} \qquad y' = \frac{yd}{z+d} = \frac{y}{1+(\frac{z}{d})} \qquad z' = 0$$

$$\begin{bmatrix} x' \\ y' \\ z' = 0 \\ 1 \end{bmatrix} = M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$M_P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1/d & 1 \end{bmatrix}$$

different!

$$M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ \frac{z}{d}+1 \end{bmatrix}$$ (homogenous division)→ $$\begin{bmatrix} \frac{x}{1+(\frac{z}{d})} \\ \frac{y}{1+(\frac{z}{d})} \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 0 \\ 1 \end{bmatrix}$$

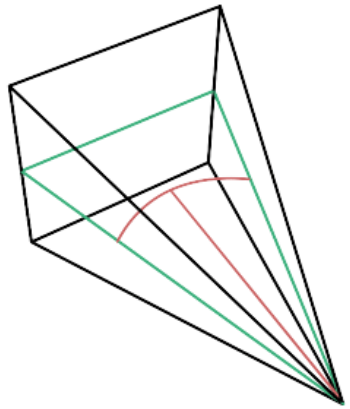
not zero!

In practice: Perspective Projection

$$M_P = \begin{bmatrix} S & 0 & 0 & 0 \\ 0 & S & 0 & 0 \\ 0 & 0 & -\frac{f}{(f-n)} & -1 \\ 0 & 0 & -\frac{f*n}{(f-n)} & 0 \end{bmatrix}$$

$$S = \frac{1}{\tan(\frac{fov}{2} * \frac{\pi}{180})}$$

fov=60 degrees



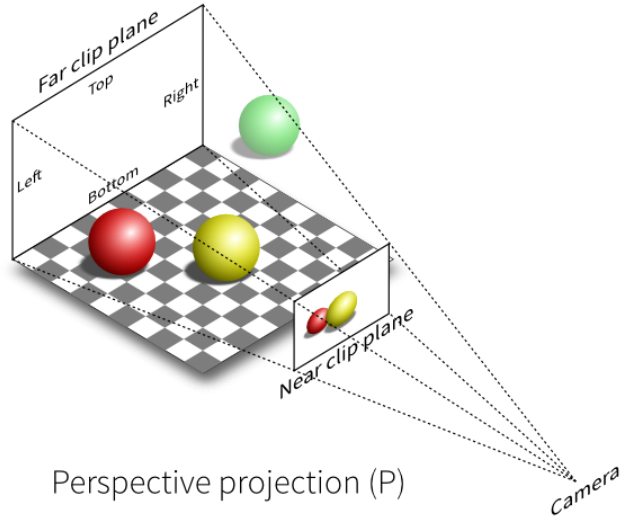
$$M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix}$$ makes sense to keep depth (normalized -1 to 1 -> NDC)

https://www.scratchapixel.com/lessons/3d-basic-rendering/perspective-and-orthographic-projection-matrix/opengl-perspective-projection-matrix

In practice: Perspective Projection

$$M_P = \begin{bmatrix} S & 0 & 0 & 0 \\ 0 & S & 0 & 0 \\ 0 & 0 & -\frac{f}{(f-n)} & -1 \\ 0 & 0 & -\frac{f*n}{(f-n)} & 0 \end{bmatrix}$$

f … far plane (distance)
n … near plane (distance)

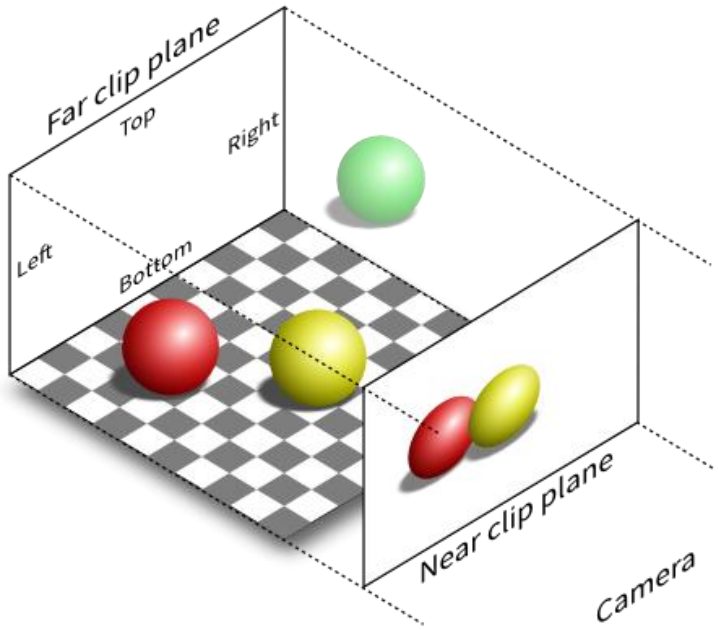


Perspective projection (P)

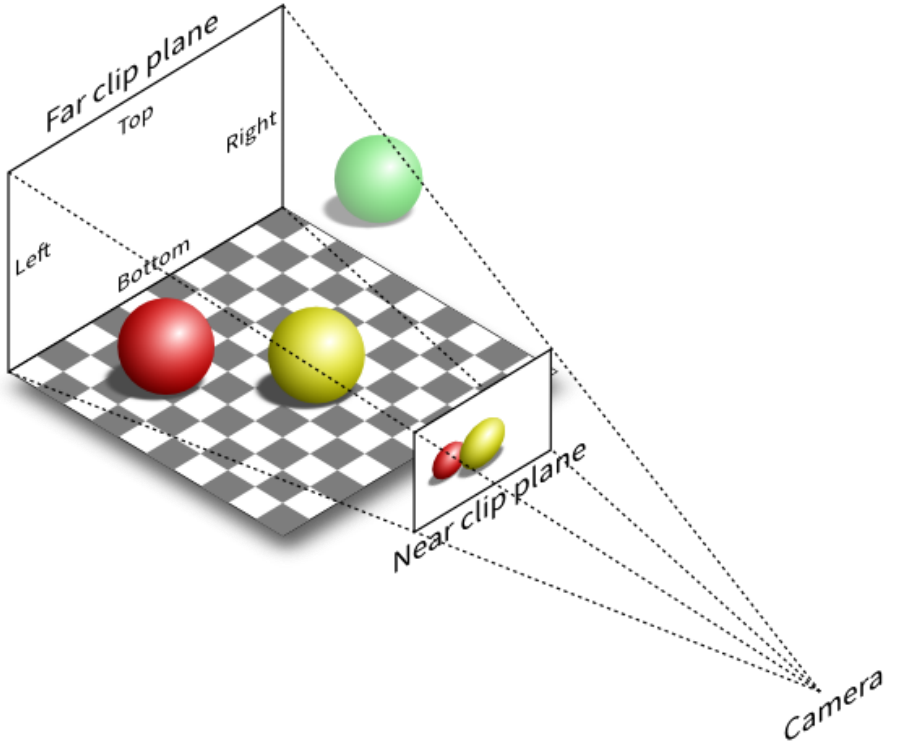
$$M_P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix}$$ makes sense to keep depth (normalized -1 to 1 -> NDC)

View Frustum

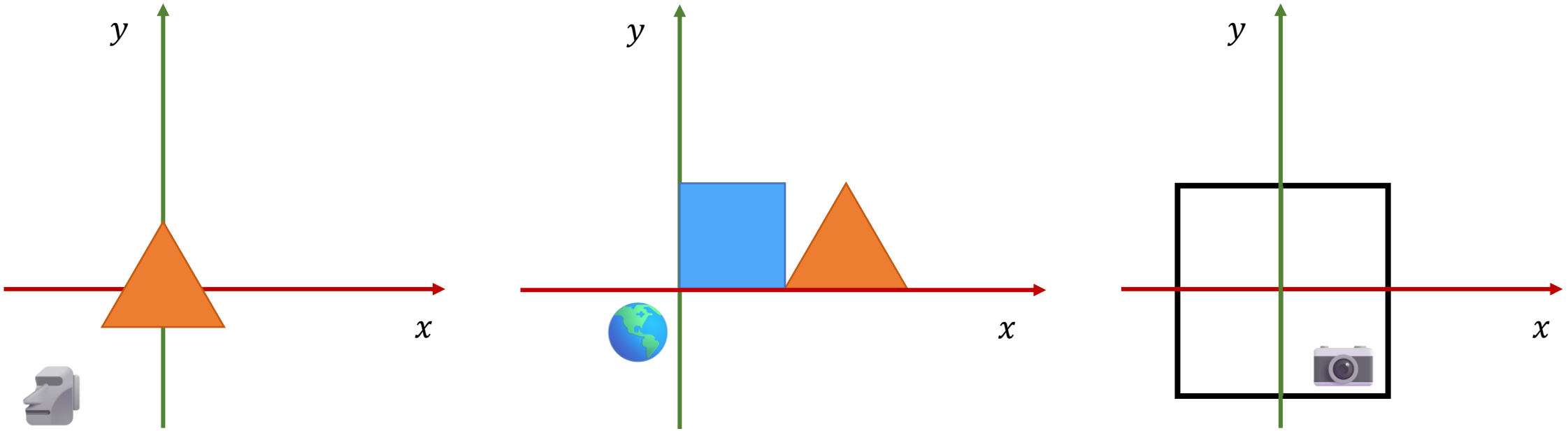
Orthogonal



Perspective



Transformation Matrices (2D)



🗿 ➔ 🌎 ➔ 📷

model-view matrix

Transformation Matrices in 3D

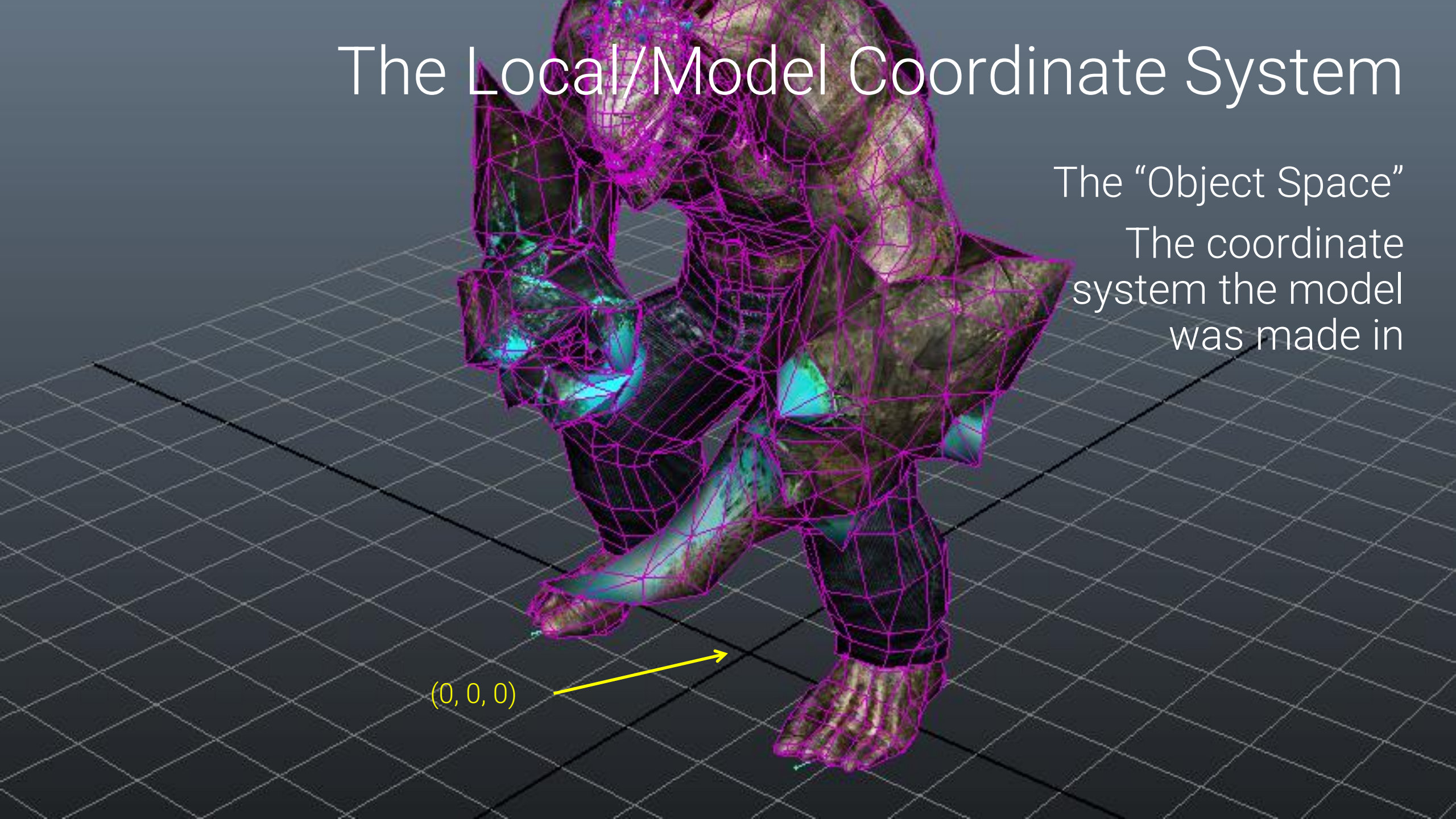


model-view-projection matrix

The Local/Model Coordinate System

The “Object Space”

The coordinate system the model was made in



The World Space

The coordinate system of the virtual environment



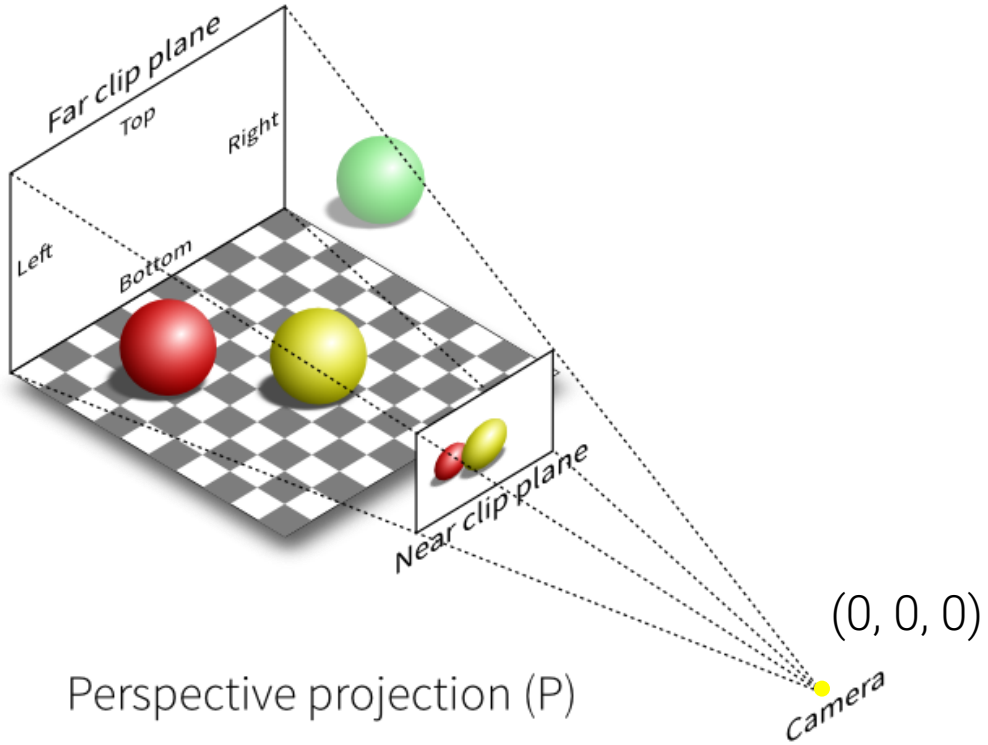
Camera Space

It’s all relative to the camera…

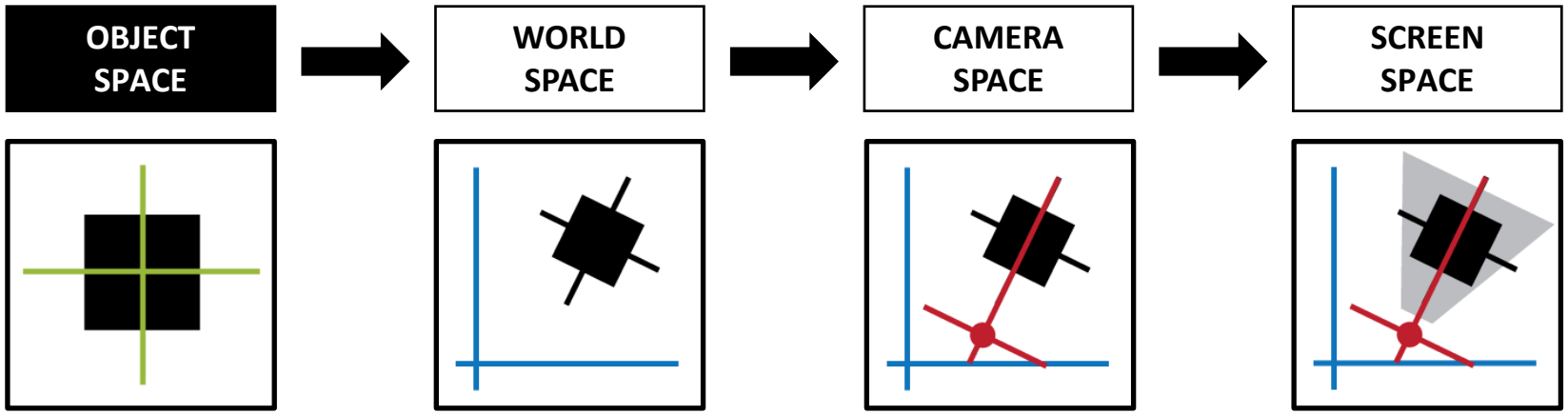


Projection Space

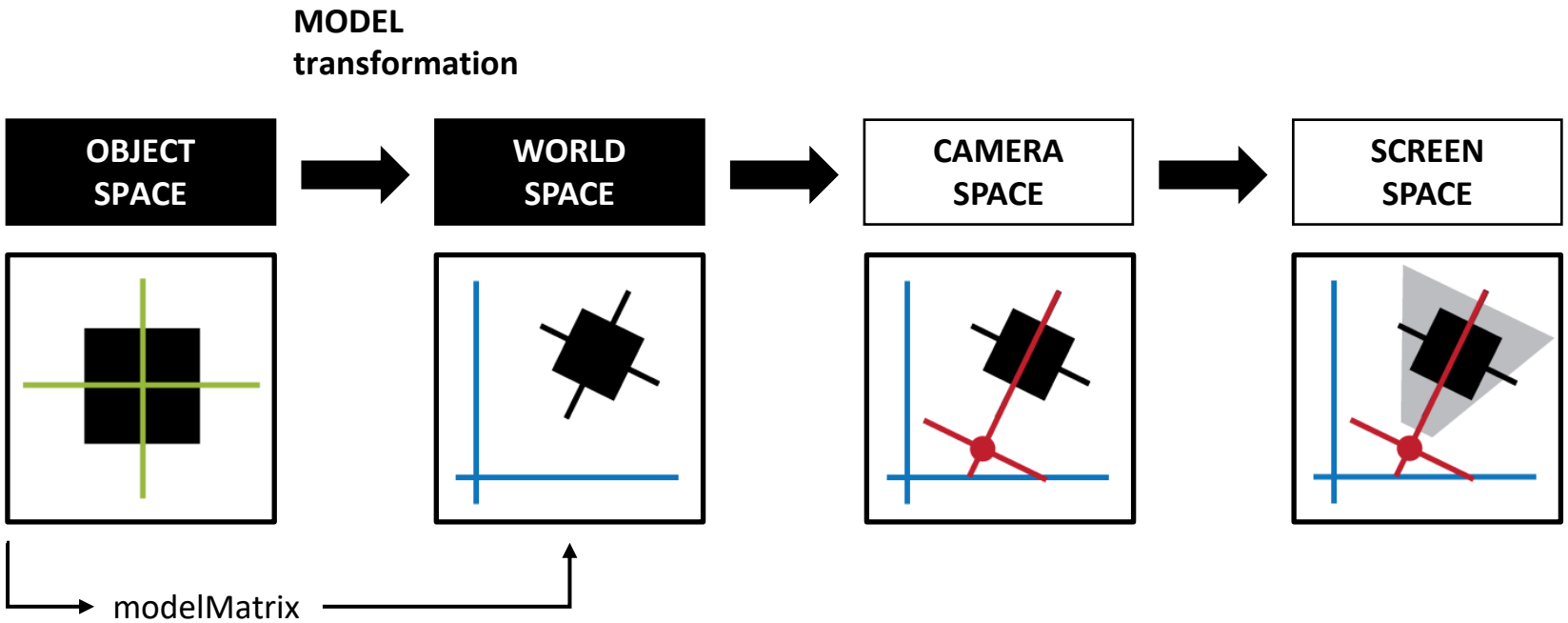
For perspective projection camera is at origin (0,0,0)



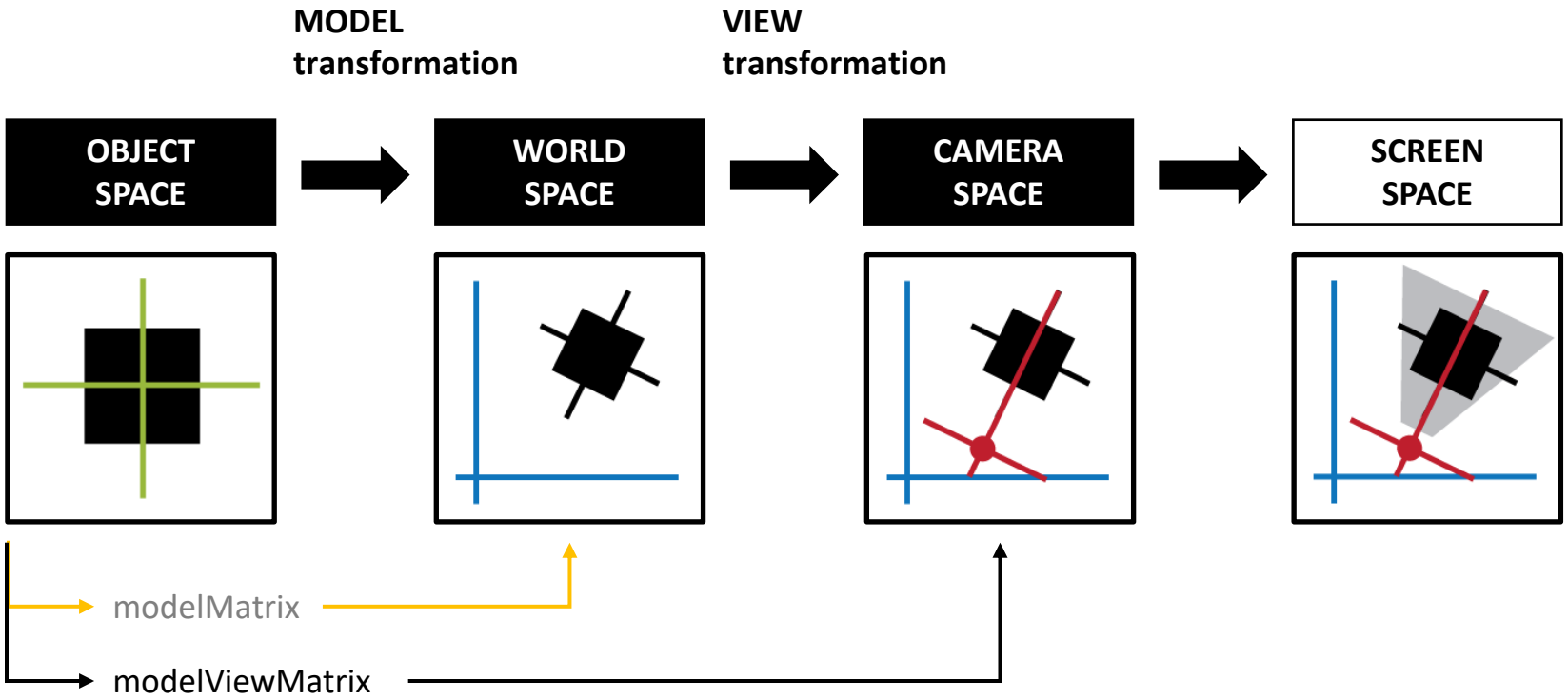
Coordinate systems



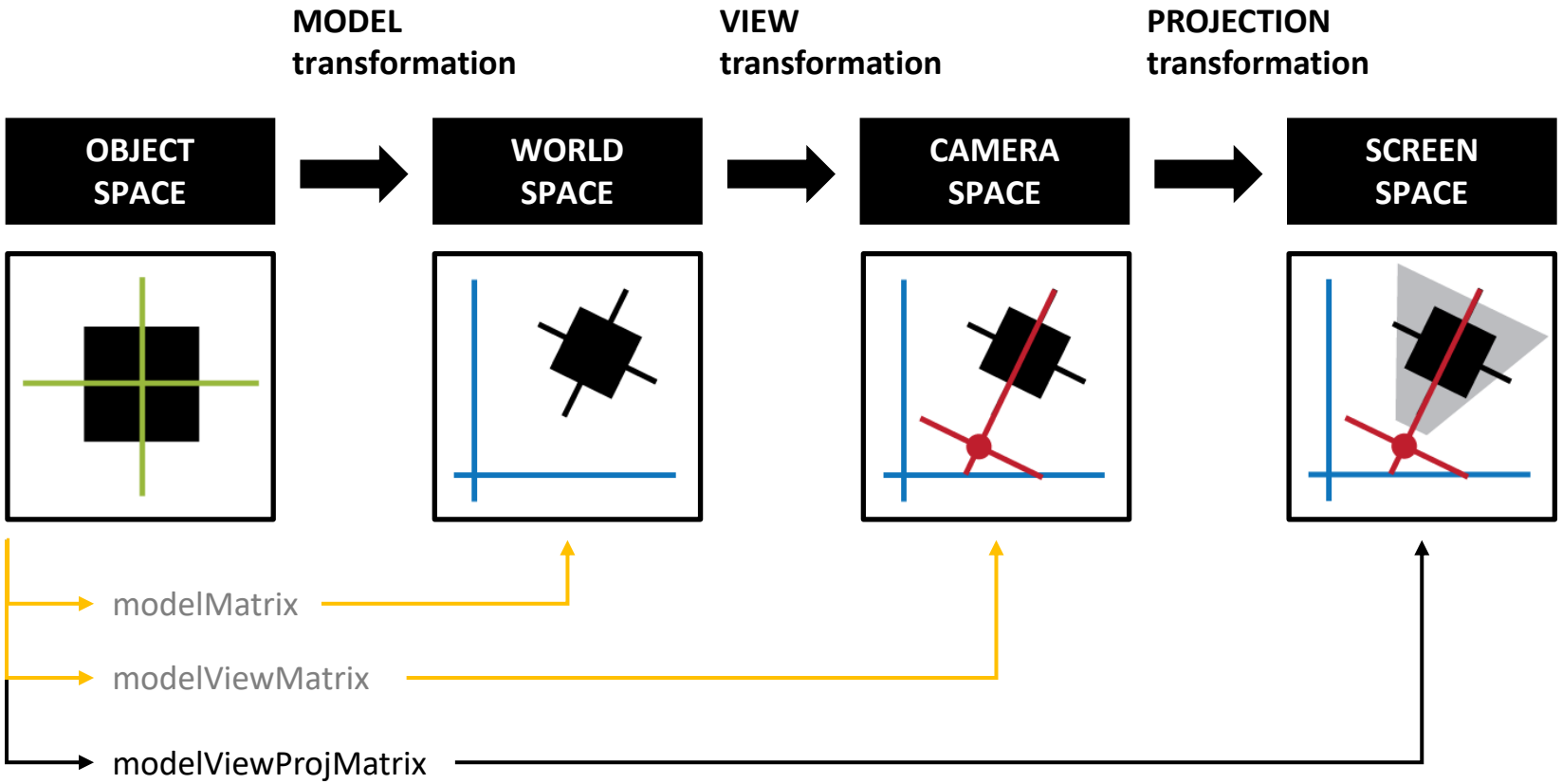
Coordinate systems



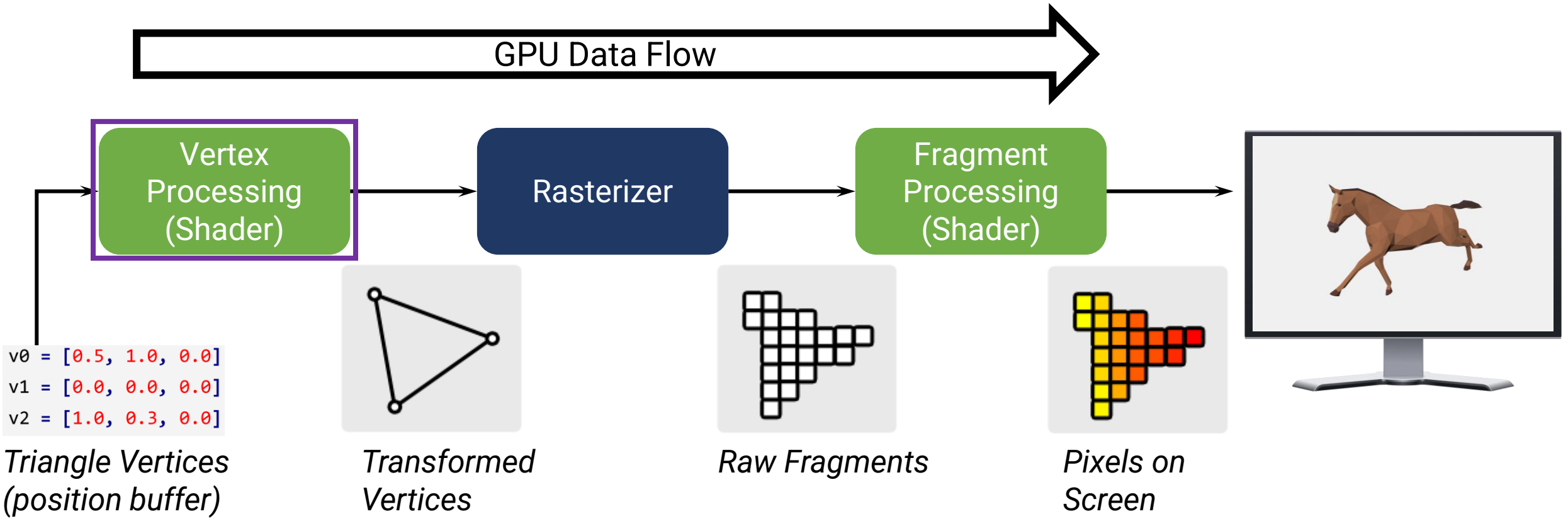
Coordinate systems



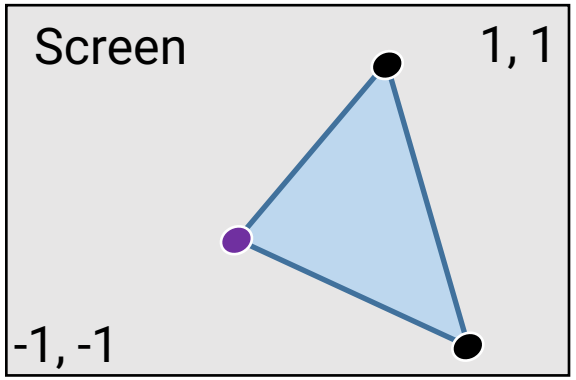
Coordinate systems



Programmable Pipeline



Our Simple Vertex Shader

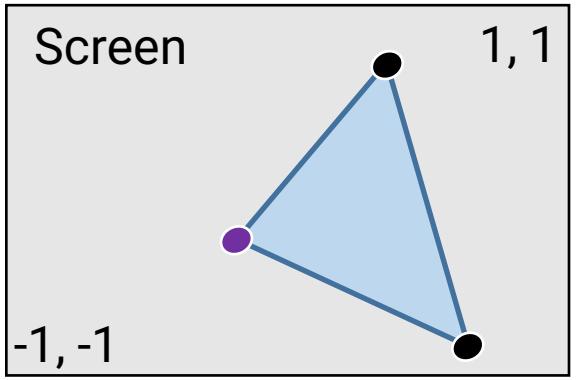


- Processes one vertex (position) of a triangle.
- Computes the *Transformed Vertex* position (`gl_Position`).
  - A position “in view” needs to be in -1 to 1 for x,y,z (yes z too!); called normalized device coordinates (NDC) or clip space

Vertex-Shader **GLSL**

```
#version 300 es
layout(location=0) in vec3 position; // 3D Vector
uniform mat4 modelMatrix; // 4x4 Matrix
void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 ); // compute NDC
}
```

Our Modified Vertex Shader

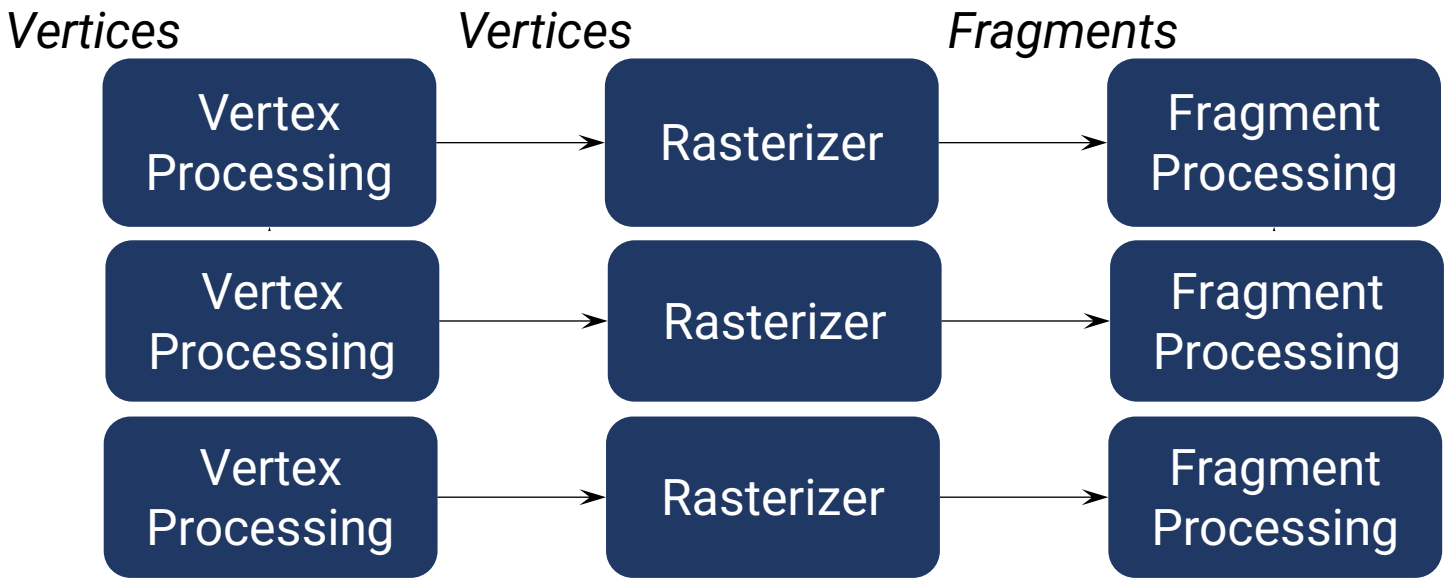


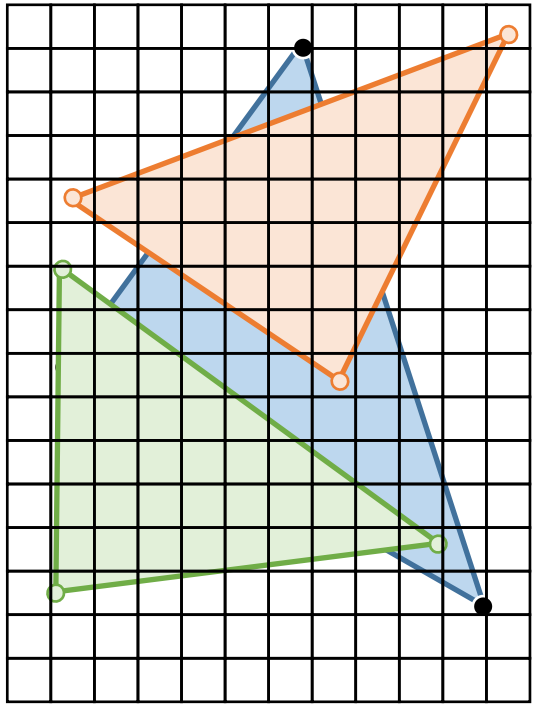
- Processes one vertex (position) of a triangle.
- Computes the *Transformed Vertex* position (`gl_Position`).
  - A position “in view” needs to be in -1 to 1 for x,y,z (yes z too!); called normalized device coordinates (NDC) or clip space

Vertex-Shader **GLSL**

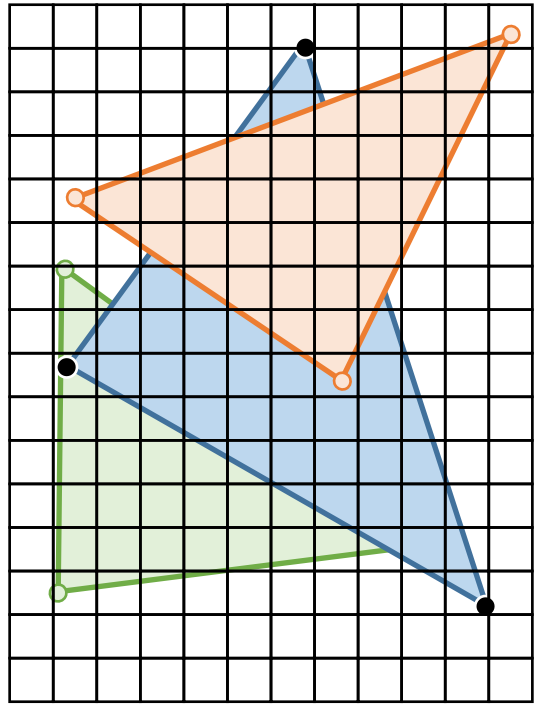
```
#version 300 es
layout(location=0) in vec3 position; // 3D Vector
uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;
void main() {
  // compute NDC
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4( position, 1.0);
}
```

Recap Rasterization





Z-Buffer Algorithm

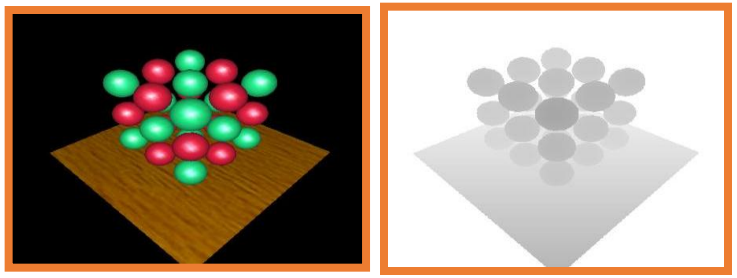


- How do we decide what is in front?

- Sort the polygons/objects in the szene!?
  ➔ Yes, but has problems with complex geometries!

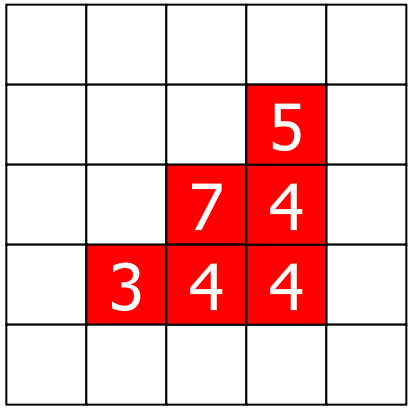
- Consider visibility at the level of each (rasterized) pixel.
  - Needs an additional depth buffer (z-Buffer)
  - Using the normalized z‘ of the projection.
  - Decision if something is in front: $z_{old} < z_{new}$

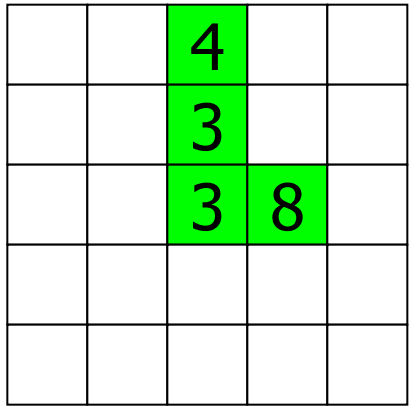
Z-Buffer Algorithm

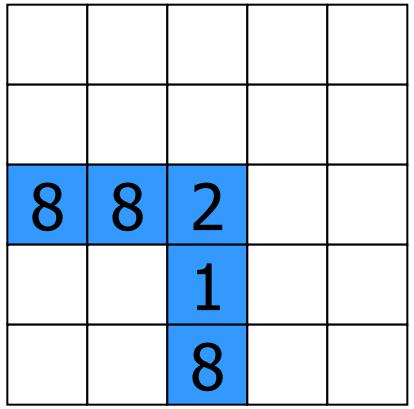


```
for each face F {
  for each pixel (x,y) covering the face {
    if (z in front of depth(x,y)) {
      color (x,y) = actualColor;
      depth (x,y) = z;
    }
    else {
      // do nothing – keep the old color & depth
    }
  }
}
```

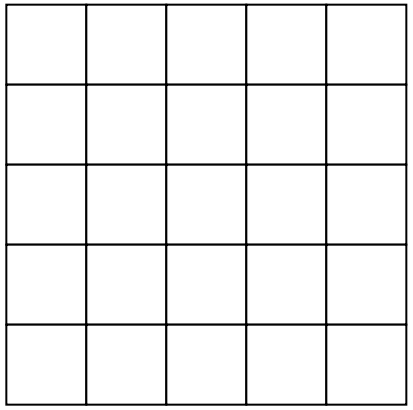
Test it...



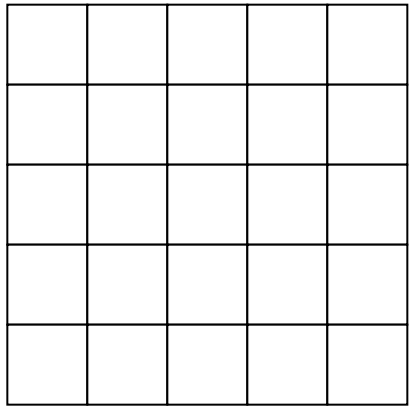




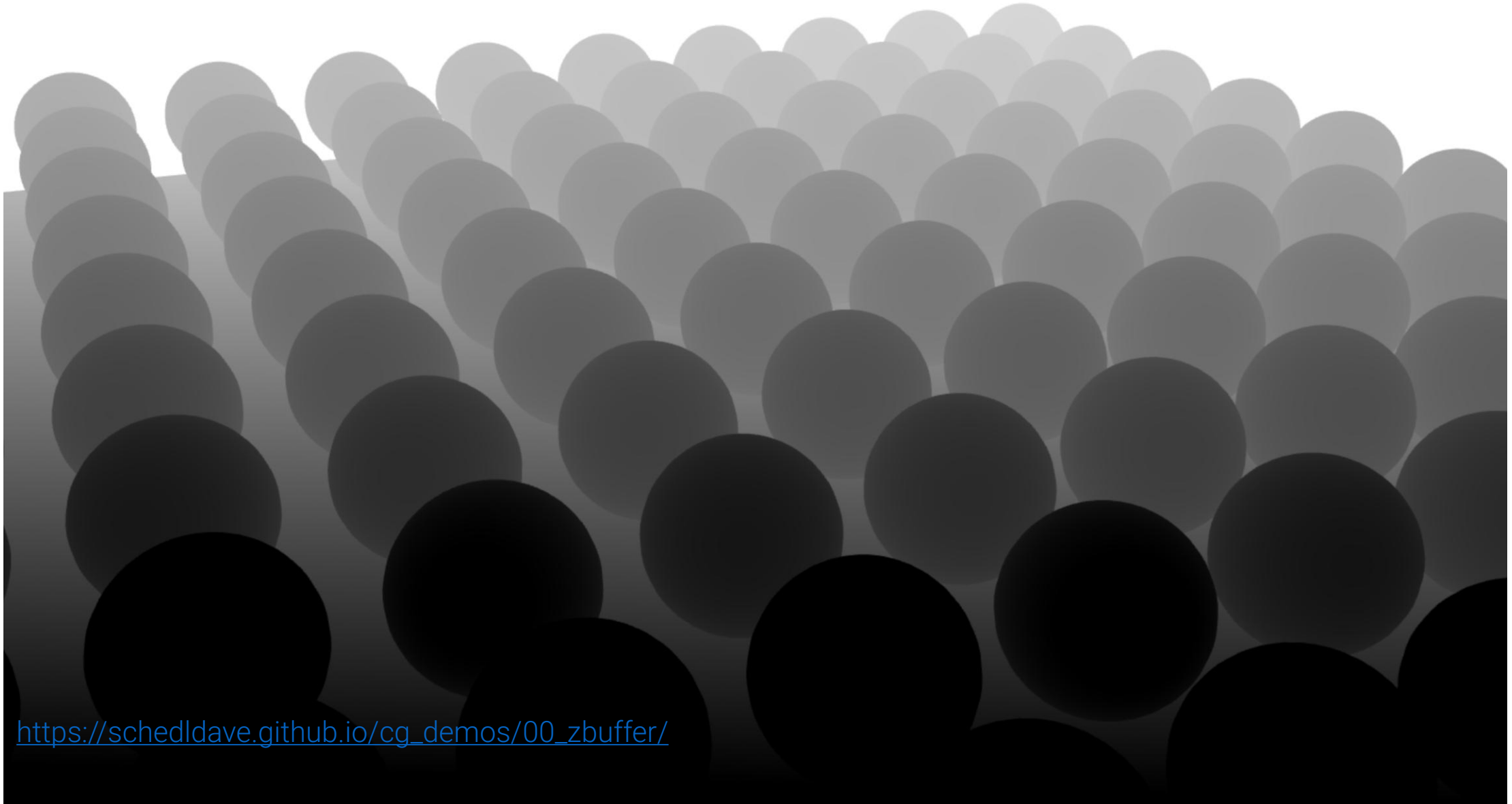
FrameBuffer



z-Buffer



Z-Buffer Demo



https://schedldave.github.io/cg_demos/00_zbuffer/

Depth Buffering Using OpenGL

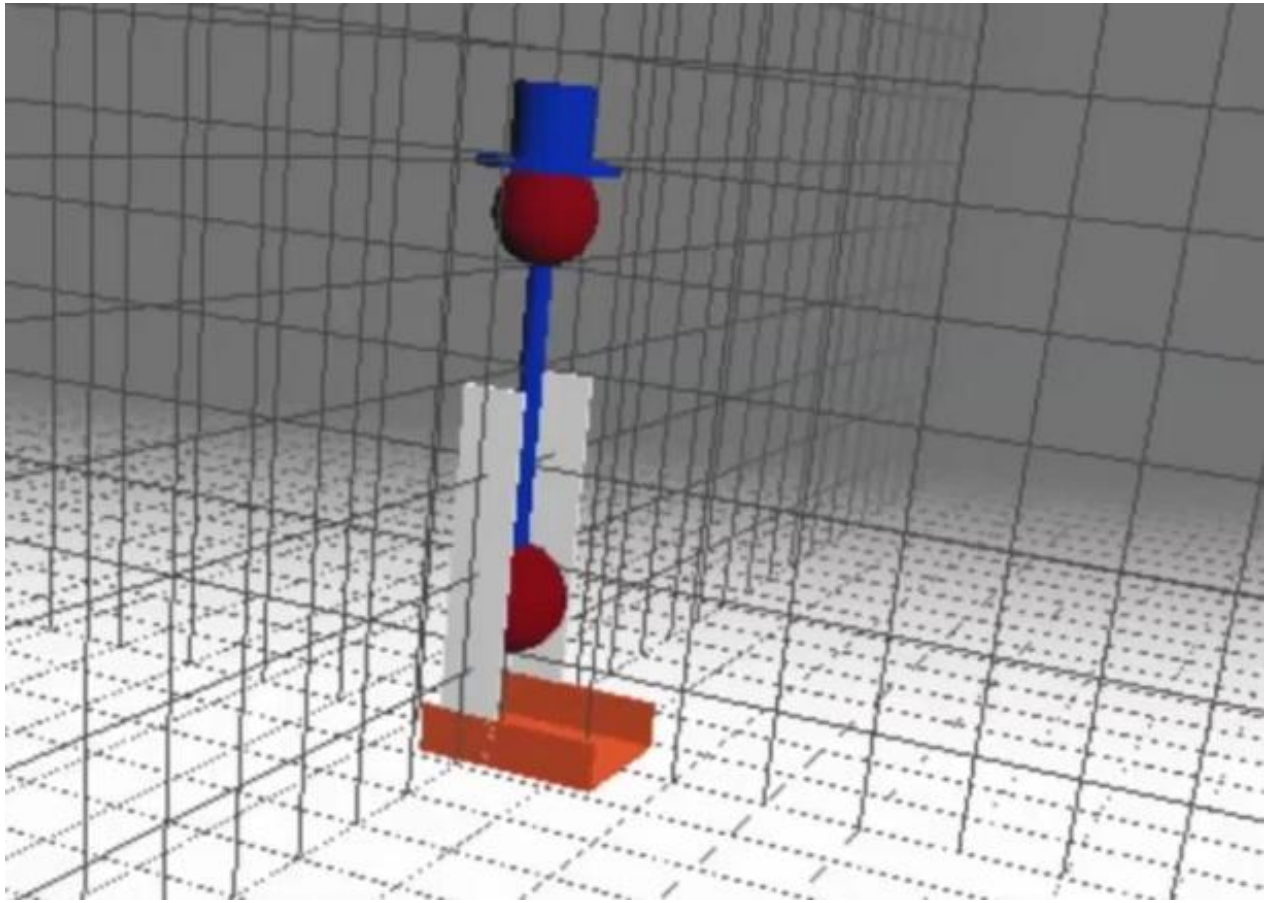
Enable depth buffering:

```
gl.enable( gl.DEPTH_TEST );
```

Before Rendering the scene make sure to
clear color and depth buffers:

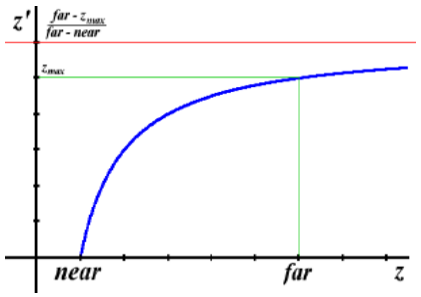
```
gl.clear( gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT );
```

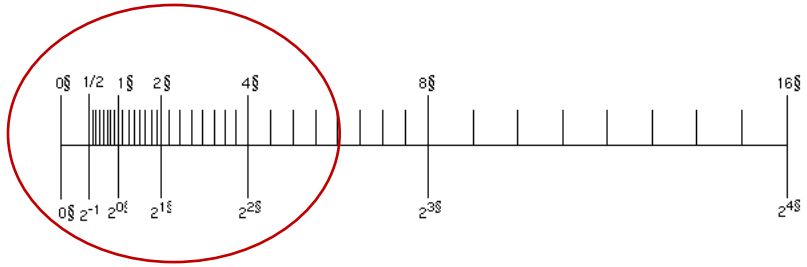
Z-Fighting

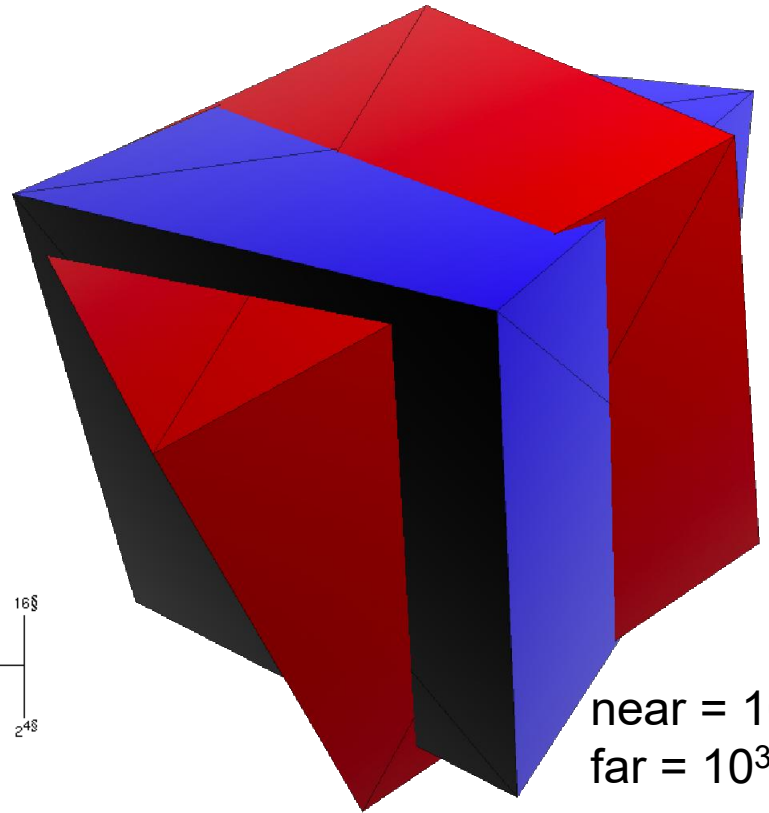


Why near/far plane clipping?

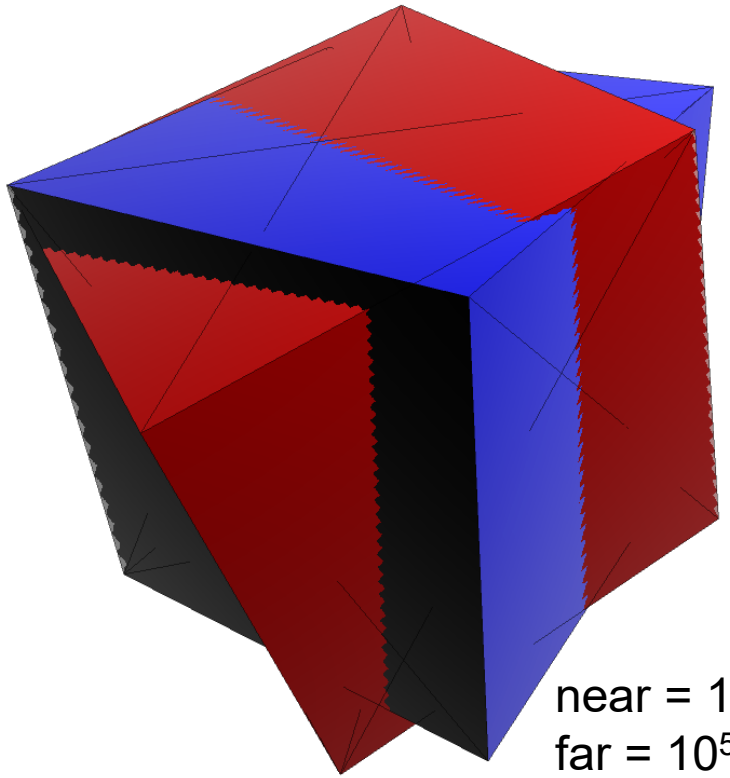
- Checking if fragments are behind camera
- Important for dealing with finite precision of depth buffer
- z’ is nonlinear!







near = 10^{-1}
far = 10^{3}



near = 10^{-5}
far = 10^{5}

Questions?



david.schedl@fh-hagenberg.at